AINOW EBOOK 2026

클로드 코드 실전 마스터 가이드

창시자 Boris Cherny의 워크플로우부터
해커톤 우승자의 비밀 무기까지
왕초보를 위한 완전 실전 가이드

- Boris의 42가지 팁
- 해커톤 우승 노하우
- GitHub 활용법
- 업종별 실전 프로젝트

에이나우 AI 교육 플랫폼 | ainow.co.kr

신영선 저 | 2026년 3월

목차

PART 0 — 시작하기 전에

왜 이 책을 만들었는가

Claude Code란 무엇인가

PART 1 — 10분 안에 시작하기

설치부터 첫 대화까지 3분 퀵스타트

왕초보가 가장 많이 하는 실수 5가지

첫 1주일 로드맵

PART 2 — 창시자 Boris Cherny의 진짜 사용법

100줄 CLAUDE.md의 비밀

Boris의 황금 규칙: 자기학습 루프

왕초보를 위한 CLAUDE.md 단계별 성장

하루 100개 PR — 병렬 작업 시스템

Boris의 42가지 팁 중 Top 15

PART 3 — 해커톤 우승자들의 비밀 무기

Everything Claude Code 리포 해부

비개발자가 해커톤에서 우승한 실화

AgentShield — 보안 점검 1분 가이드

PART 4 — GitHub로 10배 빠르게 성장하기

커뮤니티 스킬 1,234개 활용 가이드

플러그인 시스템 완전 정복

GitHub 왕초보 가이드

추천 GitHub 리포 Top 10

PART 5 — 업종별 실전 프로젝트

마케터: 랜딩페이지 + 퍼널 자동화

크리에이터: 블로그 자동화 시스템

병원 마케팅: 블로그 SEO 자동화

사업가: MVP 3일 만에 만들기

PART 6 — 프로 레벨 세팅

토큰 비용 50% 절감 고급 세팅

Agent Teams — AI 팀 운영법

자동화 끝판왕 조합

부록

A: Boris 42가지 팁 요약

B: 업종별 CLAUDE.md 템플릿 5종

C: 추천 GitHub 리포 20선

CHAPTER 00

왜 이 책을 만들었는가

안녕하세요, 에이나우의 신영선입니다.

저는 20인 규모의 디자인 에이전시 알잘스튜디오와 1만 명 규모의 AI 커뮤니티 에이나우를 운영하면서, 매일 Claude Code를 실전에서 사용합니다. 마케팅 자동화, 콘텐츠 제작, 서비스 개발, 병원 블로그 SEO까지 — Claude Code 없이는 하루도 돌아가지 않는 시스템을 구축했습니다.

이 책을 만든 이유는 단순합니다. **기존에 나와 있는 Claude Code 가이드북으로는 부족했기 때문입니다.**

기존 가이드의 한계

시중의 Claude Code 가이드북들은 대부분 공식 문서를 풀어쓴 수준에 머무릅니다. 설치법, 기본 명령어, 간단한 프로젝트 예시. 시작에는 도움이 되지만, 실전에서 마주치는 진짜 질문에는 답이 없습니다.

- "CLAUDE.md를 어떻게 써야 Claude가 진짜 똑똑해지지?"
- "해커톤에서 8시간 만에 서비스를 만든 사람들은 대체 뭘 다르게 한 거야?"
- "GitHub에 좋은 자료가 많다던데, 코딩 모르는 내가 어떻게 활용해?"
- "토큰 비용이 너무 나오는데, 전문가들은 어떻게 관리하지?"
- "마케터/크리에이터/사업가인 나에게 맞는 실전 프로젝트는 뭐야?"

이 질문들에 답하는 책이 없었습니다. 그래서 직접 만들기로 했습니다.

이 책의 소스가 다릅니다

이 책은 세 가지 특별한 소스를 기반으로 합니다.

1. **창시자의 워크플로우**
   Claude Code를 만든 Boris Cherny(Anthropic Staff Engineer)가 2026년 1~2월에 공개한 42가지 팁과 실제 CLAUDE.md 파일을 한국어로 최초 체계화했습니다.
2. **해커톤 우승자의 노하우**
   Anthropic 공식 해커톤에서 $100K API 크레딧을 수상한 팀들의 설정 파일, 워크플로우, GitHub 리포를 분석했습니다. 50K+ 스타의 Everything Claude Code를 왕초보가 바로 적용할 수 있게 풀어냈습니다.
3. **글로벌 커뮤니티 집약**
   1,234개 이상의 커뮤니티 스킬, 플러그인 생태계, 실전 사용자들의 베스트 프랙티스를 업종별로 정리했습니다.

이 책의 구성

총 6개 파트, 18개 챕터로 이루어져 있습니다. 앞에서부터 읽어도 좋고, 필요한 부분만 골라 읽어도 됩니다.

파트	내용	추천 독자
Part 0	이 책의 배경과 철학	모든 독자
Part 1	설치 ~ 첫 대화, 왕초보 실수	처음 시작하는 분
Part 2	Boris Cherny 워크플로우	모든 독자 (핵심!)
Part 3	해커톤 우승자 노하우	실전 적용하고 싶은 분
Part 4	GitHub 오픈소스 활용	성장을 가속하고 싶은 분
Part 5	업종별 실전 프로젝트	바로 만들고 싶은 분
Part 6	프로 레벨 세팅	비용 최적화/고급 자동화

AI 생산자가 되는 길

에이나우가 말하는 'AI 생산자'란 무엇인가요? AI를 소비하는 사람(챗봇에 질문하고 답을 받는 수준)이 아니라, **AI로 가치를 만들어내는 사람**입니다.

서비스를 만들고, 콘텐츠를 생산하고, 업무를 자동화하고, 수익을 만드는 사람. Claude Code는 이 과정에서 가장 강력한 도구입니다.

이 책의 핵심 메시지

AI 시대의 경쟁력은 코딩 실력이 아니라 **'자기 분야의 문제를 정확히 아는 것'**입니다. 변호사가 500명의 개발자를 이기고 해커톤 1등을 한 이유가 바로 여기에 있습니다. Claude Code는 여러분의 전문성을 10배로 증폭시키는 도구입니다.

함께 가요

이 책은 혼자 읽는 책이 아닙니다. 에이나우 커뮤니티에서 함께 실습하고, 질문하고, 서로의 프로젝트를 공유하며 성장할 수 있습니다.

- **에이나우 디스코드**: #클로드코드-스터디 채널에서 실시간 Q&A
- **주간 챌린지**: 매주 Boris 팁 하나를 따라하고 결과 공유
- **프로젝트 쇼케이스**: 독자들의 실전 프로젝트 소개

자, 그럼 시작해볼까요?

CHAPTER 00

Claude Code란 무엇인가

한 마디로

Claude Code는 Anthropic이 만든 **AI 코딩 도구**입니다. 터미널(컴퓨터에 글로 명령하는 창)에서 한국어로 대화하면, Claude가 직접 코드를 쓰고, 파일을 만들고, 테스트를 돌리고, 에러를 고칩니다.

> *옆에 앉아서 직접 키보드를 쳐주는 전문가. 그게 Claude Code입니다.*

다른 AI 도구와 뭐가 다른가요?

도구	역할	비유
ChatGPT / Claude.ai	질문하면 답변을 줌	상담사 (말만 해줌)
Cursor / Windsurf	코드 편집기 안에서 AI가 도움	조수가 옆에 앉아 조언
Claude Code	직접 파일을 만들고 수정하고 실행	**직접 일하는 팀원**

가장 큰 차이는 **실행력**입니다. ChatGPT에게 "웹사이트 만들어줘"라고 하면 코드를 보여줍니다. Claude Code에게 같은 말을 하면 **실제로 파일을 만들고, 브라우저에서 볼 수 있는 웹사이트를 완성합니다.**

누가 쓸 수 있나요?

코딩을 전혀 모르는 사람도 쓸 수 있습니다. 실제로 2026년 Anthropic 해커톤에서 1등을 한 사람은 변호사였고, 코드를 한 줄도 직접 쓰지 않았습니다.

다만, 여러분의 전문 분야가 있어야 합니다. Claude Code는 **만능이 아니라 증폭기**입니다. 마케팅을 아는 사람이 쓰면 마케팅 도구를, 디자인을 아는 사람이 쓰면 디자인 도구를, 의료를 아는 사람이 쓰면 의료 도구를 만들 수 있습니다.

비용은 얼마인가요?

플랜	월 비용	특징	추천 대상
Pro	$20 (약 2.7만원)	기본 사용량 포함	시작하는 분
Max (5x)	$100 (약 13만원)	Pro의 5배 사용량	매일 쓰는 분
Max (20x)	$200 (약 26만원)	Pro의 20배 사용량	헤비 사용자
API	사용한 만큼	토큰 단위 과금	개발자/자동화

> 💡 **꿀팁**
> 처음 시작한다면 **Pro ($20/월)**로 충분합니다. 2~3주 써보고 부족하면 업그레이드하세요. Part 6에서 비용을 50% 줄이는 고급 세팅도 알려드립니다.

PART 1

10분 안에 시작하기

설치부터 첫 대화까지 가장 빠른 길.
그리고 왕초보가 반드시 피해야 할 함정들.

CHAPTER 01

설치부터 첫 대화까지 3분 퀵스타트

사전 준비: 딱 2가지

Claude Code를 실행하려면 **Node.js**와 **Claude 계정**이 필요합니다. 그게 전부입니다.

Step 1. Node.js 설치

Node.js는 Claude Code가 돌아가는 엔진입니다. 자동차의 가솔린이라고 생각하세요.

```
# Mac 사용자
# 1. https://nodejs.org 접속
# 2. LTS 버전 다운로드 (초록색 버튼)
# 3. 설치 파일 실행 → "다음" 계속 클릭

# 설치 확인 (터미널에서)
node --version   # v18 이상이면 OK
npm --version    # 숫자가 나오면 OK
```

```
# Windows 사용자
# 1. https://nodejs.org 접속
# 2. LTS 버전 다운로드
# 3. 설치 시 "Add to PATH" 옵션 반드시 체크!
# 4. 설치 후 PowerShell 또는 CMD 열기

node --version
npm --version
```

> ℹ 알아두기
>
> **터미널이 뭔가요?** 컴퓨터에게 글로 명령하는 창입니다. Mac은 '터미널' 앱을 검색하면 나오고, Windows는 'PowerShell' 또는 'CMD'를 검색하면 됩니다. 카카오톡 채팅방처럼 생겼는데, 상대방이 컴퓨터입니다.

Step 2. Claude Code 설치

```
# 터미널에서 이 한 줄만 실행하세요
npm install -g @anthropic-ai/claude-code

# 설치가 끝나면 (1~2분 소요)
claude
```

처음 `claude`를 실행하면 브라우저가 자동으로 열리면서 로그인을 요청합니다. Claude 계정(claude.ai에서 만든 것)으로 로그인하면 연결 완료!

Step 3. 첫 대화 — 프로젝트 폴더에서 시작하기

Claude Code는 **현재 폴더를 작업 공간으로 사용**합니다. 먼저 프로젝트 폴더를 만들고 그 안에서 시작하세요.

```
# 프로젝트 폴더 만들기
mkdir my-first-project
cd my-first-project

# Claude Code 시작
claude

Claude Code v2.x.x
Opus 4.6

> 안녕! 나는 코딩을 전혀 몰라.
  간단한 자기소개 웹페이지를 하나 만들어줘.
  이름은 "홍길동", 직업은 "마케터"로 해줘.
```

Claude가 파일을 만들기 전에 허락을 구합니다. **"y"를 누르면** 실제로 파일이 생성됩니다.

```
Claude: 자기소개 웹페이지를 만들어드리겠습니다.
      다음 파일을 생성할까요?

  Create: index.html

y/n: y

✓ index.html 생성 완료
  브라우저에서 확인하려면 파일을 더블클릭하세요.
```

Step 4. 결과 확인

프로젝트 폴더(my-first-project)에 가면 `index.html` 파일이 생겨 있습니다. 이 파일을 더블클릭하면 브라우저에서 웹페이지가 열립니다. 축하합니다! 코드 한 줄 안 쓰고 웹페이지를 만들었습니다.

핵심 개념 3초 정리

용어	한 줄 설명	비유
터미널	컴퓨터에게 글로 명령하는 창	카톡 채팅방 (상대=컴퓨터)
프롬프트	Claude에게 하는 말/요청	주문서
컨텍스트	Claude가 기억하는 대화 범위	책상 위 메모 (크기 제한 있음)
토큰	Claude가 텍스트를 읽는 단위	영어 4글자 ≈ 1토큰, 한글 1자 ≈ 1~2토큰
CLAUDE.md	Claude에게 주는 규칙서	직원 매뉴얼
MCP	외부 서비스와 연결하는 통로	전화기 (Notion, GitHub 등에 전화)
스킬	미리 만든 작업 레시피	자판기 버튼 (/blog, /fix 등)
Hooks	자동 반응 규칙	센서등 (조건 충족 시 자동 실행)

> 💡 **꿀팁**
>
> **Windows 사용자 필수 팁:** PowerShell 대신 **Git Bash**나 **WSL**을 강력 추천합니다. 한국어 경로에서 인코딩 문제가 자주 생기므로, 프로젝트 폴더는 영어 이름(예: C:/projects/)을 사용하세요.

Mac vs Windows vs Linux 설치 차이점

항목	Mac	Windows	Linux
터미널	터미널 앱 (기본)	PowerShell 또는 Git Bash	터미널 (기본)
Node.js	nodejs.org에서 설치	nodejs.org + PATH 체크	apt/nvm으로 설치
한국어 경로	대부분 문제 없음	⚠ 문제 발생 가능	대부분 문제 없음
추천 터미널	iTerm2	Windows Terminal + Git Bash	기본 터미널
알림 설정	osascript (기본)	PowerShell 토스트	notify-send

설치 트러블슈팅

"command not found: node"가 나와요

Node.js가 제대로 설치되지 않았거나 PATH에 추가되지 않은 경우입니다.

```
# Mac: Homebrew로 재설치
brew install node

# Windows: 설치 프로그램을 다시 실행하고
# "Add to PATH" 옵션을 반드시 체크

# 또는 nvm(Node Version Manager) 사용 (추천)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
nvm install --lts
```

"permission denied"가 나와요

```
# Mac/Linux: sudo를 앞에 붙이세요
sudo npm install -g @anthropic-ai/claude-code

# 또는 npm 글로벌 경로를 바꾸는 게 더 좋은 방법
mkdir ~/.npm-global
npm config set prefix '~/.npm-global'
# ~/.zshrc 또는 ~/.bashrc에 추가:
# export PATH=~/.npm-global/bin:$PATH
```

"ECONNREFUSED" 또는 네트워크 에러

방화벽이나 VPN이 npm 패키지 다운로드를 차단하는 경우입니다. 회사 네트워크에서 자주 발생합니다.

```
# npm 레지스트리를 직접 지정
npm install -g @anthropic-ai/claude-code --registry https://registry.npmjs.org/
```

> ⚠ **주의**
> 회사 컴퓨터에서 설치가 안 되면, IT팀에 npm 패키지 설치 허용을 요청하세요. 또는 개인 컴퓨터에서 먼저 익힌 후, 회사에 도입을 제안하는 전략이 효과적입니다.

왕초보가 가장 많이 하는 실수 5가지

에이나우 커뮤니티에서 수백 명의 왕초보를 관찰하며 정리한 **가장 흔한 실수 패턴 5가지**입니다. 이것만 피해도 시행착오가 절반으로 줄어듭니다.

실수 #1: 한 번에 너무 큰 걸 시킨다

> *"쇼핑몰 전체를 만들어줘" — 이 한 마디로 시작하면 Claude는 자기 나름대로 전체를 만들려다가 중간에 길을 잃습니다.*

왜 문제인가? Claude의 컨텍스트 윈도우(기억력)는 유한합니다. 약 200,000 토큰인데, 큰 프로젝트를 한 번에 만들려 하면 중간에 앞에서 결정한 것을 잊어버립니다.

✕ 나쁜 예시

```
> 쇼핑몰 웹사이트를 만들어줘. 회원가입, 로그인,
  상품 목록, 장바구니, 결제, 관리자 페이지,
  리뷰 시스템, 검색, 필터링 전부 넣어줘.
```

✓ 좋은 예시

```
# 세션 1
> 쇼핑몰 메인 페이지 레이아웃을 먼저 만들어줘.
  헤더에 로고와 네비게이션, 본문에 상품 카드 4개.

# 세션 2
> 상품 상세 페이지를 추가해줘. 이미지, 가격, 설명, 구매 버튼.

# 세션 3
> 장바구니 기능을 넣어줘. 상품을 담고 수량 변경 가능하게.
```

핵심 원칙: 한 입 크기로 쪼개기

하나의 프롬프트에 하나의 작업. 작업이 끝나면 결과를 확인하고, 다음 작업을 지시합니다. 이 습관 하나가 결과 품질을 3배 이상 올립니다.

실수 #2: CLAUDE.md를 안 만든다

CLAUDE.md 없이 Claude를 쓰는 건, **직원에게 회사 규칙도 안 알려주고 일시키는 것**과 같습니다. 매번 "한국어로 해줘", "해요체로 써줘", "코드 수정 전에 설명해줘"를 반복하게 됩니다.

꿀팁

지금 당장 프로젝트 폴더에 CLAUDE.md 파일을 만들고, 딱 5줄만 넣으세요.

```
# CLAUDE.md (최소 버전)
- 한국어로 대화
- 해요체 사용
- 코드 수정 전에 먼저 설명
- 비유를 포함해서 설명
- 파일 변경 후 테스트 실행
```

이 5줄만으로도 Claude의 행동이 눈에 띄게 달라집니다. Part 2에서 Boris Cherny의 100줄 CLAUDE.md 철학을 자세히 배웁니다.

실수 #3: 에러가 나면 당황한다

빨간 글씨(에러 메시지)는 **실패가 아니라 피드백**입니다. 프로그래밍에서 에러는 일상이에요. 숙련된 개발자도 매일 에러를 만나고 고칩니다.

Claude Code의 가장 강력한 능력 중 하나가 **에러 자동 수정**입니다.

```
# 에러가 나면 이렇게만 말하세요
> 방금 에러 고쳐줘.
  고친 다음에 다시 실행해서 확인해줘.
  뭘 고쳤는지 한 줄로 설명해줘.
```

이 세 줄이면 됩니다. Claude가 에러를 읽고, 원인을 파악하고, 수정하고, 테스트까지 돌려줍니다.

실수 #4: 허락 요청을 무시하고 넘긴다

Claude가 "이 파일을 수정해도 될까요?"라고 물을 때, **뭘 하려는지 읽고** 허락하세요. 처음에는 귀찮지만, 이게 Claude가 어떻게 일하는지 배우는 가장 빠른 방법입니다.

알아두기

팁: 몇 주 사용해서 Claude의 행동이 익숙해지면, `claude --dangerously-skip-permissions` 로 자동 허락 모드를 쓸 수 있습니다. 하지만 **초보일 때는 하나하나 확인하는 게 학습에 도움**이 됩니다.

실수 #5: 컨텍스트를 관리하지 않는다

대화가 길어지면 Claude가 초반에 한 말을 잊기 시작합니다. 이건 버그가 아니라 컨텍스트 윈도우의 물리적 한계입니다.

명령어	언제 쓰나	효과
`/clear`	주제가 완전히 바뀔 때	대화 전체 삭제, 새로 시작
`/compact`	대화가 길어졌지만 맥락 유지 필요	오래된 내용 압축, 핵심만 유지
`/compact [포커스]`	특정 맥락만 보존하고 싶을 때	지정한 주제만 남기고 나머지 압축

주의

가장 흔한 증상: Claude가 '아까 그 함수'를 못 알아듣거나, 이미 수정한 코드를 또 수정하려 할 때. 이게 컨텍스트 오버플로우의 신호입니다. `/compact` 또는 `/clear` 를 쓰세요.

첫 1주일 로드맵

Claude Code를 처음 쓰는 분을 위한 단계별 가이드입니다.

1. **Day 1-2: 설치 + 간단한 대화**
   설치를 완료하고, 간단한 HTML 페이지를 만들어보세요. '자기소개 페이지 만들어줘', '배경색을 파란색으로 바꿔줘' 같은 간단한 요청부터 시작합니다.

2. **Day 3-4: CLAUDE.md 만들기**
   프로젝트 폴더에 CLAUDE.md 파일을 만들고, 5줄 최소 버전을 넣습니다. Claude의 행동이 어떻게 달라지는지 관찰하세요.

3. **Day 5-6: 실전 프로젝트 하나 시작**
   Part 5의 업종별 프로젝트 중 하나를 골라 따라해보세요. 중간에 에러가 나면 당황하지 말고 '고쳐줘'라고 하면 됩니다.

4. **Day 7: 정리 + 습관화**
   이번 주에 배운 것을 CLAUDE.md에 추가합니다. '교훈' 섹션에 실수한 것과 해결법을 기록하세요. 이게 Boris의 자기학습 루프입니다.

주요 명령어 치트시트

명령어	기능	언제 쓰나
`claude`	Claude Code 시작	매번 시작할 때
`claude --continue`	마지막 세션 이어하기	이전 대화를 이어갈 때
`claude --resume`	세션 목록에서 선택	특정 세션으로 돌아갈 때
`/clear`	대화 초기화	새 주제 시작 시
`/compact`	대화 압축	대화가 길어졌을 때
`/cost`	비용 확인	사용량이 궁금할 때
`/model`	모델 변경	속도/품질 조절 시
`Esc`	작업 중단	Claude가 잘못된 방향일 때
`Esc Esc`	체크포인트 복원	되돌리기가 필요할 때
`Shift+Tab`	모드 전환	Plan Mode ↔ 일반 모드

Part 1 핵심 정리

① Node.js 설치 → claude 명령 → 첫 대화 3분 완료
② CLAUDE.md 5줄로 시작 → 시간이 지나면서 확장
③ 한 입 크기로 쪼개서 지시
④ 에러 = 피드백. '고쳐줘' 한 마디면 해결
⑤ /clear와 /compact로 컨텍스트 관리

VS Code에서 Claude Code 사용하기

코딩을 전혀 모르는 분도 VS Code(코드 편집기)를 설치하면 Claude Code를 훨씬 편하게 쓸 수 있습니다. 코드를 보면서 동시에 Claude와 대화할 수 있거든요.

VS Code 설치 (5분)

1. **다운로드**
   code.visualstudio.com 접속 → 운영체제에 맞는 버전 다운로드
2. **설치**
   Mac: .zip 풀고 응용 프로그램에 드래그
   Windows: .exe 실행, 'Add to PATH' 체크
3. **Claude Code 확장 설치**
   VS Code 왼쪽 블록 아이콘(확장) 클릭 → 'Claude Code' 검색 → Install
4. **프로젝트 열기**
   File → Open Folder → 작업할 폴더 선택
5. **Claude 실행**
   왼쪽 Spark(반짝) 아이콘 클릭 또는 Cmd+Esc(Mac)/Ctrl+Esc(Windows)

터미널 vs VS Code, 뭘 써야 해?

상황	추천	이유
빠른 질문 하나	터미널	가볍고 빠름
여러 파일 수정	VS Code	코드 보면서 대화 가능
코드 리뷰	VS Code	인라인 diff(변경사항 비교)
서버 작업	터미널	SSH 접속 시 유일한 선택
처음 배우는 분	VS Code	시각적 피드백이 많아 이해 쉬움
병렬 세션	둘 다	터미널 탭 + VS Code 패널 조합

꿀팁

고급 사용자들은 둘 다 씁니다. VS Code에서 메인 작업을 하고, 내장 터미널(Ctrl+`)에서 보조 작업을 합니다. Claude Code 패널은 오른쪽, 코드는 가운데, 터미널은 아래 — 이 레이아웃이면 창 전환이 거의 필요 없습니다.

첫 프로젝트 따라하기: 포트폴리오 웹페이지

설치가 끝났으니, 실제로 뭔가를 만들어봅시다. 30분이면 끝나는 간단한 포트폴리오 웹페이지를 만들어봅니다.

1 프로젝트 시작

```
mkdir my-portfolio && cd my-portfolio
claude
```

2 CLAUDE.md 먼저 만들기

```
> CLAUDE.md 파일을 만들어줘. 내용은:
  - 한국어, 해요체
  - 모바일 반응형 필수
  - 깔끔하고 현대적인 디자인
```

3 메인 지시

```
> 나의 포트폴리오 웹페이지를 만들어줘.

  내 정보:
  - 이름: 홍길동
  - 직업: 디지털 마케터
  - 소개: 5년차 퍼포먼스 마케터.
    데이터 기반 마케팅으로 ROI 200% 달성.

  섹션:
  1. 히어로 (이름 + 한 줄 소개 + 프로필 이미지 자리)
  2. About (자기소개 3~4줄)
  3. Skills (마케팅, 데이터 분석, 콘텐츠 기획 등)
  4. Projects (프로젝트 카드 3개)
  5. Contact (이메일, LinkedIn 링크)

  디자인:
  - 다크 테마
  - 블루 계열 포인트 색상
  - 부드러운 스크롤 애니메이션
```

4 수정하기

```
# Claude가 만든 결과를 브라우저에서 확인한 후
> About 섹션의 텍스트를 좀 더 임팩트 있게 바꿔줘.
  "마케팅은 숫자로 말한다"를 모토로, 라는 문구를 넣어줘.

> Projects 카드에 마우스 올리면 살짝 위로 올라오는
  호버 효과를 추가해줘.

> Contact 섹션에 이메일 복사 버튼을 추가해줘.
  클릭하면 "복사되었습니다!" 알림이 뜨게.
```

5 완성 확인

```
> 모바일에서도 잘 보이는지 확인해줘.
  반응형이 제대로 되어 있나 체크하고,
  문제가 있으면 고쳐줘.
```

축하합니다! 🎉

코드 한 줄 안 쓰고 프로 수준의 포트폴리오 웹페이지를 만들었습니다. 이 파일(index.html)을 GitHub Pages나 Vercel에 올리면 전 세계 누구나 볼 수 있습니다.

프롬프트 잘 쓰는 법: 5가지 원칙

Claude에게 말하는 방식에 따라 결과 품질이 크게 달라집니다. 어렵지 않습니다. 5가지만 기억하세요.

원칙 1: 구체적으로 말하기

✕ 모호함

"웹사이트 만들어줘"

✓ 구체적

"카페 메뉴를 보여주는 웹페이지 만들어줘. 헤더에 카페 이름, 본문에 음료/디저트 카테고리별 메뉴와 가격."

원칙 2: 맥락 제공하기

✕ 맥락 없음

"로그인 기능 추가해줘"

✓ 맥락 있음

"지금 이 프로젝트는 React로 된 블로그야. 여기에 이메일+비밀번호 로그인을 추가하고 싶어. 백엔드는 Supabase를 쓸 거야."

원칙 3: 출력 형식 지정하기

```
# 형식을 지정하면 원하는 형태로 받을 수 있음
> 이 프로젝트의 파일 구조를 설명해줘.
  표 형태로, 파일명 | 역할 | 중요도 순으로 정리해줘.
```

원칙 4: 단계별로 시키기

```
# 한 번에 다 시키지 말고 나눠서
> 먼저 헤더만 만들어줘. (확인)
> 좋아, 이제 메뉴 섹션을 추가해줘. (확인)
> 이제 푸터를 만들어줘. (확인)
```

원칙 5: 역할 부여하기

```
> 너는 10년차 시니어 프론트엔드 개발자야.
  이 랜딩페이지의 성능을 최적화해줘.
  Core Web Vitals 기준으로 점수를 매기고,
  개선 포인트를 우선순위별로 알려줘.
```

XML 태그 활용: 한 단계 더

더 정확한 결과를 원할 때는 XML 태그로 구분하면 Claude가 각 부분의 역할을 명확히 이해합니다.

```
>
<context>
  독자: 30~40대 직장인, 코딩 모름
  목적: Claude Code 소개 블로그
</context>

<instructions>
  - 해요체, 1,500자
  - 비유 2개 이상
  - 전문 용어 사용 시 괄호 안에 쉬운 설명
</instructions>

<examples>
  좋은 시작: "AI가 코드를 써주는 시대가 왔어요."
  나쁜 시작: "Claude Code는 CLI 기반 AI 어시스턴트입니다."
</examples>
```

ℹ 알아두기

XML 태그는 필수가 아닙니다. 일반적인 대화로도 충분히 좋은 결과를 얻을 수 있습니다. 하지만 복잡한 지시에서 정확도를 높이고 싶을 때 유용합니다.

터미널 기초: 5가지 명령어만 알면 됩니다

Claude Code를 쓰려면 터미널에서 최소한의 명령어를 알아야 합니다. 딱 5개만 알면 충분합니다.

1. cd (이동하기)

```
# 폴더로 이동
cd my-project        # my-project 폴더로 이동
cd ..                 # 상위 폴더로 이동
cd ~                  # 홈 폴더로 이동
cd ~/Documents/work   # 경로를 직접 지정
```

2. mkdir (폴더 만들기)

```
mkdir my-project      # 폴더 생성
mkdir -p a/b/c        # 중간 폴더까지 한 번에
```

3. ls (목록 보기)

```
ls                    # 현재 폴더의 파일 목록
ls -la                # 숨김 파일까지 상세 목록
```

4. pwd (현재 위치 확인)

```
pwd                    # 현재 어디에 있는지 표시
/Users/홍길동/my-project
```

5. code (VS Code로 열기)

```
code .                 # 현재 폴더를 VS Code로 열기
code index.html       # 특정 파일 열기
```

> 💡 **꿀팁**
> 이 5개 명령어면 Claude Code를 시작하기에 충분합니다. 더 복잡한 명령어는 Claude에게 물어보면 됩니다. `> 현재 폴더에 뭐가 있는지 보여줘` 라고 하면 Claude가 알아서 ls를 실행합니다.

프로젝트 폴더 구조 이해하기

Claude가 만드는 프로젝트 폴더의 일반적인 구조입니다. 하나하나 이해할 필요는 없지만, 대략적인 역할을 알면 도움이 됩니다.

```
my-project/
├── index.html        # 메인 웹페이지
├── style.css         # 디자인 (색상, 폰트, 레이아웃)
├── script.js         # 동작 (버튼 클릭, 애니메이션)
├── CLAUDE.md         # Claude 규칙서 (우리가 만든 것)
├── package.json      # 프로젝트 설명서 (자동 생성)
├── .claude/          # Claude 설정 폴더
│   ├── settings.json # Claude 설정
│   └── commands/     # 슬래시 커맨드
└── src/              # 소스 코드 (큰 프로젝트)
```

> ℹ **알아두기**
> 이 구조를 외울 필요 없습니다. Claude에게 `'이 프로젝트의 파일 구조를 설명해줘'` 라고 하면 각 파일의 역할을 친절하게 알려줍니다.

Git 초보자 가이드: 3분이면 됩니다

Git은 코드의 '버전 관리' 시스템입니다. 문서 작업할 때 '보고서_최종.docx', '보고서_진짜최종.docx', '보고서_최종의최종.docx' 같은 경험 있으시죠? Git은 이 문제를 해결합니다.

Git이 하는 일

- **저장 (commit)**: 현재 상태를 사진 찍듯 저장
- **되돌리기 (checkout)**: 이전 사진으로 돌아가기
- **공유 (push)**: GitHub에 올려서 다른 사람과 공유

Claude에게 Git 맡기기

Git 명령어를 직접 배울 필요 없습니다. Claude가 대신 해줍니다.

```
# 프로젝트에 Git 시작하기
> 이 프로젝트에 Git을 초기화해줘.

# 현재 작업 저장하기
> 지금까지 만든 것을 커밋해줘.
  커밋 메시지는 "메인 페이지 완성"으로.

# 이전 상태로 되돌리기
> 방금 변경 취소해줘. 이전 버전으로 돌아가줘.

# GitHub에 올리기
> 이 프로젝트를 GitHub에 올려줘.
  리포 이름은 my-portfolio로.
```

⚠ **주의**

중요: Claude Code를 사용할 때 Git을 설정해두면, Claude가 실수로 파일을 잘못 수정했을 때 쉽게 되돌릴 수 있습니다. 프로젝트 시작 시 `> Git 초기화해줘` 한 마디를 꼭 하세요.

배포: 만든 것을 세상에 공개하기

만든 웹페이지를 인터넷에 올려서 다른 사람도 볼 수 있게 하는 과정입니다.

가장 쉬운 방법: Vercel

```
> 이 프로젝트를 Vercel에 배포하고 싶어.
  나는 Vercel을 한 번도 써본 적 없어.
  1단계부터 차근차근 알려줘.
  각 단계에서 뭘 클릭해야 하는지 구체적으로.
```

무료 대안: GitHub Pages

```
> 이 프로젝트를 GitHub Pages로 무료 배포해줘.
  필요한 설정을 다 해주고,
  내가 해야 할 일만 순서대로 알려줘.
```

💡 **꿀팁**

Vercel은 가입만 하면 무료로 웹사이트를 배포할 수 있습니다. GitHub과 연동하면 코드를 Push할 때마다 자동으로 업데이트됩니다. 마케터도 자주 쓰는 서비스입니다.

Claude Code 사용 시나리오별 가이드

상황별로 Claude에게 어떻게 말하면 되는지 정리했습니다.

시나리오 1: 웹페이지 만들기

```
> [서비스 이름]을 위한 [페이지 종류]를 만들어줘.
  포함할 섹션: [섹션 나열]
  디자인: [색상/스타일 선호도]
  특이사항: [모바일 반응형, 다국어 등]
```

시나리오 2: 기존 프로젝트에 기능 추가

```
> 이 프로젝트에 [기능]을 추가해줘.
  현재 기술 스택: [React, Vue 등]
  위치: [어디에 추가할지]
  조건: [기존 코드 최소 수정, 테스트 포함 등]
```

시나리오 3: 에러 해결

```
> 이 에러를 고쳐줘:
  [에러 메시지 전체 복사]

  어떤 상황에서 발생: [설명]
  시도한 것: [있으면 나열]
```

시나리오 4: 코드 이해하기

```
> @[파일명]이 뭘 하는 파일인지 설명해줘.
  프로그래밍 모르는 사람한테 설명하듯이.
  전문 용어 쓸 때는 바로 옆에 쉬운 설명 붙여줘.
```

시나리오 5: 프로젝트 전체 파악

```
> 이 프로젝트의 파일들을 전부 훑어보고,
  "이 프로젝트는 이런 거다"를 한 문단으로 요약해줘.
  그다음 중요한 파일 5개를 뽑아서 각각 한 줄로 설명해줘.
```

시나리오 6: 성능 개선

```
> 이 웹사이트의 성능을 점검해줘.
  - 페이지 로딩이 느릴 수 있는 부분
  - 이미지나 파일이 너무 큰 건 없는지
  - Lighthouse 점수를 높일 수 있는 부분
  개선할 수 있으면 바로 고쳐줘.
```

시나리오 7: 자동화 만들기

```
> [반복 작업]을 자동화하고 싶어.
  현재 수동으로 하는 프로세스: [설명]
  원하는 결과: [어떻게 되면 좋은지]
  사용 가능한 도구: [Notion, Google Sheets 등]
```

프롬프트 템플릿 활용 원칙

위 템플릿을 그대로 복사해서 [대괄호] 부분만 채우면 됩니다. 처음에는 이렇게 시작하고, 점점 자기만의 스타일을 만들어가세요.

Mac 사용자 완전 가이드

Mac에서 Claude Code를 최적으로 사용하기 위한 상세 설정입니다.

Homebrew 설치 (패키지 관리자)

Homebrew는 Mac용 '앱스토어'와 비슷한 도구입니다. 터미널에서 프로그램을 쉽게 설치할 수 있게 해줍니다.

```
# Homebrew 설치 (한 줄 실행)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 설치 확인
brew --version

# Node.js 설치 (Homebrew로)
brew install node

# 설치 확인
node --version  # v18 이상
npm --version   # 숫자가 나오면 OK
```

iTerm2 추천 설정

기본 터미널보다 iTerm2가 훨씬 편합니다.

```
# iTerm2 설치
brew install --cask iterm2

# 추천 설정
# Preferences → Profiles → Keys →
# "Option key acts as Esc+"로 변경
# → Option+← / Option+→ 으로 단어 단위 이동 가능
```

zsh 별칭 설정

자주 쓰는 명령을 짧게 줄입니다.

```
# ~/.zshrc 파일에 추가
alias cc="claude"
alias ccc="claude --continue"
alias ccr="claude --resume"

# 적용
source ~/.zshrc

# 이제 cc만 입력하면 Claude Code 시작
cc
```

💡 **꿀팁**

Mac 단축키 Top 5:

- Cmd+T: 새 터미널 탭
- Cmd+D: 화면 분할
- Cmd+W: 탭 닫기
- Ctrl+C: 실행 중인 명령 중단
- Ctrl+L: 화면 정리

Windows 사용자 완전 가이드

Windows에서 Claude Code를 쓸 때 추가로 알아야 할 것들입니다.

WSL2 설치 (강력 추천)

WSL(Windows Subsystem for Linux)은 Windows 안에서 Linux를 돌리는 기능입니다. Claude Code는 Linux 환경에서 가장 안정적입니다.

```
# PowerShell을 관리자 권한으로 실행한 후
wsl --install

# 재부팅 후 Ubuntu가 자동 설치됨
# 사용자명과 비밀번호를 설정하면 완료

# WSL 안에서 Node.js 설치
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Claude Code 설치
npm install -g @anthropic-ai/claude-code
```

한국어 경로 문제 해결

Windows에서 가장 흔한 문제입니다. 프로젝트 폴더 경로에 한글이 포함되면 에러가 날 수 있습니다.

```
# ✗ 나쁜 경로
C:\Users\홍길동\바탕화면\내프로젝트

# ✓ 좋은 경로
C:\dev\my-project
D:\projects\my-site
```

⚠ 주의

Windows 사용자 필수 확인:

1. 프로젝트 폴더에 한글 없는지 확인
2. 'Add to PATH' 체크했는지 확인
3. PowerShell 대신 Git Bash 또는 WSL 사용 권장
4. 바이러스 백신이 npm을 차단하는지 확인

Windows Terminal 설정

```
# Microsoft Store에서 "Windows Terminal" 설치
# 설정 → 기본 프로필 → Git Bash 또는 WSL Ubuntu 선택
# 색상 테마: One Half Dark 추천
```

Linux 사용자 가이드

```
# Ubuntu/Debian
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
npm install -g @anthropic-ai/claude-code

# Node.js 버전 관리 (nvm 추천)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
nvm install --lts
nvm use --lts
```

ℹ 알아두기

Linux는 Claude Code와 가장 궁합이 좋습니다. 서버에서 돌리거나 EC2에서 작업할 때 기본 환경이 됩니다. AWS Lightsail이나 EC2에 Claude Code를 설치하면 어디서든 접속 가능한 개발 환경이 됩니다.

Claude Code + VS Code 통합 워크플로우

VS Code를 메인 에디터로 쓰면서 Claude Code를 최대한 활용하는 방법입니다.

레이아웃 설정

```
# VS Code 화면 구성 (추천)

┌──────────────────────────────────────────────────────────────────────────┐
│ 왼쪽: 파일 탐색기    │ 가운데: 코드 편집    │ 오른쪽: Claude Code  │
│                      │                      │ 패널              │
│ - index.html         │ <html>               │ > 안녕, 이 파일을   │
│ - style.css          │   <head>...          │   수정해줘.         │
│ - CLAUDE.md          │   <body>             │                    │
│                      │     ...              │ Claude: 네, 수정    │
│                      │                      │ 하겠습니다.         │
├──────────────────────┴──────────────────────┴────────────────────┤
│ 아래: 내장 터미널 (추가 Claude 세션 가능)                          │
│ $ claude                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

핵심 단축키

단축키 (Mac/Win)	기능
Cmd/Ctrl + `	내장 터미널 열기/닫기
Cmd/Ctrl + B	사이드바 열기/닫기
Cmd/Ctrl + P	파일 빠른 열기
Cmd/Ctrl + Shift + P	명령 팔레트
Cmd/Ctrl + D	같은 단어 다중 선택
Cmd/Ctrl + /	주석 토글
Alt + ↑/↓	줄 이동
Cmd/Ctrl + Shift + K	줄 삭제

추천 VS Code 확장

확장	용도	필요도
Claude Code	Claude Code 통합	★★★ 필수
Prettier	코드 자동 정리	★★★ 필수
Korean Language Pack	한국어 메뉴	★★ 추천
GitLens	Git 변경 이력 표시	★★ 추천
Live Server	HTML 실시간 미리보기	★★ 추천
Material Icon Theme	파일 아이콘	★ 선택

💡 **꿀팁**

Live Server 꿀팁: HTML 파일을 만든 후 VS Code 우하단의 'Go Live'를 클릭하면 브라우저에서 실시간 미리보기가 됩니다. 코드를 수정할 때마다 자동으로 새로고침됩니다.

Claude Code 활용 사례 모음: 비개발자 편

에이나우 커뮤니티에서 실제로 Claude Code를 활용한 비개발자들의 사례입니다.

사례 1: 부동산 중개사 — 매물 소개 웹사이트

서울의 부동산 중개사 김 대표는 각 매물마다 예쁜 소개 페이지를 만들고 싶었습니다. 기존에는 디자이너에게 맡기면 건당 30~50만 원, 일주일이 걸렸습니다.

```
> 아파트 매물 소개 웹페이지를 만들어줘.
  매물 정보: 서울 강남구 역삼동, 34평, 15억
  고급스러운 느낌으로.
  사진 자리 4개, 평면도 자리 1개,
  주변 편의시설 지도 링크,
  문의하기 버튼 (전화/카톡).
```

결과: 15분 만에 완성. 디자이너 비용 절감, 매물 올라오는 당일 페이지 제작 가능.

사례 2: 요가 강사 — 수강생 관리 도구

요가 강사 이 선생님은 수강생 출석, 결제, 스케줄을 엑셀로 관리하다가 한계를 느꼈습니다.

```
> 요가 수업 관리 웹앱을 만들어줘.
  기능: 수강생 목록, 출석 체크, 월별 결제 현황,
  주간 시간표. Supabase 사용.
```

결과: 3일 만에 MVP 완성. 수강생 30명 관리를 수작업에서 자동화.

사례 3: 카페 사장님 — 인스타 콘텐츠 자동화

매일 인스타에 올릴 메뉴 사진과 캡션을 만드는 게 부담이었습니다.

```
> /instagram-caption 아메리카노 신메뉴 출시
```

결과: 스킬 하나로 매일 5분이면 인스타 포스팅 준비 완료.

공통점

모든 사례의 공통점: **본인의 문제를 정확히 알고 있었다.**

Claude Code에게 '무엇이 필요한지' 설명할 수 있으면, 기술적인 건 Claude가 해결합니다.

사례 4: 학원 원장 — 학부모 알림 시스템

학원 원장 박 선생님은 매주 학부모에게 학습 진도와 숙제를 카카오톡으로 알려야 했습니다. 30명 학생의 개별 메시지를 수동으로 보내는 데 매주 2시간이 걸렸습니다.

```
> 학생별 주간 학습 리포트를 자동 생성하는 도구를 만들어줘.

  입력: Google Sheets (학생 이름, 과목, 진도, 숙제 완료율)
  출력: 학생별 카카오톡 메시지 텍스트

  메시지 형식:
  "[학생이름] 학부모님 안녕하세요.
   이번 주 [과목] 진도: [진도]
   숙제 완료율: [완료율]%
   다음 주 예고: [내용]"

  모든 학생의 메시지를 한 번에 생성해줘.
```

결과: 2시간 → 10분으로 단축. Google Sheets만 업데이트하면 메시지 자동 생성.

사례 5: 쇼핑몰 운영자 — 상세페이지 자동 생성

쿠팡, 스마트스토어에 올릴 상세페이지를 매번 디자이너에게 맡기면 5~10만 원씩 들었습니다.

```
> 쿠팡 상세페이지 HTML을 만들어줘.

  제품: 유기농 그래놀라 (500g)
  가격: 15,900원

  포함할 섹션:
  1. 메인 이미지 + 한 줄 카피
  2. 제품 특장점 3가지 (아이콘 + 텍스트)
  3. 성분표
  4. 이런 분께 추천 (타겟 3종)
  5. 리뷰 하이라이트 3개
  6. 배송/교환 안내

  디자인: 깔끔, 고급스러운 느낌, 모바일 최적화
```

결과: 상세페이지 제작 시간 3일 → 1시간. 디자인 비용 90% 절감.

사례 6: 프리랜서 번역가 — 포트폴리오 + 견적 자동화

```
> 번역가 포트폴리오 웹사이트를 만들어줘.
  자동 견적 계산기 포함:
  - 원문 언어, 번역 언어 선택
  - 분량 (글자수 또는 페이지수) 입력
  - 난이도 선택 (일반/전문/법률)
  - 자동으로 예상 비용과 소요일 계산
```

결과: 문의 → 견적서 과정이 자동화. 고객이 직접 견적을 확인하므로 상담 시간 70% 절감.

> 💡 **꿀팁**
>
> **사례에서 배우는 핵심 패턴:**
> 1. 자기 업무의 **반복적인 고통**을 정확히 파악
> 2. Claude에게 **구체적으로** 설명 (입력/출력/형식 명시)
> 3. MVP부터 만들고, **피드백 반영해서 개선**

Claude Code 학습 체크리스트

각 레벨별로 마스터해야 할 항목의 체크리스트입니다. 인쇄해서 체크하면서 진행하세요.

Level 1: 왕초보 (1주)

☐	항목	참고
☐	Node.js 설치 완료	Ch.1
☐	Claude Code 설치 완료	Ch.1
☐	첫 대화 성공 (hello world)	Ch.1
☐	CLAUDE.md 5줄 버전 생성	Ch.2
☐	간단한 웹페이지 1개 완성	Ch.2
☐	/clear 사용해보기	Ch.2
☐	에러 발생 → "고쳐줘" 경험	Ch.2

Level 2: 활용 (2~3주)

☐	항목	참고
☐	CLAUDE.md에 교훈 5개 이상 추가	Ch.3
☐	Plan Mode 사용해보기 (Shift+Tab)	Ch.3
☐	서브에이전트 사용해보기	Ch.5
☐	실전 프로젝트 1개 완성	Part 5
☐	Vercel 또는 GitHub Pages 배포	Part 5
☐	Git 초기화 + 커밋 경험	부록
☐	/compact 사용해보기	Ch.5

Level 3: 실전 (1~2개월)

☐	항목	참고
☐	Slash Command 3개 이상 생성	Ch.5
☐	병렬 세션 (2개 이상) 사용	Ch.4
☐	CLAUDE.md 교훈 20개 이상	Ch.3
☐	Everything Claude Code 보안 스캔	Ch.8
☐	스킬 1개 직접 제작	Ch.9
☐	비용 최적화 설정 적용	Ch.16
☐	프로젝트 2개 이상 완성	Part 5

Level 4: 프로 (2~3개월)

☐	항목	참고
☐	Hook 3개 이상 설정	Ch.18
☐	MCP 서버 1개 이상 연결	Ch.18
☐	LSP 플러그인 설치	Ch.10
☐	CLAUDE.md 교훈 40개 이상	Ch.3
☐	자동화 파이프라인 1개 구축	Ch.18
☐	팀 또는 동료에게 소개	확장
☐	비용 50% 이상 절감 달성	Ch.16

PART 2

창시자 Boris Cherny가 알려주는 진짜 사용법

Claude Code를 만든 사람은 어떻게 쓸까?
2026년 1~2월, Boris가 공개한 워크플로우를
왕초보 눈높이에서 완전 해부합니다.

CHAPTER 03

100줄 CLAUDE.md의 비밀

Boris Cherny는 누구인가

Boris Cherny는 Anthropic의 Staff Engineer이자 **Claude Code의 창시자**입니다. 2025년 중반 출시된 Claude

Code는 불과 1년 만에 전 세계 개발자들의 일하는 방식을 바꿔놓았고, 그 도구를 설계하고 만든 사람이 Boris입니다.

2026년 1월 2일, Boris는 Threads에서 이례적으로 자신의 전체 워크플로우를 공개했습니다. 그 후 1월 31일, 2월 11일에 추가 팁을 공유해 총 42가지 팁이 되었습니다. 인터넷이 뒤집어졌습니다. 이유는 간단합니다 — **도구를 만든 사람이 쓰는 방법을 알려줬으니까요.**

> *대부분의 개발자들이 500줄, 800줄짜리 CLAUDE.md를 씁니다. 제 것은 약 100줄이에요. 그리고 더 잘 작동합니다.*
>
> — Boris Cherny, Anthropic Staff Engineer

왜 짧을수록 강한가

Boris의 CLAUDE.md 파일은 약 **2,500 토큰, 약 100줄**입니다. 대부분의 사용자가 쓰는 파일의 1/8 수준이죠. 비결은 세 가지입니다.

1. **모든 줄이 실제 문제를 해결한다**
   혹시 모르니까 넣은 줄은 하나도 없습니다. 모든 규칙은 '이거 때문에 실제로 문제가 생겼다'에서 출발합니다.
2. **CLAUDE.md는 매 세션마다 로드된다**
   500줄짜리 CLAUDE.md는 매 세션 시작 시 수천 토큰을 잡아먹습니다. 100줄이면 1/5의 토큰만 쓰면서 더 정확합니다.
3. **Claude는 간결한 지시를 더 잘 따른다**
   긴 규칙서는 Claude가 우선순위를 혼동하게 만듭니다. 핵심만 담긴 짧은 규칙이 정확도가 훨씬 높습니다.

> ⚠ 주의
> CLAUDE.md에 '절대 ~하지 마'를 너무 많이 넣지 마세요. 금지 규칙이 많으면 Claude가 과도하게 조심스러워집니다. '~해라' 형태의 긍정적 지시가 더 효과적입니다.

Boris CLAUDE.md의 4가지 핵심 규칙

Boris의 100줄 CLAUDE.md를 분석하면 4가지 핵심 원칙이 드러납니다.

규칙 1: Plan Mode가 기본이다

3단계 이상의 작업은 무조건 계획부터 세웁니다. Boris는 Shift+Tab을 두 번 눌러 Plan Mode로 전환하고, Claude와 계획을 주고받은 뒤에야 실행합니다.

```
# Boris의 패턴
> [Shift+Tab 두 번 → Plan Mode]

> 사용자 인증 기능을 추가하려고 해.
  어떤 파일을 수정해야 하는지,
  어떤 순서로 진행할지 계획을 세워줘.

Claude: 계획을 세워보겠습니다...
1. src/auth/ 폴더 생성
2. login.ts, register.ts 파일 작성
3. 기존 라우터에 인증 미들웨어 추가
4. 테스트 파일 작성

> 좋아. 2번에서 비밀번호 해싱도 포함해줘.
  그리고 4번 테스트에 실패 케이스도 넣어.

# 계획이 확정되면 Plan Mode 해제 → 실행
> [Shift+Tab → 일반 모드]
> 위 계획대로 진행해줘.
```

💡 **꿀팁**

Plan Mode의 진짜 가치: Boris 팀에서 한 사람은 Plan Mode에서 계획을 세운 뒤, **두 번째 Claude 세션을 열어 그 계획을 시니어 엔지니어 관점에서 리뷰**시킵니다. 계획의 품질이 올라가면 실행이 한 번에 끝납니다.

규칙 2: 서브에이전트를 적극 활용한다

메인 Claude의 컨텍스트(기억력)를 깨끗하게 유지하기 위해, 탐색이나 분석 작업은 **서브에이전트(조수 AI)**에게 맡깁니다.

```
> 서브에이전트를 써서 src/ 폴더의 모든 API 엔드포인트를
  찾아서 목록으로 정리해줘.
```

서브에이전트가 파일 50개를 뒤져도, 메인 Claude의 컨텍스트에는 결과 요약 한 단락만 올라갑니다.

규칙 3: 자기학습 루프를 돌린다

이것이 Boris 팀의 **가장 강력한 습관**입니다. Claude가 실수할 때마다 규칙을 추가합니다.

규칙 4: 증명 없이 완료라고 하지 않는다

작업이 끝났다고 말하기 전에 반드시 증거를 보여줍니다. 테스트 통과, 로그 확인, 실제 실행 결과 — "시니어 엔지니어가 이걸 승인할까?" 기준으로 검증합니다.

Boris의 황금 규칙: 자기학습 루프

> *Claude가 뭔가 잘못할 때마다, CLAUDE.md를 업데이트하세요. 같은 실수를 다시 안 하도록. Claude는 자기 자신을 위한 규칙을 놀라울 정도로 잘 씁니다.*
>
> — Boris Cherny

이것이 6단계 태스크 관리 루프입니다.

단계	행동	예시
1	작업 지시	"로그인 기능 만들어줘"
2	Claude 실행	코드 작성 + 테스트
3	결과 검토	코드 확인, 테스트 실행
4	문제 발견	"비밀번호 해싱이 빠졌네"
5	교정	"비밀번호를 bcrypt로 해싱해"
6	**규칙화**	**"CLAUDE.md에 추가해: 인증 시 항상 bcrypt 적용"**

6단계가 마법입니다. Claude에게 딱 한 마디만 하면 됩니다:

```
> 방금 실수한 걸 기반으로 CLAUDE.md에 규칙을 추가해줘.
  다음에 같은 실수를 반복하지 않도록.
```

Boris에 따르면, Claude는 자기 자신을 위한 규칙을 "소름 끼칠 정도로 잘" 씁니다. 한 문장이면 됩니다. 그리고 시간이 지날수록 Claude가 여러분의 프로젝트를 점점 더 깊이 이해하게 됩니다.

실전: 자기학습 루프 따라하기

```
# 1. 작업 지시
> 회원가입 폼을 만들어줘. 이메일, 비밀번호, 비밀번호 확인.

# 2-3. Claude 실행 + 결과 검토
Claude: ✓ 회원가입 폼을 만들었습니다.

# 4. 문제 발견 — 직접 테스트해보니...
> 비밀번호가 8자 미만이어도 가입이 돼. 검증이 빠졌어.

# 5. 교정
> 비밀번호 8자 이상, 영문+숫자 조합 검증을 추가해줘.

# 6. 규칙화 ← 이게 핵심!
> 잘 됐어. 이제 CLAUDE.md에 이 교훈을 추가해줘:
  "폼 입력 검증은 프론트엔드와 백엔드 양쪽에서 구현할 것.
   비밀번호는 최소 8자, 영문+숫자 조합 필수."
```

1주일이면 변합니다

매일 1~2개씩 교훈이 쌓이면, 1주일 후에는 10~15개의 프로젝트 맞춤 규칙이 생깁니다. Claude의 실수율이 눈에 띄게 떨어지는 걸 체감할 수 있습니다.

왕초보를 위한 CLAUDE.md 단계별 성장

Boris의 철학을 적용한 3단계 성장 템플릿입니다.

Level 1: 최소 버전 (첫날)

```
# CLAUDE.md — 프로젝트 규칙

## 기본
- 한국어로 대화
- 해요체 사용
- 코드 수정 전에 먼저 뭘 할지 설명

## 교훈
- (Claude가 실수할 때마다 여기에 추가)
```

Level 2: 실전 버전 (1주 후)

```
# CLAUDE.md — 프로젝트 규칙

## 기본
- 한국어로 대화, 해요체
- 3단계 이상 작업은 Plan Mode에서 시작
- 코드 수정 전에 먼저 설명

## 코드 스타일
- 변수명은 영어 camelCase
- 주석은 한국어
- 함수는 30줄 이내로 유지

## 검증
- 수정 후 반드시 테스트 실행
- 에러 없이 통과해야 완료

## 교훈
- 폼 검증은 프론트+백엔드 양쪽에서
- 비밀번호는 해싱 처리 필수
- API 응답에 에러 핸들링 항상 포함
- 파일 삭제 전에 백업 확인
```

Level 3: 프로 버전 (1개월 후)

```
# CLAUDE.md — 프로젝트 규칙

## 원칙
- 한국어, 해요체
- 3단계 이상 → Plan Mode 필수
- 일이 꼬이면 즉시 멈추고 re-plan
- 서브에이전트로 탐색, 메인은 구현에 집중

## 기술 스택
- Frontend: React + Tailwind CSS
- Backend: Supabase
- Deploy: Vercel

## 코드 규칙
- 함수 30줄 이내, 파일 300줄 이내
- 새 기능 → 테스트 먼저 작성 (TDD)
- API 엔드포인트마다 에러 핸들링

## 검증
- npm test 통과 필수
- 수정 후 관련 테스트 자동 실행
- "시니어 엔지니어가 승인할까?" 기준

## 교훈 (30개 이상 축적됨)
- ... (실전에서 쌓인 규칙들)
```

> **ℹ 알아두기**
>
> Level 3에 도달하면 Boris와 같은 방식으로 Claude를 운영하는 것입니다. 100줄 이내에서 핵심만 담고, 교훈이 계속 쌓이면서 Claude가 점점 더 여러분의 프로젝트를 이해하게 됩니다.

CHAPTER 04

하루 100개 PR — 병렬 작업 시스템

Boris는 Claude를 몇 개나 동시에 돌릴까?

답: **10~15개.**

- 터미널에서 5개 (탭 1~5로 번호 매김)
- 웹(claude.ai/code)에서 5~10개
- 심지어 아침에 휴대폰(Claude iOS 앱)에서 세션을 시작하고, 밤에 확인

이것이 Claude Code 팀 전체가 공통으로 꼽는 **생산성 비밀 #1**입니다.

> *Git Worktrees로 3~5개 세션을 동시에 돌리세요. 팀 전체의 탑 1 팁입니다.*
>
> — Boris Cherny, Claude Code 팀 팁 공유 (2026.1.31)

Git Worktrees: 핵심 개념

여러 Claude를 동시에 돌리려면, 각각 **별도의 작업 공간**이 필요합니다. 같은 폴더에서 여러 Claude가 동시에 파일을 수정하면 충돌이 나니까요.

> ℹ 알아두기
>
> **Git Worktree란?** 하나의 프로젝트를 여러 '분신'으로 만드는 Git 기능입니다. 원본은 그대로 두고, 각 분신에서 독립적으로 작업할 수 있습니다. 작업이 끝나면 원본에 합칩니다.

```
# Worktree 만들기 (터미널에서)
git worktree add ../project-feature-a main
git worktree add ../project-feature-b main
git worktree add ../project-bugfix main

# 각 폴더에서 별도의 Claude 실행
cd ../project-feature-a && claude  # 터미널 탭 1
cd ../project-feature-b && claude  # 터미널 탭 2
cd ../project-bugfix && claude     # 터미널 탭 3
```

왕초보용: 2개부터 시작하기

처음부터 15개는 무리입니다. **2개만으로도 체감 차이가 큽니다.**

세션 1 (메인 작업)	세션 2 (보조 작업)
새 기능 개발	테스트 코드 작성
프론트엔드 UI	백엔드 API
코드 작성	코드 리뷰 요청
본문 작업	문서화/README 작성

Worktree 없이 간단하게 병렬 작업하기

Git Worktree가 어려우면, 더 간단한 방법도 있습니다.

방법 1: 폴더 복사

```
# 프로젝트 폴더를 복사
cp -r my-project my-project-copy

# 각 폴더에서 Claude 실행
cd my-project && claude       # 탭 1
cd my-project-copy && claude  # 탭 2
```

방법 2: 터미널 + 웹 조합

```
# 터미널에서 메인 작업
claude

# 브라우저에서 claude.ai/code 열기
# → 보조 작업 (리서치, 코드 리뷰 등)
```

> 💡 **꿀팁**
>
> **텔레포트 기능:** 터미널에서 작업하다가 외출해야 할 때, `/teleport` 로 세션을 웹으로 옮길 수 있습니다. 반대로 웹 세션을 터미널로 가져올 수도 있습니다. Boris는 아침에 폰으로 세션을 시작하고, 출근해서 터미널로 이어받는 패턴을 씁니다.

알림 설정: 여러 세션의 필수 도우미

여러 세션을 동시에 돌리면, Claude가 허락을 구하거나 작업이 끝났을 때 알아야 합니다.

Mac 알림 설정

```
# Claude Code에서 이렇게 말하세요
> Claude가 알림을 보낼 때 macOS 알림이 뜨도록
  훅을 설정해줘. ~/.claude/settings.json에 추가해줘.
```

Windows 알림 설정

```
# PowerShell 토스트 알림
> Claude가 알림을 보낼 때 Windows 토스트 알림이
  뜨도록 훅을 설정해줘.
```

Boris의 병렬 작업 실전 패턴

단계	세션 수	작업 내용
계획 단계	1~2개	Plan Mode로 설계 + 리뷰
구현 단계	3~5개	각 기능을 별도 세션에서 동시 구현
테스트 단계	1~2개	통합 테스트 + 버그 수정
배포 단계	1개	최종 검토 + 배포

생산성 공식

병렬 세션 수 × 작업 품질 = 산출량

Boris가 하루 100개 PR을 만들 수 있는 이유는 15개 세션 × 높은 CLAUDE.md 품질입니다. 여러분은 2~3개 세션부터 시작해서 점진적으로 늘려가세요.

CHAPTER 05

Boris의 42가지 팁 중 Top 15

Boris가 2026년 1월~2월에 5개 시리즈로 공개한 42가지 팁 중, **왕초보가 즉시 써먹을 수 있는 Top 15**를 선별했습니다.

> 💎 **Tier 1: 지금 당장 적용하세요 (5개)**

팁 #1: /clear를 자주 쓰세요

새로운 작업을 시작할 때마다 `/clear`로 대화를 초기화하세요. 이전 대화의 잔재가 새 작업을 방해합니다.

```
# ❌ 나쁜 습관: 한 세션에서 여러 주제
> 로그인 기능 만들어줘  # 30분 작업
> 이제 결제 기능도 만들어줘  # 앞의 맥락이 방해

# ✅ 좋은 습관: 주제 바뀌면 새로 시작
> 로그인 기능 만들어줘  # 30분 작업
> /clear
> 결제 기능을 만들어줘  # 깨끗한 컨텍스트에서 시작
```

팁 #2: 피드백 루프를 포함시키세요

Claude에게 작업을 시킬 때, 검증 방법도 함께 알려주세요. Boris는 이것만으로 **품질이 2~3배 올라간다**고 말합니다.

```
# ❌ 피드백 루프 없음
> 로그인 API를 만들어줘.

# ✅ 피드백 루프 포함
> 로그인 API를 만들어줘.
  만든 다음에 npm test를 실행해서 통과하는지 확인해.
  실패하면 고치고 다시 테스트해. 전부 통과해야 완료.
```

> 💡 **꿀팁**
>
> **Boris의 핵심 인사이트:** 피드백 루프가 있으면 Claude가 스스로 실수를 발견하고 수정합니다. 여러분이 일일이 확인할 필요가 줄어듭니다. 이것이 '에이전틱 코딩'의 핵심입니다.

팁 #3: Plan Mode에서 시작하세요

`Shift+Tab`을 두 번 누르면 Plan Mode입니다. 코드를 건드리지 않고 설계만 합니다.

```
> [Shift+Tab × 2 → Plan Mode]
> 블로그 시스템을 만들려고 해.
  어떤 파일 구조가 좋을지, 어떤 순서로 만들지
  계획을 세워줘. 코드는 아직 쓰지 마.
```

팁 #4: Slash Command로 반복 작업 자동화

매일 하는 작업이 있다면, Slash Command로 만드세요. Boris는 `/commit-push-pr` 같은 명령을 매일 사용합니다.

```
# .claude/commands/blog.md 파일 만들기

블로그 글 작성 워크플로우:
1. 주제: $ARGUMENTS
2. SEO 키워드를 먼저 리서치해줘
3. 제목은 숫자 + 키워드 조합으로 3개 제안
4. 선택한 제목으로 본문 작성
5. 메타 설명 추가
6. 맞춤법 검사 실행
```

```
# 사용법
> /blog AI 마케팅 자동화 트렌드 2026
```

팁 #5: "이게 맞다고 증명해봐" — 공격적 프롬프팅

> *prove to me this works. (이게 맞다고 증명해봐.)*
>
> — Boris Cherny의 실제 프롬프팅 스타일

Boris의 프롬프팅 스타일은 놀랍게도 **공격적**입니다. Claude를 아끼지 않고 도전합니다.

```
# Boris 스타일 프롬프트 예시
> 이게 맞다고 증명해봐. 테스트를 돌려서 보여줘.

> 지금까지 한 거 전부 버려.
  처음부터 우아한 방법으로 다시 해.

> 이 변경사항을 면밀히 검토해줘.
  내가 테스트를 통과할 때까지 PR을 올리지 마.
```

💎 Tier 2: 1주 안에 적용하세요 (5개)

팁 #6: 서브에이전트로 컨텍스트 보호

```
> 서브에이전트로 이 프로젝트의 모든 TODO 주석을
  찾아서 목록으로 정리해줘.
```

메인 컨텍스트를 아끼면서 필요한 정보를 얻는 방법입니다.

팁 #7: Esc+Esc로 체크포인트 복원

Claude가 잘못된 방향으로 가고 있다면 `Esc`로 멈추고, `Esc+Esc`로 이전 상태로 되돌립니다. 40% 확신인 접근도 부담 없이 시도할 수 있습니다.

팁 #8: ! 접두사로 빠른 명령 실행

```
# Claude에게 부탁하지 않고 직접 실행
> !git status
> !npm test
> !ls -la src/
```

명령어 앞에 `!`를 붙이면 Claude를 거치지 않고 바로 실행됩니다. 결과는 컨텍스트에 올라가므로 Claude도 볼 수 있습니다.

팁 #9: Opus 모델을 기본으로 쓰세요

Boris는 모든 작업에 Opus(가장 강력한 모델)를 사용합니다. Sonnet보다 느리지만, **조종할 필요가 적어서 실제로는 더 빠릅니다.**

팁 #10: LSP 플러그인을 설치하세요

```
> /plugin install typescript-lsp@claude-plugins-official
```

Boris가 말하는 "임팩트가 가장 큰 단일 플러그인"입니다. Claude가 파일을 수정할 때마다 자동으로 타입 에러, 미사용 변수 등을 감지합니다.

💎 Tier 3: 1개월 안에 적용하세요 (5개)

팁 #11: CLAUDE.md를 Git에 커밋하세요

Boris 팀은 CLAUDE.md를 **팀 전체가 공유**합니다. 한 사람이 발견한 교훈이 팀 전체의 자산이 됩니다. Git에 커밋하면 변경 이력도 추적됩니다.

팁 #12: 작업마다 notes 디렉토리를 만드세요

Boris 팀의 한 엔지니어는 Claude에게 모든 작업/프로젝트마다 notes 디렉토리를 유지하게 합니다. PR을 올린 후에도 학습 내용이 남습니다.

```
# CLAUDE.md에 추가
- 모든 작업에 대해 tasks/notes/ 디렉토리에 메모를 남겨줘
- PR을 올린 후에는 lessons.md를 업데이트해줘
```

팁 #13: Post-tool-use Hook으로 자동 포맷팅

```
# settings.json
{{
  "hooks": {{
    "PostToolUse": [
      {{
        "matcher": "Edit|Write",
        "hooks": [{{
          "type": "command",
          "command": "npx prettier --write $FILE_PATH"
        }}]
      }}
    ]
  }}
}}
```

Claude가 파일을 수정할 때마다 Prettier가 자동으로 코드를 정리합니다. Boris 팀도 이 Hook을 사용합니다.

팁 #14: /init으로 새 프로젝트 시작

```
> /init
```

이 명령 하나로 Claude가 프로젝트를 분석하고 CLAUDE.md 초안을 자동 생성합니다. 0에서 시작하지 않아도 됩니다.

팁 #15: context: fork로 스킬을 격리 실행

```
# 스킬을 서브에이전트에서 실행
# 메인 컨텍스트에는 결과만 올라옴
context: fork
```

무거운 스킬을 메인 컨텍스트에서 실행하면 기억력을 많이 잡아먹습니다. `context: fork`로 격리 실행하면 결과만 깔끔하게 받을 수 있습니다.

Boris 팁 적용 우선순위

① 지금 당장: /clear 자주 쓰기 + 피드백 루프 + Plan Mode
② 이번 주: 서브에이전트 + Esc+Esc + ! 명령
③ 이번 달: LSP 플러그인 + Hook 자동화 + notes 디렉토리

한 번에 다 하려고 하지 마세요. 하나씩 습관화하면 자연스럽게 Boris 레벨에 도달합니다.

CLAUDE.md 실전 사례: 마케팅 에이전시

알잘스튜디오(20인 디자인 에이전시)에서 실제로 사용하는 CLAUDE.md 구조를 공유합니다.

```
# CLAUDE.md — 알잘스튜디오 프로젝트

## 프로젝트 정보
- 서비스: 병원 마케팅 대행 (블로그, 유튜브, 광고)
- 기술 스택: React + Next.js + Supabase
- 배포: Vercel

## 작업 규칙
- 한국어, 해요체
- 3단계 이상 → Plan Mode 필수
- 코드 수정 전에 반드시 설명
- 서브에이전트로 탐색 후 구현

## 마케팅 콘텐츠 규칙
- 고객 pain point부터 시작
- 숫자/데이터 근거 필수
- CTA는 명확하고 구체적으로
- A/B 테스트 가능한 구조로 제작

## 병원 콘텐츠 특수 규칙
- 의료법 광고 규정 준수
- 과장/허위 표현 절대 금지
- 시술 부작용 반드시 명시
- 전후 사진 사용 불가

## 코드 품질
- TypeScript strict mode
- 컴포넌트는 50줄 이내
- 모든 API에 에러 핸들링
- Lighthouse 점수 90+ 유지

## 교훈 (2026년 3월 기준, 47개 축적)
- 네이버 SEO: 제목 40자 이내, 소제목에 키워드
- Supabase RLS 정책 빠뜨리면 데이터 노출 위험
- Next.js Image 컴포넌트 미사용 시 Lighthouse 감점
- 카카오 공유 시 og:image 1200x630 필수
- Vercel 환경변수 설정 누락 주의
- ...(42개 더)
```

> 💡 **꿀팁**
>
> **교훈 47개의 힘:** 3개월간 매일 Claude와 작업하면서 실수할 때마다 추가한 결과입니다. 이제 Claude는 병원 마케팅 프로젝트에서 거의 실수하지 않습니다. 여러분도 1주일에 5~10개씩 쌓으면 한 달이면 체감됩니다.

Boris의 프롬프팅 스타일 상세 분석

Boris는 Claude를 "부드럽게 부탁하는" 스타일이 아닙니다. 놀랍게도 **도전적이고 공격적**입니다. 이게 더 나은 결과를 만듭니다.

패턴 1: 증명을 요구한다

```
# Boris 스타일
> 이게 맞다고 증명해봐. 테스트를 돌려서 보여줘.
> 이 변경사항이 기존 기능을 깨뜨리지 않는다는 걸
  어떻게 보장할 수 있어?

# 일반적인 스타일 (덜 효과적)
> 잘 된 것 같아. 다음으로 넘어가자.
```

패턴 2: 다시 하라고 한다

```
> 지금까지 한 거 다 버려.
  네가 지금까지 배운 것을 기반으로,
  처음부터 우아한 방법으로 다시 구현해.
```

이 프롬프트가 강력한 이유: Claude는 첫 시도에서 배운 것을 활용해서 **두 번째 시도에서 훨씬 나은 코드**를 작성합니다.

패턴 3: 역할을 부여하고 기준을 높인다

```
> 이 코드를 시니어 엔지니어 관점에서 리뷰해줘.
  내가 너의 테스트를 통과할 때까지 PR을 올리지 마.

> 이 아키텍처를 면밀히 검토해.
  1년 후에 사용자가 100배 늘어도 버틸 수 있어?
  아니면 어디서 병목이 생기는지 알려줘.
```

패턴 4: 마이크로매니징하지 않는다

```
# Boris가 하지 않는 것
> 15번 줄의 변수명을 x에서 count로 바꾸고,
  23번 줄에 null 체크를 추가하고...

# Boris가 하는 것
> 이 버그 고쳐. (에러 메시지 붙여넣기)
  어떻게 고칠지는 네가 판단해.
```

Claude에게 **"무엇을"** 시키되, **"어떻게"**는 Claude가 결정하게 합니다. 이게 더 나은 결과를 만듭니다.

Boris 스타일 프롬프팅 요약

① 증명을 요구하라 — '이게 맞다고 증명해봐'

② 다시 하게 하라 — '다 버리고 우아하게 다시 해'

③ 기준을 높여라 — '시니어가 이걸 승인할까?'

④ 마이크로매니징하지 마라 — '뭘' 시키되 '어떻게'는 맡겨라

Plan Mode 심화 활용법

Boris 팀의 가장 강력한 습관 중 하나입니다. 단순히 "계획 세워줘"가 아니라, Plan Mode를 전략적으로 활용하는 방법입니다.

활용법 1: 계획 → 리뷰 → 실행 (Two-Claude Pattern)

```
# 세션 1: 계획 수립
> [Plan Mode]
> 회원 인증 시스템을 설계해줘.
  JWT + 리프레시 토큰 방식으로.
  각 파일별로 뭘 만들지 상세하게.

# 세션 2: 계획 리뷰 (별도 터미널)
> 이 계획을 시니어 백엔드 엔지니어 관점에서
  리뷰해줘. 보안 취약점이나 설계 결함이 있는지.
  [세션 1의 계획을 붙여넣기]

# 리뷰 반영 후 실행
> [일반 모드]
> 리뷰를 반영한 최종 계획대로 구현해줘.
```

활용법 2: 디버깅에서 Plan Mode

```
> [Plan Mode]
> 이 에러를 분석해줘. 코드는 건드리지 말고,
  가능한 원인 3가지와 각각의 확인 방법을 알려줘.

Claude: 원인 분석...
1. API 응답 형식 변경 → GET /api/user 응답 확인
2. 환경변수 누락 → .env 파일 확인
3. DB 스키마 불일치 → migration 상태 확인

> 2번이 맞을 것 같아. 확인하고 고쳐줘.
[일반 모드로 전환]
```

활용법 3: 리팩토링 전 영향도 분석

```
> [Plan Mode]
> auth.js 파일을 리팩토링하려고 해.
  이 파일을 수정하면 영향받는 파일 목록과
  각각 어떤 변경이 필요한지 먼저 분석해줘.
  코드는 아직 건드리지 마.
```

ℹ 알아두기

Plan Mode의 핵심 가치: 코드를 건드리기 전에 '지도'를 그리는 것입니다. 지도 없이 길을 나서면 돌아오는 시간이 2배 걸립니다. Plan Mode에 10분 투자하면, 구현에서 30분을 아낍니다.

Slash Command 실전 활용

Boris는 매일 반복하는 모든 워크플로우를 Slash Command로 만들어놨습니다. .claude/commands/ 폴더에 마크다운 파일을 넣으면 됩니다.

필수 Command 5종

1. /commit — 커밋+푸시+PR

```
# .claude/commands/commit.md

현재 변경사항을 분석해서:
1. 의미 있는 커밋 메시지를 한국어로 작성
2. git add + commit
3. git push
4. (선택) PR이 필요하면 생성

커밋 메시지 규칙:
- [feat] 새 기능
- [fix] 버그 수정
- [refactor] 리팩토링
- [docs] 문서
- [style] 스타일링
```

2. /review — 코드 리뷰

```
# .claude/commands/review.md

현재 변경사항을 시니어 엔지니어 관점에서 리뷰:
1. 버그 가능성
2. 보안 취약점
3. 성능 이슈
4. 코드 가독성

규칙:
- 중요한 것만 지적 (nitpick 금지)
- 각 이슈에 심각도 표시 (🔴 높음 / 🟡 중간 / 🟢 낮음)
- 수정 방법도 함께 제안
```

3. /blog — 블로그 글 작성

```
# .claude/commands/blog.md

주제: $ARGUMENTS

1. SEO 키워드 3개 추출
2. 제목 5개 제안 (숫자+키워드 조합)
3. 사용자 선택 후 본문 작성 (1,500~2,500자)
4. 메타 설명 (155자 이내)
5. 내부 링크 3개 추천

스타일: 해요체, 비유 2개+, 도입 hook 3줄 이내
```

4. /fix — 에러 수정

```
# .claude/commands/fix.md

에러 수정 워크플로우:
1. 에러 메시지 분석
2. 원인 코드 찾기
3. 수정 방법 설명 (1줄)
4. 수정 실행
5. 테스트 실행으로 확인
6. 같은 에러 재발 방지를 위해 CLAUDE.md에 교훈 추가
```

5. /start — 새 세션 시작

```
# .claude/commands/start.md

새 작업 세션 시작 체크리스트:
1. 프로젝트 구조 빠르게 파악 (서브에이전트 사용)
2. 최근 변경사항 확인 (git log -5)
3. 열려 있는 이슈 확인
4. 오늘 할 작업 계획 수립 (Plan Mode)
5. 계획 확정 후 실행
```

> 💥 **꿀팁**
>
> Command를 만들 때 `$ARGUMENTS` 를 넣으면, /blog 뒤에 적는 텍스트가 자동으로 들어갑니다. 예: `/blog AI 마케팅` → $ARGUMENTS = 'AI 마케팅'

CLAUDE.md가 안 되는 것 vs Hook이 되는 것

Boris의 팁 중 매우 중요하지만 놓치기 쉬운 것: **CLAUDE.md는 가이드라인이고, Hook은 법률입니다.**

규칙	CLAUDE.md	Hook	추천
한국어로 대화	✅ 80% 따름	❌ 불가능	CLAUDE.md
코드 포맷팅	△ 가끔 빠뜨림	✅ 100% 실행	**Hook**
.env 파일 보호	△ 무시할 수 있음	✅ 완전 차단	**Hook**
비유로 설명	✅ 대부분 따름	❌ 불가능	CLAUDE.md
테스트 실행	△ 가끔 빠뜨림	✅ 커밋 전 자동	**Hook**
Plan Mode로 시작	✅ 대부분 따름	❌ 불가능	CLAUDE.md
알림 보내기	❌ 불가능	✅ 자동 실행	**Hook**

> ℹ 알아두기
>
> **판단 기준:**
> - 스타일/태도/방법론 → CLAUDE.md
> - 절대 빠뜨리면 안 되는 것 → Hook
> - 외부 서비스 연동 → Hook (http 타입)

Boris의 일일 루틴 재현하기

Boris가 Anthropic에서 하루를 어떻게 보내는지 복원한 타임라인입니다. 여러분도 이 패턴을 자기 업무에 맞게 적용할 수 있습니다.

시간	Boris의 행동	여러분의 적용
출근 전	폰으로 세션 시작, 오늘 할 일 정리	Claude 앱에서 오늘 계획 세우기
오전	5개 터미널 탭에서 병렬 작업 시작	2개 세션으로 메인+보조 작업
점심 전	Plan Mode로 오후 작업 설계	큰 작업 계획 수립
오후	구현 집중, 웹에서 추가 세션	계획대로 구현, 에러 수정
저녁	코드 리뷰, PR 생성, lessons.md 업데이트	커밋, CLAUDE.md에 교훈 추가

💡 꿀팁

왕초보 버전 일일 루틴:
아침: Claude 시작 → CLAUDE.md 확인
오전: 메인 작업 1개 (Plan Mode → 구현)
점심: /clear로 리셋
오후: 메인 작업 1개 더
저녁: 커밋 + CLAUDE.md에 오늘의 교훈 추가

모델 선택 가이드: Opus vs Sonnet vs Haiku

Claude Code에서 사용할 수 있는 3가지 모델의 실전 비교입니다.

	Opus (최강)	Sonnet (균형)	Haiku (경량)
속도	느림	보통	빠름
품질	최고	높음	보통
비용	높음	중간	낮음
조종 필요	적음	보통	많음
추천 용도	설계, 복잡한 구현	일반 코딩, 수정	탐색, 간단한 질문

Boris의 선택: Opus 기본

> *Opus는 Sonnet보다 느리지만, 조종할 필요가 적어서 실제로는 더 빠릅니다. 한 번에 맞추니까요.*
>
> — Boris Cherny

실전 추천 전략

```
# 모델 전환 명령
> /model opus     # 복잡한 설계/구현
> /model sonnet   # 일반 작업
> /model haiku    # 간단한 질문
> /model opusplan # 최적 조합 (계획=Opus, 실행=Sonnet)
```

비용별 추천

월 예산	추천 전략
$20 (Pro)	기본 설정 그대로 (자동 전환)
$100 (Max 5x)	Opus 기본 + 헤비한 날만 Sonnet
$200 (Max 20x)	Opus 올인 + 병렬 세션 마음껏
API $50~100	Sonnet 기본 + 중요 작업만 Opus
API $100~200	Opusplan 모드 (최적 가성비)

ℹ 알아두기

Pro 구독($20/월)을 사용한다면 비용 걱정은 거의 없습니다. 사용량이 많으면 속도 제한(Rate Limit)이 걸릴 수 있는데, 이때 /clear를 자주 쓰고 불필요한 탐색을 줄이면 도움이 됩니다.

실전 디버깅 프레임워크

에러가 나면 당황하지 말고, 이 5단계 프레임워크를 따르세요.

1 에러 메시지 전체를 Claude에게 붙여넣기

에러 메시지를 절대 요약하지 마세요. 통째로 복사+붙여넣기 하면 Claude가 정확히 파악합니다.

```
> 이 에러를 분석하고 고쳐줘:
[에러 메시지 전체 복사]
```

2 맥락 제공

에러가 언제, 어디서, 어떤 상황에서 발생했는지 알려주세요.

```
> 이 에러는 로그인 버튼을 클릭했을 때 발생해.
  어제까지는 잘 됐는데 오늘 갑자기 안 돼.
```

3 이전 시도 공유

이미 시도한 해결법이 있다면 알려주세요. Claude가 같은 시도를 반복하지 않습니다.

```
> 이미 시도한 것:
  - npm install 다시 실행 → 안 됨
  - 캐시 삭제 → 안 됨
```

4 고친 후 검증 요청

단순히 '고쳐줘'가 아니라 검증까지 요청하세요.

```
> 고친 다음에 다시 실행해서 에러가 사라졌는지 확인해줘.
```

5 교훈 추가

같은 에러를 반복하지 않도록 CLAUDE.md에 기록합니다.

```
> 이 에러의 원인과 해결법을 CLAUDE.md 교훈에 추가해줘.
```

에러 해결이 안 될 때: 리셋 전략

같은 에러를 3번 이상 시도해도 안 고쳐지면, 접근을 바꿔야 합니다.

```
# 전략 1: 리셋하고 다시 시작
> 지금까지 시도한 것 전부 되돌려.
  에러의 근본 원인부터 다시 분석해줘.
  다른 접근 방법으로 해결해.

# 전략 2: 서브에이전트에게 리서치 맡기기
> 서브에이전트로 이 에러에 대해 리서치해줘.
  비슷한 사례에서 어떻게 해결했는지 찾아서 정리해줘.

# 전략 3: Plan Mode로 전환
> [Plan Mode]
> 이 에러를 해결하는 3가지 다른 접근법을
  각각의 장단점과 함께 제안해줘.
  코드는 건드리지 말고.
```

> ⚠ **주의**
> 같은 에러를 5번 이상 시도해도 안 되면, `/clear` 로 완전히 새로 시작하는 것이 가장 빠릅니다. 이전 대화의 잘못된 맥락이 Claude를 오히려 방해할 수 있습니다.

컨텍스트 관리 마스터 가이드

Claude Code의 가장 중요한 기술 중 하나입니다. 컨텍스트를 잘 관리하면 비용이 줄고 품질이 올라갑니다.

컨텍스트란 무엇인가?

Claude가 한 번에 기억할 수 있는 **대화 + 파일 + 규칙의 총량**입니다. 약 200,000 토큰으로, 영어로 15만 단어, 한국어로 약 5~7만 자 정도입니다.

컨텍스트는 이렇게 채워집니다

요소	소비량	줄이는 방법
CLAUDE.md	매 세션 고정	100줄 이내로 유지
MCP 도구 정의	MCP당 2~5K	안 쓰는 MCP 끄기
대화 이력	누적	/clear, /compact
파일 읽기	파일당 수백~수천	필요한 파일만 참조
Claude의 응답	누적	간결한 응답 요청

컨텍스트 관리 전략

1. **50% 규칙**
   AUTOCOMPACT_PCT를 50으로 설정하면 절반만 차도 자동 압축이 시작됩니다.
2. **주제 전환 시 /clear**
   새로운 작업을 시작할 때 이전 대화를 완전히 지웁니다.
3. **탐색은 서브에이전트에게**
   파일을 많이 읽어야 하는 작업은 서브에이전트에게 맡깁니다.
4. **@ 멘션으로 정확히 지정**
   @filename.js로 특정 파일만 참조하면 불필요한 파일 로드를 방지합니다.
5. **긴 작업은 세션 분리**
   한 세션에서 3가지 이상 큰 작업을 하지 않습니다.

```
# @ 멘션 사용법
> @src/auth.js 의 login 함수를 수정해줘.
  비밀번호 검증 로직을 추가해.

# 서브에이전트 활용
> 서브에이전트로 src/ 폴더의 모든 파일을 훑어보고
  인증 관련 파일만 목록으로 정리해줘.
```

컨텍스트 = 돈 + 품질

컨텍스트를 절약하면:

① 토큰 비용이 줄어듭니다

② Claude의 응답 품질이 올라갑니다 (집중력 향상)

③ Rate Limit에 덜 걸립니다

④ 세션이 더 오래 유지됩니다

Output Style 커스터마이징

Boris의 팁 #28입니다. Claude의 응답 스타일을 커스터마이징할 수 있습니다.

```
# ~/.claude/output-styles/concise.md

# 간결한 스타일
- 답변은 3줄 이내
- 코드만 보여주고 설명은 최소화
- "했습니다" 대신 바로 결과
- 이모지 사용 금지
```

```
# ~/.claude/output-styles/teacher.md

# 교육자 스타일
- 모든 단계를 상세히 설명
- 비유를 적극 활용
- 전문 용어 사용 시 괄호 안에 쉬운 설명
- 각 단계마다 "왜 이렇게 하는지" 설명
- 실수하기 쉬운 포인트를 미리 경고
```

```
# 사용법
> /style concise   # 간결하게
> /style teacher   # 교육자 모드
```

settings.json 완전 가이드

Claude Code의 모든 설정을 관리하는 파일입니다. 위치는 두 곳입니다.

위치	범위	용도
`~/.claude/settings.json`	모든 프로젝트	글로벌 설정
`.claude/settings.json`	해당 프로젝트만	프로젝트 설정

추천 글로벌 설정

```
{{
  "model": "opus",
  "permissions": {{
    "allow": [
      "Read(**)",
      "Edit(**)",
      "Bash(npm test*)",
      "Bash(npx prettier*)",
      "Bash(git *)"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Edit(.env*)"
    ]
  }},
  "env": {{
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }},
  "hooks": {{
    "Notification": [{{
      "hooks": [{{
        "type": "command",
        "command": "osascript -e 'display notification \\"$NOTIFICATION_MESSAGE\\" with title \\"Claude\\"'"
      }}]
    }}]
  }}
}}
```

> ⚠ **주의**
> settings.json을 수정한 후에는 Claude Code를 재시작해야 적용됩니다. 터미널에서 `exit` → `claude` 로 다시 시작하세요.

멀티머신 설정 동기화

회사 PC, 집 PC, 노트북에서 같은 Claude Code 설정을 사용하는 방법입니다.

방법 1: Git으로 동기화 (추천)

```
# Claude 설정을 Git 리포로 관리
mkdir ~/claude-config && cd ~/claude-config
git init

# 설정 파일 복사
cp ~/.claude/settings.json .
cp ~/.claude/CLAUDE.md .
cp -r ~/.claude/commands/ .

# GitHub에 올리기
git add . && git commit -m "Claude 설정 동기화"
git remote add origin [GitHub URL]
git push

# 다른 PC에서 가져오기
git clone [GitHub URL] ~/claude-config
cp ~/claude-config/settings.json ~/.claude/
cp ~/claude-config/CLAUDE.md ~/.claude/
```

방법 2: 심볼릭 링크 + 클라우드 동기화

```
# Google Drive 또는 Dropbox에 설정 폴더 만들기
mkdir -p ~/GoogleDrive/claude-config
cp ~/.claude/settings.json ~/GoogleDrive/claude-config/

# 심볼릭 링크 연결
ln -sf ~/GoogleDrive/claude-config/settings.json ~/.claude/settings.json
```

팀에서 Claude Code 도입하기

혼자 쓰다가 팀에 도입할 때의 전략입니다.

1. **개인 성공 사례 만들기**
   먼저 혼자서 2~4주 사용하며 확실한 성과를 만드세요. '이걸로 00을 만들었는데, 평소 3일 걸리는 게 3시간이면 됩니다.'같은 구체적인 사례.
2. **팀장/대표에게 데모**
   실제로 Claude Code로 무언가를 만드는 과정을 시연합니다. 30분이면 충분합니다.
3. **파일럿 팀 구성**
   관심 있는 2~3명으로 시작. 전체 도입은 파일럿 성공 후.
4. **팀 CLAUDE.md 만들기**
   팀 공통 규칙을 Git에 커밋. 프로젝트별 CLAUDE.md를 표준화.
5. **비용 관리 프로세스**
   팀 전체 사용량을 추적하고 월별 리포트를 만듭니다.

ℹ 알아두기

Boris 팀은 CLAUDE.md를 Git에 커밋합니다. 한 사람이 발견한 교훈이 팀 전체의 자산이 되고, PR 리뷰를 통해 교훈의 품질도 검증됩니다.

Boris의 팁 상세 해설: 16~25번

앞서 Top 15를 다뤘습니다. 나머지 팁 중 왕초보에게도 유용한 10가지를 추가로 해설합니다.

팁 #16: 이미지로 컨텍스트를 제공하세요

텍스트로 설명하기 어려운 것은 **스크린샷**을 보여주세요. Claude Code는 이미지를 이해합니다.

```
# Mac: Cmd+Shift+4로 캡처 후 붙여넣기
# Windows: Win+Shift+S로 캡처 후 붙여넣기

> 이 디자인처럼 만들어줘. [이미지 붙여넣기]

> 이 에러 화면 보여줄게. 어떻게 고쳐야 해?
  [에러 스크린샷 붙여넣기]
```

💡 꿀팁

Figma나 노트에 그린 손그림도 가능합니다. UI 레이아웃을 손으로 그려서 사진 찍고 Claude에게 보여주면, 그 구조를 이해하고 코드로 만들어줍니다.

팁 #17: 에러 메시지를 통째로 붙여넣으세요

에러 메시지를 요약하지 마세요. 통째로 복사해서 붙여넣으면 Claude가 더 정확하게 진단합니다.

```
# ✗ 요약하면 정보가 손실됨
> 로그인할 때 401 에러가 나요.

# ✓ 통째로 붙여넣기
> 이 에러 고쳐줘:
  Error: Request failed with status code 401
    at createError (createError.js:16:15)
    at settle (settle.js:17:12)
    at XMLHttpRequest.handleLoad (xhr.js:62:7)
  Response: {{"message":"Invalid token","code":"AUTH_EXPIRED"}}
```

팁 #18: 구체적으로 지시하세요 (파일명/함수명 명시)

```
# ✗ 모호한 지시
> 인증 관련 코드를 수정해줘.

# ✓ 구체적 지시
> @src/auth/login.ts의 validateToken 함수에서
  토큰 만료 시간 검사 로직을 수정해줘.
  다른 파일은 건드리지 마.
```

팁 #19: 한 세션 한 주제

로그인 기능을 만들다가 갑자기 "이메일 템플릿도 만들어줘"라고 하면, Claude는 두 작업의 컨텍스트가 섞여서 품질이 떨어집니다.

```
# 주제가 바뀌면
> /clear
> 이제 이메일 템플릿을 만들자.
```

팁 #20: Esc+Esc = 만능 되돌리기

Claude가 잘못된 방향으로 갈 때:

```
# 1. Esc로 즉시 중단
# 2. Esc+Esc로 이전 상태 복원
# 3. 다른 접근으로 다시 시도

> 아까 접근은 틀렸어.
  이번에는 [다른 방법]으로 해줘.
```

> **ℹ 알아두기**
> Boris는 Esc+Esc를 하루에 수십 번 씁니다. 40% 확신인 접근도 부담 없이 시도하고, 아니면 바로 되돌립니다. 이 습관이 실험 속도를 크게 높입니다.

팁 #21: Claude iOS 앱으로 아침에 시작

Boris는 출근 전 폰에서 Claude 앱을 열고 오늘 할 일 계획을 세웁니다. 출근하면 터미널로 세션을 텔레포트합니다.

```
# 폰에서 (Claude iOS/Android 앱)
> 오늘 할 일:
  1. 로그인 버그 수정
  2. 대시보드 차트 추가
  3. README 업데이트

  각각 어떤 파일을 수정해야 하는지
  간단히 정리해줘.

# 출근 후 터미널에서
> /teleport ← 폰 세션을 터미널로 가져오기
```

팁 #22: Plan Mode에서 검증 단계도 계획

```
> [Plan Mode]
> 결제 기능을 만드는 계획을 세워줘.
  구현 단계뿐만 아니라,
  각 단계의 검증 방법도 함께 계획해.
  예: "1단계 완료 → 어떤 테스트로 확인?"
```

팁 #23: notes 디렉토리로 지식 축적

```
# CLAUDE.md에 추가
- 모든 작업에 대해 notes/ 디렉토리에 메모를 남겨줘
- 메모에는: 작업 내용, 결정 이유, 배운 점을 포함
- 중요한 결정의 이유(WHY)를 반드시 기록
```

시간이 지나면 notes/ 폴더가 프로젝트의 '역사서'가 됩니다. 나중에 왜 이렇게 만들었는지 Claude에게 물어보면 notes/를 참조해서 답해줍니다.

팁 #24: 터미널 설정 최적화

```
> /terminal-setup
```

이 명령 하나로 터미널 설정이 최적화됩니다. 특히 **Shift+Enter로 줄바꿈**이 가능해지는데, 긴 프롬프트를 여러 줄로 작성할 때 필수입니다.

팁 #25: High Effort 모드 사용

```
> /config set effort high
```

Boris는 항상 High Effort 모드를 씁니다. Claude가 더 꼼꼼하게 작업하므로 재작업이 줄어듭니다.

팁 16~25 요약

① 스크린샷으로 보여줘라
② 에러 메시지 통째로 붙여넣어라
③ 파일명/함수명을 콕 집어라
④ 한 세션 한 주제
⑤ Esc+Esc는 만능 되돌리기
⑥ 폰으로 시작, 터미널로 이어받기
⑦ Plan Mode에서 검증도 계획
⑧ notes/로 지식 축적
⑨ /terminal-setup 실행
⑩ High Effort 모드 켜기

PART 3

해커톤 우승자들의 비밀 무기

Anthropic 공식 해커톤에서 우승한 팀들은
어떤 설정과 전략을 썼을까?
50K 스타 리포부터 비개발자 우승 사례까지.

CHAPTER 06

Everything Claude Code — 50K 스타 리포 해부

해커톤에서 태어난 오픈소스

2025년 9월, Anthropic x Forum Ventures 해커톤(뉴욕). Affaan Mustafa와 팀메이트는 **8시간 만에 zenith.chat**

이라는 서비스를 완성하고 $15,000 API 크레딧을 수상했습니다.

비결은 10개월 이상 매일 Claude Code를 사용하면서 다듬어온 설정 파일과 워크플로우입니다. 그리고 이걸 통째로 오픈소스로 공개했습니다.

숫자로 보는 Everything Claude Code

☆ 50,000+ GitHub 스타 | 🍴 6,000+ 포크 | 👥 30명 기여자 | 🌍 7개 언어 지원

단순한 설정 모음이 아닙니다. 스킬, 본능(instincts), 메모리 최적화, 보안 스캐닝까지 포함한 **완전한 시스템**입니다.

이 리포가 특별한 이유

일반적인 Claude Code 설정 공유는 "내 settings.json 공유합니다" 수준입니다. Everything Claude Code는 완전히 다릅니다.

일반 설정 공유	Everything Claude Code
settings.json 하나	스킬 + 에이전트 + 훅 + 보안 스캐너 통합
"이렇게 해보세요"	Shorthand/Longform/Security 3종 가이드
개인 경험 기반	해커톤 실전 + 프로덕션 검증
설치법 없음	npx 한 줄로 스캔/수정/초기화

핵심 설정 #1: 토큰 절약

```
{{
  "model": "sonnet",
  "env": {{
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }}
}}
```

MAX_THINKING_TOKENS=10000: Claude의 "생각" 시간을 제한해 비용 절감. **AUTOCOMPACT=50**: 컨텍스트 50%만 차도 자동 압축 시작.

핵심 설정 #2: Strategic Compact 스킬

기본 자동 압축은 95%에서 동작합니다. 너무 늦습니다. 이 스킬은 **논리적 전환점**(기능 완료, 주제 변경)에서 미리 /compact를 제안합니다.

⚠ 주의

MCP 서버를 한꺼번에 다 켜지 마세요. 각 MCP 도구 설명이 토큰을 잡아먹습니다. 200K 윈도우가 MCP만으로 70K로 줄어들 수 있습니다.

왕초보가 5분 만에 적용하는 법

1 GitHub 방문

github.com/affaan-m/everything-claude-code 접속

2 보안 스캔 실행

현재 내 Claude Code 설정이 안전한지 진단합니다.

```
# 터미널에서 한 줄 실행
npx ecc-agentshield scan

# 결과 예시:
✓ settings.json - 3 issues found
  ⚠ MCP server count: 8 (recommend max 3-4)
  ⚠ No AUTOCOMPACT override set
  ✓ Permission mode: ask (safe)
```

3 자동 수정

안전한 문제는 한 번에 고칩니다.

```
npx ecc-agentshield scan --fix

✓ Applied 2 safe fixes
  ✓ Added AUTOCOMPACT_PCT_OVERRIDE=50
  ✓ Added MAX_THINKING_TOKENS=10000
```

4 설정 확인

수정된 설정을 확인합니다.

```
cat ~/.claude/settings.json
```

핵심 파일 구조 해설

파일/폴더	역할	왕초보 필요도
settings.json	기본 설정 (모델, 토큰 제한, 권한)	★★★ 필수
CLAUDE.md	프로젝트 규칙	★★★ 필수
skills/	커스텀 스킬 모음	★★ 추천
hooks/	자동화 규칙	★★ 추천
agents/	에이전트 정의	★ 나중에
AgentShield/	보안 스캐너	★★ 추천

💡 꿀팁

처음에는 **settings.json + CLAUDE.md**만 가져와서 적용하세요. 나머지는 필요할 때 하나씩 추가하면 됩니다. 한꺼번에 다 적용하면 오히려 혼란스럽습니다.

CHAPTER 07

비개발자가 해커톤에서 우승한 실화

변호사가 500명의 개발자를 이겼다

2026년 2월, Anthropic의 'Built with Opus 4.6' 해커톤. 13,000명이 지원하고 500명이 선발된 이 대회에서, **1등은 캘리포니아의 변호사 Mike Brown**이었습니다.

Mike의 친구는 뒷마당 별채(ADU)를 짓는 사업을 합니다. 허가증 신청이 수개월씩 걸리는 이유는 건축 도면 문제가 아니라, 코드 인용 오류나 주/지역 규정 충돌 같은 **문서 문제** 때문이었습니다.

Mike는 6일 만에 이 문제를 해결하는 앱을 만들었습니다. 코드는 한 줄도 직접 쓰지 않았습니다.

> *가장 가치 있는 AI 스킬은 코드가 아니다. 진짜 문제를 깊이 이해하는 것이다. Mike는 허가 절차의 고통을 뼛속까지 알고 있었다. 그게 500명의 개발자가 가지지 못한 것이었다.*
>
> — 2nd Order Thinkers 분석

다른 우승자들의 공통점

우승자	직업	만든 것	핵심 무기
Mike Brown	변호사	건축 허가 처리 앱	규제 도메인 전문성
브뤼셀 의사	심장내과	환자 추적 도구	의료 워크플로우 이해
우간다 기술자	도로 엔지니어	인프라 평가 시스템	현장 문제 파악
Affaan Mustafa	개발자	zenith.chat	10개월 설정 축적

비개발자 우승자들에게는 공통점이 있습니다.

- **진짜 문제를 알고 있었다** — 본인이 또는 주변 사람이 겪는 구체적인 고통
- **해결책의 형태를 알고 있었다** — "이런 게 있으면 좋겠다"가 명확했음
- **Claude Code에게 잘 설명할 수 있었다** — 도메인 용어와 맥락을 정확히 전달

AI 생산자의 공식

도메인 전문성 × Claude Code = 비개발자도 서비스를 만든다

이 공식이 에이나우가 말하는 'AI 생산자'의 핵심입니다. 코딩을 배우는 게 아닙니다. 여러분이 이미 가지고 있는 전문성을 Claude Code로 증폭시키는 겁니다.

Mike Brown의 실전 프로세스 분석

Mike가 해커톤에서 실제로 어떻게 작업했는지 복원해봅니다.

1. **문제 정의 (Day 1)**
   건축 허가 거절의 80%는 코드 인용 오류. 지역별 예외 규정이 너무 복잡해서 전문 변호사도 자주 실수. → '법규 데이터베이스를 검색하고, 올바른 인용을 자동 생성하는 도구'
2. **데이터 수집 (Day 1-2)**
   캘리포니아 건축 코드, 지역별 조례, 허가 신청 양식을 Claude에게 제공. 자신의 법률 지식으로 데이터의 우선순위와 관계를 설명.
3. **Plan Mode로 설계 (Day 2-3)**
   Claude와 Plan Mode에서 앱 구조를 설계. 변호사답게 요구사항을 법적 정밀도로 작성.
4. **구현 (Day 3-5)**
   Claude가 코드를 작성하는 동안, Mike는 법적 정확성을 검증. 코드 품질은 Claude가, 법적 품질은 Mike가 담당.
5. **테스트 (Day 5-6)**
   실제 허가 거절 사례를 입력해보며 앱의 제안이 맞는지 검증. 본인의 법률 경험이 최고의 테스트 케이스.

당신의 해커톤은 내일 시작된다

해커톤에 나가지 않아도 됩니다. 여러분의 일상이 이미 해커톤입니다.

당신의 전문성	만들 수 있는 것	참고 챕터
마케터	리드 수집 랜딩페이지, 퍼널 자동화	Ch.12
크리에이터	블로그 자동화, 콘텐츠 리퍼포징	Ch.13
병원 담당자	블로그 SEO 자동화, 시술 비교 콘텐츠	Ch.14
사업가	MVP 프로토타입, 데이터 대시보드	Ch.15
교육자	커리큘럼 생성기, 퀴즈 자동화	부록
디자이너	포트폴리오 사이트, 디자인 시스템	부록

CHAPTER 08

AgentShield — 보안 점검 1분 가이드

왜 보안 점검이 필요한가

Claude Code는 여러분의 컴퓨터에서 **파일을 읽고, 쓰고, 명령어를 실행**합니다. 강력한 만큼, 잘못된 설정은 보안 위험이 될 수 있습니다.

AgentShield는 2026년 2월 Cerebral Valley x Anthropic 해커톤에서 만들어진 보안 스캐너입니다. Everything Claude Code 리포에 포함되어 있으며, **1,282개 테스트, 98% 커버리지, 102개 정적 분석 규칙**으로 구성됩니다.

무엇을 스캔하나

검사 영역	검사 내용	위험도
비밀 정보 감지	API 키, 비밀번호가 설정에 노출되었는지 (14개 패턴)	🔴 높음
권한 감사	불필요하게 넓은 권한이 부여되었는지	🟠 중간
Hook 인젝션	악성 스크립트가 Hook에 삽입되었는지	🔴 높음
MCP 서버 위험	신뢰할 수 없는 MCP 서버가 연결되었는지	🟠 중간
에이전트 설정	에이전트 권한이 과도한지	🟠 중간

3단계로 끝나는 보안 점검

1. **빠른 스캔**
   현재 설정의 문제점을 진단합니다.

```
npx ecc-agentshield scan
```

2. **자동 수정**
   안전한 문제는 자동으로 수정합니다.

```
npx ecc-agentshield scan --fix
```

3. **안전한 설정 초기화 (선택)**
   처음부터 안전한 설정으로 시작하고 싶다면:

```
npx ecc-agentshield init
```

> 💡 **꿀팁**
> 보안 스캔은 **한 달에 한 번** 정도 실행하는 것을 추천합니다. 특히 새로운 MCP 서버를 연결하거나, 다른 사람의 설정을 가져왔을 때는 반드시 스캔하세요.

해커톤 참가자들의 공통 전략

여러 해커톤 우승자들의 인터뷰와 공개 자료를 분석하면, 공통적인 전략 패턴이 보입니다.

전략 1: 처음 2시간은 코딩하지 않는다

대부분의 우승자들은 해커톤 시작 후 처음 2시간을 **문제 정의와 설계**에 투자했습니다. Plan Mode로 Claude와 대화하며 스펙을 다듬었습니다.

> *해커톤에서 가장 큰 실수는 너무 빨리 코딩을 시작하는 것이다. 처음 2시간 동안 문제를 제대로 정의하면, 나머지 6시간이 훨씬 생산적이다.*
>
> — Affaan Mustafa, Everything Claude Code 창시자

전략 2: CLAUDE.md를 미리 준비한다

Affaan은 10개월간 다듬어온 CLAUDE.md와 설정 파일을 해커톤에 가져왔습니다. 처음부터 최적화된 상태에서 시작한 것이 큰 이점이었습니다.

전략 3: 검증 가능한 작은 단위로 나눈다

8시간 안에 완성해야 하므로, 각 단계마다 "이것까지 되면 데모 가능"한 마일스톤을 설정합니다.

시간	마일스톤	데모 가능?
0~2h	문제 정의 + 설계	-
2~4h	핵심 기능 1개 동작	✅ 최소 데모
4~6h	핵심 기능 3개 + UI	✅ 기본 데모
6~7h	전체 연동 + 테스트	✅ 완성 데모
7~8h	발표 준비 + 여유분	✅ 프레젠테이션

전략 4: 병렬 세션을 적극 활용한다

2인 팀이라면 각자 다른 부분을 동시에 만듭니다. Git Worktree로 충돌을 방지하면서 속도를 2배로 끌어올립니다.

해커톤 우승 공식

① 처음 2시간: Plan Mode로 문제+설계
② 사전 준비된 CLAUDE.md + 설정
③ 작은 마일스톤으로 나누기
④ 병렬 세션으로 속도 2배
⑤ 마지막 1시간은 발표 준비

Everything Claude Code: Selective Install 가이드

Everything Claude Code 리포를 통째로 가져오면 부담스러울 수 있습니다. **Manifest-driven install** 시스템으로 필요한 것만 골라서 설치할 수 있습니다.

왕초보 추천 세트 (3개만)

```
# 1. 보안 스캔 (가장 먼저)
npx ecc-agentshield scan --fix

# 2. 토큰 최적화 설정
# settings.json에 추가:
{{
  "env": {{
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }}
}}

# 3. strategic-compact 스킬 설치
# Claude Code에서:
> skills/strategic-compact/ 폴더를 만들고
   SKILL.md에 다음 내용을 넣어줘:
   "컨텍스트가 50% 이상 찼거나, 작업 주제가
    바뀔 때 /compact를 제안할 것"
```

중급자 추천 세트 (+3개)

```
# 4. Code Simplifier 에이전트
# Claude가 작업 후 코드를 간소화

# 5. 자동 포맷팅 Hook
# PostToolUse → Prettier 자동 실행

# 6. commit-push 슬래시 커맨드
# /commit으로 한 번에 커밋+푸시
```

고급자 추천 세트 (+3개)

```
# 7. LSP 플러그인 (언어별)
# 8. Verify App 에이전트 (E2E 테스트)
# 9. 커스텀 보안 Hook (파일 접근 제어)
```

> ⚠ **주의**
>
> 한 번에 모든 설정을 적용하면 오히려 역효과입니다. 왕초보 세트(3개)부터 시작하고, 2주마다 하나씩 추가하세요.

PART 4

GitHub 오픈소스로 10배 빠르게 성장하기

1,234개 이상의 커뮤니티 스킬,
플러그인 생태계, 그리고 GitHub을
왕초보가 활용하는 완전 가이드.

CHAPTER 09

커뮤니티 스킬 1,234개 활용 가이드

스킬이란? (1분 복습)

스킬은 Claude에게 특정 작업을 잘하는 방법을 알려주는 **레시피 파일**입니다. SKILL.md라는 마크다운 파일로 구성되

며, 안에는 작업 단계, 품질 기준, 주의사항 등이 담겨 있습니다.

```
# 스킬 사용 예시
> /blog AI 마케팅 트렌드 2026
# → blog 스킬이 로드되어 Claude가 SEO 최적화된 블로그 글을 작성

> /fix
# → fix 스킬이 로드되어 에러 진단/수정 워크플로우 실행
```

2026년 3월 기준: 스킬 생태계

구분	설명	규모	품질
공식 Anthropic	Anthropic이 직접 만듦	~20개	★★★
검증된 서드파티	커뮤니티 검증 완료	~100개	★★★
커뮤니티 기여	전 세계 개발자 제작	1,234개+	★~★★★

ℹ 알아두기

SKILL.md는 범용 포맷입니다. 같은 스킬 파일이 Claude Code, Cursor, Gemini CLI, Codex CLI, Antigravity IDE 등에서 모두 작동합니다. 한 번 만든 스킬은 어디서든 사용 가능합니다.

역할별 추천 스킬 Top 5

📣 마케터용

- **frontend-design** (공식, 277K+ 설치) — AI 느낌 안 나는 프로 수준 웹 디자인
- **seo-optimizer** — 콘텐츠 SEO 자동 최적화
- **landing-page** — 전환율 높은 랜딩페이지 생성
- **social-media** — SNS 콘텐츠 포맷 자동 변환
- **analytics-dashboard** — 데이터 시각화

🎨 크리에이터용

- **blog-writer** — 키워드 기반 블로그 자동 생성
- **youtube-script** — 영상 대본 + 구성
- **content-repurpose** — 하나의 콘텐츠 → 여러 포맷
- **newsletter** — 뉴스레터 자동 작성
- **card-news** — 카드뉴스 HTML 자동 생성

💼 사업가용

- **mvp-builder** — 최소 기능 제품 빠른 구현
- **api-design** — API 설계 자동화
- **lint-and-validate** — 코드 품질 자동 검사
- **create-pr** — Pull Request 자동 생성
- **security-audit** — 보안 취약점 점검

스킬 설치 3가지 방법

방법 1: 플러그인으로 설치 (가장 쉬움)

```
> /plugin install frontend-design@anthropic-official
> /plugin install seo-optimizer@community
```

방법 2: GitHub에서 직접 설치

```
npx skills add https://github.com/example/blog-writer
```

방법 3: 직접 만들기

```
# .claude/skills/my-blog/SKILL.md 생성

---
name: my-blog
description: "내 블로그 스타일에 맞는 글 작성"
---

# 블로그 글 작성 스킬

## 단계
1. 주제에서 SEO 키워드 3개 추출
2. 제목 3개 제안 (숫자 + 키워드 조합)
3. 본문 작성 (1,500~2,000자)
4. 메타 설명 작성 (155자 이내)

## 스타일
- 해요체
- 비유 최소 1개 포함
- 소제목마다 핵심 1줄 요약

## 금지
- 과장 표현
- 근거 없는 주장
```

> 💡 **꿀팁**
>
> **스킬 description은 트리거입니다.** '언제 이 스킬을 실행할지'를 Claude에게 알려주는 역할입니다. 당연한 내용을 적지 말고, Claude가 기본 동작과 다르게 행동해야 할 부분만 적으세요.

스킬 고급 구조

Boris가 공유한 팁: 스킬은 파일이 아니라 **폴더**입니다.

```
# 프로 수준 스킬 구조
.claude/skills/my-blog/
├── SKILL.md          # 메인 스킬 파일
├── references/       # 참고 자료
│   ├── style-guide.md
│   └── seo-checklist.md
├── examples/         # 좋은 예시 / 나쁜 예시
│   ├── good-post.md
│   └── bad-post.md
└── scripts/          # 자동화 스크립트
    └── check-seo.sh
```

`references/`, `examples/`, `scripts/`를 활용하면 Claude가 더 정확하게 작업합니다. 특히 examples/에 좋은 예시와 나쁜

예시를 넣으면 효과가 큽니다.

CHAPTER 10

플러그인 시스템 완전 정복

플러그인이란?

2026년 2월, Boris의 세 번째 팁 시리즈에서 공개된 기능입니다. 플러그인은 LSP(코드 자동 진단), MCP(외부 서비스 연결), 스킬, 에이전트, 커스텀 훅을 **한 번에 설치하고 관리**하는 시스템입니다.

```
# 플러그인 브라우저 열기
> /plugin

# 특정 플러그인 설치
> /plugin install typescript-lsp@claude-plugins-official
> /plugin install pyright-lsp@claude-plugins-official
```

Boris가 말하는 "임팩트 #1 플러그인": LSP

LSP(Language Server Protocol) 플러그인은 Claude가 파일을 수정할 때마다 **자동으로 코드 에러를 감지**합니다. 타입 에러, 미사용 변수, 누락된 import 등을 실시간으로 잡아냅니다.

언어	플러그인	설치 명령
TypeScript/JS	typescript-lsp	`/plugin install typescript-lsp@claude-plugins-official`
Python	pyright-lsp	`/plugin install pyright-lsp@claude-plugins-official`
Rust	rust-analyzer-lsp	`/plugin install rust-analyzer-lsp@claude-plugins-official`
Go	gopls-lsp	`/plugin install gopls-lsp@claude-plugins-official`
C#, Java, Kotlin 등	각 언어별 LSP	플러그인 브라우저에서 검색

꿀팁

왕초보도 LSP 플러그인은 꼭 설치하세요. 여러분이 직접 코드를 읽을 필요 없이, Claude가 자동으로 에러를 잡고 수정합니다. Boris가 '단일 플러그인 중 임팩트가 가장 크다'고 말한 이유입니다.

플러그인 관리

```
# 설치된 플러그인 목록
> /plugin list

# 플러그인 제거
> /plugin remove typescript-lsp

# 팀 공유: settings.json에 마켓플레이스 등록
# → 팀원이 /plugin 실행 시 회사 플러그인이 자동 표시
```

CHAPTER 11

GitHub 왕초보 가이드

GitHub이 뭔가요?

GitHub은 **코드를 저장하고 공유하는 플랫폼**입니다. 구글 드라이브가 문서를 저장하듯, GitHub은 코드를 저장합니다. 전 세계 1억 명 이상의 개발자가 사용하며, 좋은 코드/도구/스킬이 무료로 공개되어 있습니다.

> ℹ 알아두기
> 여러분이 GitHub에서 코드를 직접 읽을 필요는 없습니다. Claude Code에게 '이 리포를 내 프로젝트에 적용해줘'라고 시키면 됩니다. GitHub은 **좋은 자료를 찾는 곳**이라고 생각하세요.

GitHub 용어 5개만 알면 됩니다

용어	의미	비유
Repository (리포)	프로젝트 하나	구글 드라이브의 폴더 하나
Star (스타)	좋아요	인스타 좋아요 ❤
Fork (포크)	다른 사람 프로젝트 복사	레시피 복사해서 내 노트에
Clone (클론)	내 컴퓨터로 다운로드	구글 드라이브 파일 다운로드
README.md	프로젝트 설명서	제품 사용 설명서

좋은 리포를 찾는 법

- **Star 수**: 1,000개 이상이면 검증된 프로젝트
- **최근 업데이트**: 6개월 이내에 업데이트가 있어야 활성 프로젝트
- **README 품질**: 설치법, 사용법이 잘 적혀 있는지 확인
- **Issues/Discussions**: 질문에 답변이 달리는지 확인

리포를 Claude Code에 적용하는 패턴

```
# 1. 리포 URL을 Claude에게 공유
> 이 GitHub 리포의 설정을 내 프로젝트에 적용해줘:
  https://github.com/affaan-m/everything-claude-code
  특히 settings.json과 CLAUDE.md 부분만.

# 2. README 기반으로 적용 지시
> 이 리포의 README를 읽고,
  내 프로젝트에 맞게 설치해줘.

# 3. 특정 파일만 가져오기
> 이 리포에서 skills/ 폴더만 가져와서
  내 .claude/skills/에 복사해줘.
```

추천 GitHub 리포 Top 10

#	리포	Star	용도
1	everything-claude-code	50K+	설정/스킬/보안 통합 시스템
2	claude-code-best-practice	10K+	Boris 팁 + 커뮤니티 정리
3	awesome-claude-code-skills	22K+	1,234개 스킬 큐레이션
4	anthropic-cookbook	15K+	Anthropic 공식 예제
5	gws (Google Workspace CLI)	4.9K+	Google 서비스 통합 MCP
6	BMAD	5K+	AI SDLC 프레임워크
7	Claude Code GitHub App	공식	PR 자동 리뷰
8	MCP Servers (공식)	공식	공식 MCP 서버 모음
9	frontend-design skill	공식	프로 수준 웹 디자인
10	howborisusesclaudecode	커뮤니티	Boris 57가지 팁 통합

꿀팁

GitHub 계정이 없어도 리포를 볼 수 있습니다. 하지만 계정을 만들면 Star(좋아요)를 눌러 저장해두고 나중에 찾아볼 수 있습니다. 무료이니 만들어두세요.

스킬 만들기 마스터 클래스

남이 만든 스킬을 쓰는 것을 넘어서, 자신만의 스킬을 만드는 방법입니다.

좋은 스킬의 3가지 조건

1. **명확한 트리거**
   description에 '언제 이 스킬을 실행할지'를 정확히 적습니다. Claude가 자동으로 적절한 스킬을 선택하는 핵심입니다.
2. **구체적인 단계**
   모호한 지시 대신 정확한 순서와 기준을 적습니다.
3. **검증 규칙**
   완료 기준을 명시합니다. '이것까지 되어야 완료'를 적으면 품질이 올라갑니다.

스킬 작성 템플릿

```
# .claude/skills/[스킬이름]/SKILL.md

---
name: [스킬 이름]
description: "[언제 이 스킬을 사용할지 설명]"
---

# [스킬 이름]

## 입력
- $ARGUMENTS: [무엇을 받는지]

## 단계
1. [첫 번째 할 일]
2. [두 번째 할 일]
3. [세 번째 할 일]
...

## 품질 기준
- [이것이 되어야 완료]
- [이것도 확인]

## 스타일 규칙
- [문체, 형식 등]

## 금지 사항
- [하면 안 되는 것]

## 참고 자료
- references/ 폴더 참조
```

실전 스킬: 주간 보고서 자동 작성

```
# .claude/skills/weekly-report/SKILL.md

---
name: weekly-report
description: "주간 보고서, 위클리 리포트, 주간 정리를 요청할 때 사용"
---

# 주간 보고서 자동 작성

## 단계
1. 이번 주 git log를 분석 (최근 7일)
2. 커밋을 카테고리별로 분류 (기능추가/버그수정/개선/문서)
3. 각 카테고리별 주요 변경사항 요약
4. 이번 주 핵심 성과 3줄 정리
5. 다음 주 계획 제안 (미완료 이슈 기반)

## 출력 형식
```markdown
# 주간 보고서 (MM/DD ~ MM/DD)

## 이번 주 핵심 성과
1. ...

## 상세 내역
### 기능 추가
- ...
### 버그 수정
- ...

## 다음 주 계획
- ...
```

규칙
- 기술 용어는 팀원도 이해할 수 있게
- 숫자(커밋 수, 파일 수) 포함
- 2분 내로 읽을 수 있는 분량
```

> 💡 **꿀팁**
> 이 스킬을 만들어두면 `/weekly-report` 한 줄로 주간 보고서가 자동 생성됩니다. 매주 금요일 30분 절약.

MCP 서버 자세히 알아보기

MCP(Model Context Protocol)는 Claude를 외부 세계와 연결하는 다리입니다. 실전에서 가장 많이 쓰는 패턴을 정리합니다.

MCP 작동 원리 (간단 버전)

```
# 비유로 이해하기

Claude Code ─── MCP 서버 ─── 외부 서비스
  (직원)        (전화기)       (거래처)

# Claude가 Notion MCP를 통해 데이터를 가져오는 과정:
1. 여러분: "Notion에서 블로그 아이디어를 가져와줘"
2. Claude → Notion MCP 서버에 요청
3. Notion MCP → Notion API 호출
4. Notion → 데이터 반환
5. Claude ← 결과 수신 → 여러분에게 보여줌
```

MCP 연결 시 주의사항

주의사항	설명
토큰 소비	MCP 하나당 2~5K 토큰이 컨텍스트에 추가됨. 동시에 3~4개가 적정.
보안	MCP는 API 키를 가지므로, 신뢰할 수 있는 서버만 연결.
속도	외부 API 호출이므로 응답이 느릴 수 있음. 타임아웃 설정 권장.
호환성	MCP 서버 버전과 Claude Code 버전 호환 확인.

공식 MCP 서버 연결 예시

GitHub MCP

```
{{
  "mcpServers": {{
    "github": {{
      "command": "npx",
      "args": ["-y", "@anthropic/mcp-server-github"],
      "env": {{
        "GITHUB_TOKEN": "ghp_xxxxxxxxxxxx"
      }}
    }}
  }}
}}
```

Slack MCP

```
{{
  "mcpServers": {{
    "slack": {{
      "command": "npx",
      "args": ["-y", "@anthropic/mcp-server-slack"],
      "env": {{
        "SLACK_BOT_TOKEN": "xoxb-xxxxxxxxxxxx"
      }}
    }}
  }}
}}
```

> ⚠ 주의
>
> **API 키 보안:** MCP 설정에 넣는 API 키는 settings.json에 평문으로 저장됩니다. 이 파일을 GitHub에 올리지 마세요! .gitignore에 .claude/ 폴더를 추가하세요.

실전 Git 워크플로우

Claude Code와 함께 Git을 사용하는 실전 패턴입니다. Git 명령어를 외울 필요 없이, Claude에게 맡기면 됩니다.

패턴 1: 매일 커밋 루틴

```
# 하루 끝에
> 오늘 변경사항을 확인하고,
  의미 있는 단위로 나눠서 커밋해줘.
  커밋 메시지는 한국어로, [feat]/[fix]/[docs] 접두사로.
```

패턴 2: 브랜치 전략

```
> 새 기능을 만들 건데, 브랜치를 만들어줘.
  이름은 feature/user-auth로.
  작업 끝나면 main에 머지하는 것도 도와줘.
```

패턴 3: 실수 되돌리기

```
# 방금 커밋을 되돌리기
> 방금 커밋을 취소하고 이전 상태로 돌아가줘.
  파일 변경사항은 유지하고 커밋만 취소해줘.

# 특정 파일만 되돌리기
> index.html만 이전 커밋 상태로 되돌려줘.
  다른 파일은 건드리지 마.
```

패턴 4: PR 생성 + 설명 자동 작성

```
> 지금까지의 변경사항으로 PR을 만들어줘.
  제목: "사용자 인증 기능 추가"
  설명에 변경된 파일 목록과 주요 변경사항을 정리해줘.
  리뷰어는 @팀원이름 으로 지정해줘.
```

패턴 5: 충돌 해결

```
> main 브랜치와 머지 충돌이 났어.
  충돌 내용을 보여주고, 어떻게 해결할지 제안해줘.
  내가 확인하면 해결해줘.
```

ℹ 알아두기

Git을 모르는 분을 위한 핵심 요약:
Git = 저장 버튼의 업그레이드. '되돌리기'가 무한으로 가능한 저장 시스템. Claude에게 맡기면 됩니다. 직접 명령어를 칠 필요 없어요.

PART 5

업종별 실전 프로젝트

마케터, 크리에이터, 사업가, 병원 담당자.
각자의 업무에 맞는 실전 프로젝트를
Claude Code로 처음부터 끝까지 만들어봅니다.

CHAPTER 12

마케터: 랜딩페이지 + 퍼널 자동화

프로젝트 개요

항목	내용
목표	리드 수집 랜딩페이지 + 이메일 수집 자동화
소요 시간	30분~1시간
난이도	☆ (왕초보 가능)
결과물	실제 배포 가능한 반응형 웹페이지

사전 준비

1 프로젝트 폴더 생성

```
mkdir landing-page && cd landing-page
```

2 CLAUDE.md 생성

마케터용 템플릿을 사용합니다.

```
# CLAUDE.md
- 한국어, 해요체
- 마케팅 카피는 고객 pain point부터
- 모바일 퍼스트 반응형
- CTA 버튼은 한 화면에 반드시 1개 이상
- 페이지 로딩 3초 이내
```

Step 1: Plan Mode로 구조 설계

```
claude
> [Shift+Tab × 2 → Plan Mode]

> 리드 수집 랜딩페이지를 만들어줘.

  서비스: AI 마케팅 자동화 툴
  타겟: 1인 사업자, 소규모 마케터

  필요한 섹션:
  1. 히어로 - 핵심 가치 제안 + CTA
  2. 문제 제기 - 타겟의 고통 3가지
  3. 솔루션 - 기능 소개 3개
  4. 소셜 프루프 - 사용자 후기 3개
  5. 가격 - 플랜 2개 비교
  6. FAQ - 자주 묻는 질문 5개
  7. 최종 CTA - 이메일 수집 폼

  먼저 각 섹션별 구조와 카피 방향을
  계획으로 정리해줘.
```

Claude가 계획을 보여주면 검토하고, 수정 사항이 있으면 대화로 조정합니다.

Step 2: 구현

```
> [Shift+Tab → 일반 모드]
> 위 계획대로 구현해줘.
  색상: 블루 계열 (#1a73e8 메인)
  폰트: 깔끔한 고딕체
  frontend-design 스킬 스타일로
  프로페셔널하게 만들어줘.
```

Step 3: 수정 및 개선

```
# 카피 수정
> 히어로 섹션 텍스트를 바꿔줘.
  "매출을 2배로" 대신
  "잠자는 동안 리드가 쌓인다"로.

# 디자인 수정
> CTA 버튼 색상을 오렌지(#ff6b35)로 바꿔서
  더 눈에 띄게 해줘.

# 기능 추가
> 이메일 입력 시 유효성 검사를 추가해줘.
  올바른 이메일 형식이 아니면 빨간 메시지 표시.
```

Step 4: 이메일 수집 연동

```
> 이메일 폼에서 제출하면
  Google Sheets에 자동 저장되게 해줘.
  Google Apps Script를 활용해서.
  시트 이름은 "리드_수집"으로.
```

Step 5: 배포

```
> 이 페이지를 Vercel에 배포할 수 있게
  설정해줘. 배포 방법도 단계별로 알려줘.
```

완성!

30분~1시간 만에 완성된 결과물:

✅ 반응형 랜딩페이지 (7개 섹션)

✅ 이메일 수집 → Google Sheets 자동 저장

✅ Vercel 무료 배포

✅ 커스텀 도메인 연결 가능

💡 꿀팁

이 랜딩페이지를 A/B 테스트하고 싶다면:

`> 이 페이지를 복사해서 variant-b.html을 만들어줘. 히어로 섹션의 카피만 다르게 해줘.`

CHAPTER 13

크리에이터: 블로그 자동화 시스템

프로젝트 개요

항목	내용
목표	/blog 스킬로 SEO 최적화 블로그 자동 생성
소요 시간	스킬 제작 30분 + 이후 글 당 5분
난이도	☆☆ (기본 사용 경험 필요)
결과물	재사용 가능한 블로그 작성 시스템

Step 1: /blog 스킬 만들기

```
# .claude/commands/blog.md 생성

블로그 글 작성 워크플로우

주제: $ARGUMENTS

## 단계
1. 주제에서 SEO 타겟 키워드 3개 추출
2. 제목 후보 5개 제안 (숫자+키워드 조합)
3. 사용자가 제목 선택하면 아웃라인 작성
4. 본문 작성 (1,500~2,500자)
5. 메타 설명 작성 (155자 이내)
6. 내부 링크 추천 3개

## 스타일 규칙
- 해요체, 대화하듯 친근하게
- 소제목마다 핵심 한 줄 요약으로 시작
- 비유 최소 2개 포함
- 도입부 3줄 이내에 hook
- 마무리에 CTA (댓글, 구독, 다음 글 링크)

## SEO 규칙
- 타겟 키워드를 제목, 첫 문단, 소제목에 포함
- H2 소제목 4~6개
- 본문 내 키워드 밀도 1~2%
- 이미지 alt 텍스트에 키워드 포함

## 금지
- 과장 표현, 근거 없는 주장
- "~입니다" 체 (딱딱함)
- 키워드 과다 삽입 (스팸 느낌)
```

Step 2: 사용하기

```
> /blog AI를 활용한 인스타그램 마케팅 자동화

Claude: SEO 키워드를 분석했습니다.
  1. AI 인스타그램 마케팅
  2. 인스타 자동화 도구
  3. AI 마케팅 자동화

제목 후보:
  1. "2026년 AI 인스타그램 마케팅 자동화 7가지 방법"
  2. "인스타 마케팅, AI로 하루 30분이면 충분한 이유"
  ...

> 2번으로 해줘.
```

💡 **꿀팁**

콘텐츠 리퍼포징 활용: 블로그 글이 완성되면 다음과 같이 재활용하세요.

`> 이 블로그 글을 기반으로 인스타 캐러셀(카드뉴스) 10장짜리 텍스트를 만들어줘.`

`> 이 블로그 글을 유튜브 5분 대본으로 변환해줘.`

CHAPTER 14

병원 마케팅: 블로그 SEO 자동화

프로젝트 개요

항목	내용
목표	시술 비교/소개 블로그 콘텐츠 자동 생성
소요 시간	시스템 구축 1시간 + 이후 글 당 10분
난이도	☆☆ (기본 사용 경험 필요)
결과물	의료법 준수 + SEO 최적화 블로그 시스템

CLAUDE.md: 병원 마케팅 특화

```
# CLAUDE.md — 병원 블로그 프로젝트

## 기본
- 한국어, 존댓말
- 의학 용어 사용 시 괄호 안에 쉬운 설명
  예: 콜라겐 바이오스티뮬레이터(피부 속 콜라겐 생성을 촉진하는 물질)

## 의료 콘텐츠 규칙 (필수)
- 과장/허위 표현 절대 금지
- "최고", "유일한", "100% 효과" 등 사용 불가
- 부작용/주의사항 반드시 포함
- 시술 효과는 "개인차가 있을 수 있습니다" 명시
- 의료법 광고 규정 준수
- 전후 사진 사용 불가 (의료법)

## SEO
- 네이버 SEO 최적화 (제목, 소제목, 키워드)
- 글 길이: 2,000~3,000자
- 관련 시술 내부 링크 포함

## 교훈
- 레이저 시술 설명 시 파장(nm) 정보 포함
- 시술 비교 시 반드시 표 형태로 정리
- 가격 정보는 "상담 시 안내" 처리
```

실전: 시술 비교 글 생성

```
> 콜라겐 바이오스티뮬레이터 비교 블로그 글을 작성해줘.
  비교 대상: 래디에스, 스컬트라, 쥬벨룩 볼륨

  포함할 내용:
  1. 각 시술의 원리 (쉽게 설명)
  2. 효과 지속 기간
  3. 시술 부위
  4. 주의사항/부작용
  5. 비교표
  6. 어떤 사람에게 어떤 시술이 적합한지
```

⚠ 주의

의료 콘텐츠 주의사항: Claude가 생성한 의료 콘텐츠는 반드시 담당 의사의 검수를 받으세요. Claude는 최신 의학 정보를 완벽하게 반영하지 못할 수 있으며, 의료법 위반 가능성을 100% 차단하지 못합니다.

CHAPTER 15

사업가: MVP 3일 만에 만들기

프로젝트 개요

항목	내용
목표	아이디어 → 동작하는 MVP → 배포까지 3일
소요 시간	Day 1: 기획 / Day 2: 개발 / Day 3: 배포+피드백
난이도	⭑⭑⭑ (Part 1~4 학습 후 추천)
결과물	실제 사용자가 쓸 수 있는 웹 서비스

Day 1: 기획 (Plan Mode)

```
> [Plan Mode]

  나는 소규모 카페 사장이야.
  매일 인스타그램에 올릴 메뉴 사진과
  캡션을 만드는 게 너무 귀찮아.

  이런 서비스를 만들고 싶어:
  - 메뉴 이름을 입력하면
  - AI가 매력적인 캡션을 자동 생성
  - 해시태그도 자동 추가
  - 일주일치를 한 번에 만들 수 있으면 좋겠어

  이걸 웹 앱으로 만드는 계획을 세워줘.
  최소한의 기능만. 3일 안에 배포 목표.
```

Day 2: 개발 (병렬 세션)

```
# 세션 1: 프론트엔드
> 계획의 프론트엔드 부분을 구현해줘.
  React + Tailwind CSS.
  모바일 퍼스트로.

# 세션 2: 백엔드 (별도 터미널)
> 계획의 백엔드 API를 구현해줘.
  Supabase를 사용해서.
  캡션 생성은 Claude API를 호출.
```

Day 3: 배포 + 피드백

```
> 이 프로젝트를 Vercel에 배포해줘.
  환경변수 설정 방법도 알려줘.

# 배포 후
> Google Analytics를 추가해줘.
  사용자가 어떤 기능을 가장 많이 쓰는지
  추적하고 싶어.
```

3일 MVP의 핵심 원칙

① 기능은 3개 이내로 제한 (더 넣고 싶어도 참기)

② Day 1에 Plan Mode로 충분히 설계

③ Day 2에 병렬 세션으로 속도 ↑

④ Day 3에 배포하고 실제 사용자 피드백 수집

⑤ 피드백 기반으로 2주차에 개선

마케터 프로젝트: A/B 테스트 페이지 만들기

랜딩페이지를 만들었으니, 이제 A/B 테스트로 전환율을 높여봅시다.

```
> 현재 랜딩페이지를 기반으로 A/B 테스트 버전을 만들어줘.

  변경 사항 (Variant B):
  1. 히어로 섹션: 제목을 질문형으로
     "마케팅 자동화, 아직도 수동으로?" →
     "잠자는 동안 리드 100개, 어떻게?"
  2. CTA 버튼: "무료 체험" → "지금 시작하기 (30초)"
  3. 소셜 프루프: 위치를 히어로 바로 아래로

  variant-a.html / variant-b.html 두 파일로 만들어줘.
```

Google Sheets로 A/B 결과 추적

```
> 각 버전에서 CTA 클릭 시
  Google Sheets에 기록되게 해줘.
  기록할 정보: 버전(A/B), 클릭 시간, UTM 파라미터

  나중에 어느 버전이 더 클릭률이 높은지
  비교할 수 있게.
```

Notion 연동 자동화

```
# MCP를 활용한 Notion 연동
> Notion MCP를 연결하고,
  이메일이 수집될 때마다 Notion 데이터베이스에
  자동으로 추가되게 해줘.

  필드: 이름, 이메일, 유입 경로, 수집 일시
```

> 💡 **꿀팁**
>
> **퍼널 자동화 전체 흐름:**
> ① 랜딩페이지(리드 수집) → ② Google Sheets(데이터 저장) → ③ Notion(CRM 관리) → ④ 이메일 자동 발송(후속 조치)
> 이 전체 흐름을 Claude Code + MCP로 구현할 수 있습니다.

크리에이터 프로젝트: 콘텐츠 리퍼포징 시스템

하나의 콘텐츠를 여러 플랫폼에 맞게 변환하는 시스템입니다.

```
# 유튜브 대본 → 블로그 글
> 이 유튜브 대본을 블로그 글로 변환해줘.
  [대본 붙여넣기]

  규칙:
  - 구어체 → 해요체 문어체
  - "여러분" 호칭 유지
  - 타임스탬프는 소제목으로 변환
  - SEO 키워드 자연스럽게 포함
```

```
# 블로그 글 → 인스타 캐러셀
> 이 블로그 글을 인스타 캐러셀 10장으로 만들어줘.

  규칙:
  - 첫 장: 후킹 제목 (질문형)
  - 2~8장: 핵심 포인트 (장당 3줄 이내)
  - 9장: 요약
  - 10장: CTA ("저장하고 나중에 다시 보세요")
  - 각 장의 텍스트만 출력 (디자인은 별도)
```

```
# 블로그 글 → 뉴스레터
> 이 블로그 글을 이메일 뉴스레터 형태로 변환해줘.

  규칙:
  - 제목: 30자 이내 (오픈율 최적화)
  - 미리보기 텍스트: 50자
  - 본문: 원문의 40% 분량으로 압축
  - 링크: "전체 읽기" → 블로그로 연결
  - HTML 이메일 형태로 출력
```

자동화 파이프라인

```
# 전체 리퍼포징을 하나의 커맨드로
# .claude/commands/repurpose.md

원본 콘텐츠: $ARGUMENTS

이 콘텐츠를 다음 4가지 형태로 변환:
1. 블로그 글 (2,000자, SEO 최적화)
2. 인스타 캐러셀 텍스트 (10장)
3. 트위터/X 스레드 (10개 트윗)
4. 이메일 뉴스레터

각각 별도 파일로 저장:
- blog.md
- instagram.md
- twitter.md
- newsletter.html
```

1 → 4 변환 효과

한 번의 콘텐츠 제작으로 4개 플랫폼에 발행. 기존 대비 시간 75% 절감. /repurpose 커맨드 하나로 5분이면 완료.

사업가 프로젝트: 데이터 대시보드

여러 채널의 데이터를 한눈에 보는 대시보드를 Claude Code로 만들어봅니다.

```
> [Plan Mode]
> 우리 비즈니스의 핵심 지표를 한 화면에서 보는
  대시보드를 만들고 싶어.

  표시할 지표:
  - 일일 매출 (그래프)
  - 신규 가입자 수 (그래프)
  - 이번 달 누적 매출 (숫자)
  - 전환율 (어제 vs 오늘)
  - 인기 상품 Top 5 (리스트)
  - 채널별 유입 비율 (파이 차트)

  데이터는 일단 더미 데이터로 시작.
  나중에 실제 API를 연결할 수 있는 구조로.
  React + Chart.js 또는 Recharts 사용.
```

실제 데이터 연결

```
# 더미 데이터 → 실제 API 연결
> 더미 데이터 대신 Supabase에서 실제 데이터를
  불러오도록 연결해줘.

  테이블 구조:
  - orders (id, amount, created_at)
  - users (id, email, created_at)
  - products (id, name, category, price)

  Supabase URL과 API Key는 .env에서 읽어오게.
```

자동 새로고침 + 알림

```
> 대시보드가 5분마다 자동으로 데이터를 새로고침하게 해줘.
  그리고 일일 매출이 목표(100만원)를 달성하면
  화면에 축하 배너가 뜨게 해줘.
```

> 💡 **꿀팁**
>
> 이 대시보드를 팀원들과 공유하려면 Vercel에 배포하세요. 비밀번호 보호를 추가하면 내부용으로 안전하게 사용할 수 있습니다.
>
> `> 이 대시보드에 간단한 비밀번호 보호를 추가해줘.`

마케터 프로젝트: 이메일 마케팅 자동화

리드를 수집한 후, 자동으로 환영 이메일을 보내는 시스템을 구축합니다.

```
> 이메일 마케팅 자동화 시스템을 만들어줘.

  흐름:
  1. 랜딩페이지에서 이메일 수집
  2. Google Sheets에 저장
  3. 환영 이메일 자동 발송
  4. 3일 후 후속 이메일 발송

  이메일 내용:
  - 환영 메일: 서비스 소개 + 무료 가이드 링크
  - 후속 메일: 성공 사례 + CTA(무료 상담 예약)

  Google Apps Script를 활용해서 구현해줘.
```

크리에이터 프로젝트: 유튜브 자동화 시스템

```
# .claude/commands/youtube.md

유튜브 콘텐츠 제작 워크플로우

주제: $ARGUMENTS

## 단계
1. 주제에서 검색 트렌드 키워드 분석
2. 영상 제목 5개 제안 (CTR 최적화)
3. 선택한 제목으로 영상 구성 작성
   - 후킹 도입부 (15초)
   - 본문 (문제-해결 구조)
   - CTA (구독/좋아요/다음 영상)
4. 상세 대본 작성 (5~10분 분량)
5. 영상 설명 + 태그 + 타임스탬프
6. 썸네일 텍스트 3개 제안
```

```
# 사용법
> /youtube 클로드 코드로 30분 만에 웹사이트 만들기
```

병원 프로젝트: 시술 비교 콘텐츠 시리즈

```
# .claude/commands/treatment-compare.md

시술 비교 블로그 글 작성

비교 대상: $ARGUMENTS

## 포함 항목
1. 각 시술의 원리 (비전문가도 이해할 수 있게)
2. 시술 부위 및 효과
3. 효과 지속 기간
4. 시술 시간 및 회복 기간
5. 주의사항 및 부작용 (필수!)
6. 비교 표 (한눈에 보기)
7. 어떤 분에게 적합한지

## 규칙
- 의료법 광고 규정 준수
- 과장 금지, 객관적 서술
- "개인차가 있을 수 있습니다" 명시
- 부작용/주의사항 반드시 포함
- 가격은 "상담 시 안내"로 처리
```

```
> /treatment-compare 래디에스 vs 스컬트라 vs 쥬벨룩 볼륨
```

꿀팁

이 커맨드를 한 번 만들어두면, 매주 새로운 시술 비교 글을 10분 만에 생성할 수 있습니다. 의사의 검수만 거치면 바로 블로그에 올릴 수 있는 품질입니다.

사업가 프로젝트: SaaS 랜딩페이지 + 대기자 목록

아직 제품을 만들기 전에, 랜딩페이지와 대기자 목록으로 수요를 검증하는 프로젝트입니다.

```
> SaaS 제품의 사전 출시 랜딩페이지를 만들어줘.

  제품: AI 기반 인스타그램 캡션 자동 생성기
  타겟: 소규모 카페, 레스토랑, 쇼핑몰 사장님

  섹션:
  1. 히어로: 핵심 가치 제안
     "매일 인스타 고민, AI가 5초 만에 해결"
  2. 문제: 사장님들의 3가지 고통
  3. 솔루션: 기능 3개 소개 + 데모 GIF 자리
  4. 대기자 등록: 이메일 + "사전 등록하면 50% 할인"
  5. FAQ

  기술:
  - 순수 HTML/CSS/JS (프레임워크 없이)
  - 이메일 수집 → Google Sheets
  - Vercel 배포
  - 카운터: "현재 000명 대기 중" (Google Sheets에서 실시간)
```

대기자 수 실시간 표시

```
> Google Sheets의 대기자 수를 실시간으로
  랜딩페이지에 표시하는 기능을 추가해줘.
  "현재 127명이 기다리고 있습니다" 형태로.
  숫자가 올라가는 카운트업 애니메이션도 넣어줘.
```

MVT (Minimum Viable Test) 체크리스트

MVP(제품)를 만들기 전에 MVT(테스트)로 수요를 먼저 검증하세요.

단계	행동	기준
1	랜딩페이지 제작	30분 이내 완성
2	SNS에 공유	인스타, 페이스북, 블로그
3	1주일 대기	대기자 모으기
4	수요 판단	50명 이상 → 제작 시작
5	MVP 제작	Part 5 Ch.15 따라하기

사업 아이디어 검증 공식

랜딩페이지(30분) → SNS 공유 → 대기자 수 확인

50명 미만 → 피봇(다른 아이디어)
50~200명 → MVP 제작 시작
200명 이상 → 빠르게 출시 + 초기 사용자 확보

모든 프로젝트의 공통 배포 가이드

어떤 프로젝트든 만들었으면 배포해야 합니다. 가장 쉬운 Vercel 배포를 단계별로 안내합니다.

1. **Vercel 가입**
   vercel.com 접속 → 'Start Deploying' → GitHub 계정으로 가입
2. **GitHub에 코드 올리기**

```
> 이 프로젝트를 GitHub에 올려줘.
  리포 이름은 my-landing-page로.
  public 리포로 만들어줘.
```

3. **Vercel에서 Import**
   Vercel 대시보드 → 'Add New Project' → GitHub 리포 선택 → 'Deploy' 클릭
4. **환경변수 설정 (필요한 경우)**

```
# API 키 등이 필요한 프로젝트라면
# Vercel Settings → Environment Variables에 추가
# 예: SUPABASE_URL, SUPABASE_ANON_KEY
```

5. **커스텀 도메인 연결 (선택)**

```
> Vercel에 배포된 사이트에
  내 도메인 example.com을 연결하는 방법을 알려줘.
  도메인은 가비아에서 구매했어.
```

배포 후 체크리스트

항목	확인
PC에서 정상 표시	☐
모바일에서 정상 표시	☐
이메일 폼 제출 → 데이터 저장 확인	☐
페이지 로딩 3초 이내	☐
모든 링크 정상 작동	☐
OG 이미지 (SNS 공유 시 미리보기)	☐
Google Analytics 연동	☐
SSL 인증서 (https)	☐ Vercel 자동

```
# Claude에게 한 번에 확인 요청
> 이 프로젝트를 배포하기 전에 최종 점검해줘.
  모바일 반응형, 로딩 속도, 링크 동작,
  OG 태그, 접근성을 모두 확인하고
  문제가 있으면 고쳐줘.
```

프로젝트 유지보수 가이드

프로젝트를 만들고 끝이 아닙니다. 유지보수 습관을 들이세요.

주간 유지보수 루틴

요일	작업	Claude 프롬프트
월	에러/버그 확인	"이번 주에 발생한 에러 로그를 확인해줘"
수	성능 점검	"Lighthouse 점수를 확인하고 개선점 알려줘"
금	보안 업데이트	"npm audit 실행하고 취약점 있으면 고쳐줘"

월간 유지보수 루틴

- **의존성 업데이트**: `> 오래된 패키지를 안전하게 업데이트해줘`
- **CLAUDE.md 정리**: 쌓인 교훈을 카테고리별로 정리
- **백업 확인**: GitHub에 모든 변경이 Push되었는지 확인
- **보안 스캔**: `npx ecc-agentshield scan`

프로젝트 확장 전략

MVP가 성공하면, 다음 단계로 확장하는 전략입니다.

1. **사용자 피드백 수집**
   Google Forms 또는 Tally.so로 간단한 설문 → 가장 원하는 기능 파악

2 **핵심 기능 1개 추가**
가장 많이 요청된 기능 하나만 추가. 욕심 부리지 말 것.

3 **사용자 추적 강화**
Google Analytics 이벤트 추적으로 어떤 기능이 가장 많이 쓰이는지 파악

4 **수익화 검토**
유료 플랜 도입 or 광고 or 어필리에이트

에이나우의 성장 공식

아이디어 검증(1주) → MVP(3일) → 첫 사용자(1주) → 피드백(2주) → 개선(반복)

이 사이클을 1개월 단위로 돌리면, 6개월 후에는 수익이 나는 서비스가 됩니다. Claude Code가 이 속도를 가능하게 합니다.

크리에이터 프로젝트: 카드뉴스 자동 생성기

인스타그램, 블로그에 올릴 카드뉴스를 Claude Code로 자동 생성합니다.

```
> 인스타그램 캐러셀(카드뉴스) HTML을 만들어줘.

  주제: "Claude Code 시작하는 5가지 이유"

  구성:
  - 총 10장
  - 1장: 후킹 제목 (질문형)
  - 2~8장: 포인트당 1장 (아이콘 + 짧은 텍스트)
  - 9장: 요약
  - 10장: CTA + 프로필 태그

  디자인:
  - 1080x1080 정사각형
  - 다크 블루 배경, 흰색 텍스트
  - 깔끔한 모던 스타일
  - 각 장을 별도 div로 (스크린캡처용)

  브라우저에서 열면 10장이 세로로 나열되게 해줘.
  각 장을 스크린캡처해서 인스타에 올릴 수 있게.
```

카드뉴스 Slash Command

```
# .claude/commands/card-news.md

카드뉴스 생성 워크플로우

주제: $ARGUMENTS

## 단계
1. 주제에서 핵심 포인트 5~7개 추출
2. 후킹 제목 3개 제안 (질문형)
3. 사용자 선택 후 10장 구성
4. HTML 파일 생성 (1080x1080)
5. 각 장의 텍스트도 별도 파일로 저장

## 디자인 규칙
- 정사각형 (1080x1080)
- 장당 텍스트 3줄 이내
- 폰트 크기: 제목 48px, 본문 32px
- 배경색: 브랜드 컬러 또는 다크

## 출력
- card-news.html (브라우저에서 확인용)
- card-news-text.md (텍스트만)
```

```
# 사용 예시
> /card-news AI 시대에 살아남는 마케터의 7가지 습관
```

꿀팁

카드뉴스 HTML을 만든 후, 브라우저에서 각 장을 스크린캡처하세요. Mac은 Cmd+Shift+4, Windows는 Win+Shift+S를 사용합니다. 이 과정도 자동화할 수 있습니다.

사업가 프로젝트: 고객 CRM 간이 버전

엑셀로 고객을 관리하던 분들을 위한 간단한 CRM 웹앱입니다.

```
> 간단한 고객 관리(CRM) 웹앱을 만들어줘.

  기능:
  1. 고객 목록 (이름, 연락처, 이메일, 메모)
  2. 고객 추가/수정/삭제
  3. 검색 + 필터링
  4. 태그 기능 (VIP, 신규, 휴면 등)
  5. 최근 연락 날짜 기록
  6. 간단한 통계 (총 고객수, 이번 달 신규, VIP 비율)

  기술:
  - React + Tailwind CSS
  - Supabase (데이터베이스 + 인증)
  - 모바일 반응형 (폰에서도 사용 가능)
```

자동 리마인더 기능 추가

```
> CRM에 자동 리마인더 기능을 추가해줘.
  - 30일 이상 연락 안 한 고객 → "팔로업 필요" 알림
  - VIP 고객의 생일 1주일 전 → 알림
  - 신규 고객 등록 후 3일 → "환영 연락" 알림
```

교육자 프로젝트: 퀴즈 자동 생성기

```
> 학습 자료를 기반으로 퀴즈를 자동 생성하는 도구를 만들어줘.

  입력: 텍스트 (학습 내용 붙여넣기)
  출력: 4지선다 퀴즈 10문제

  기능:
  - 난이도 선택 (쉬움/보통/어려움)
  - 정답 + 해설 포함
  - 학생에게 보낼 수 있는 깔끔한 HTML 형태
  - 정답 확인 버튼 (클릭하면 정답 표시)
```

```
# 사용 예시
> 다음 학습 자료를 기반으로 중간고사 퀴즈를 만들어줘:
  [학습 자료 붙여넣기]
  난이도: 보통
  문제 수: 15개
```

모든 프로젝트의 공통 패턴

① 문제 정의 (내가 아는 것)

② Plan Mode로 설계 (Claude와 함께)

③ 구현 (Claude가 코드 작성)

④ 테스트 (내가 확인, Claude가 수정)

⑤ 배포 (Vercel에 올리기)

⑥ 개선 (피드백 → 반복)

PART 6

프로 레벨 세팅

토큰 비용 절감, Agent Teams,
자동화 끝판왕 조합까지.
이것까지 알면 상위 1%입니다.

CHAPTER 16

토큰 비용 50% 절감 고급 세팅

비용이 어디서 새는지 파악하기

Claude Code 비용의 80%는 세 곳에서 발생합니다:

- **불필요하게 긴 CLAUDE.md** — 매 세션마다 로드됨
- **안 쓰는 MCP 서버** — 도구 정의만으로 토큰 소모
- **비효율적인 대화 패턴** — 긴 대화, 넓은 탐색

세팅 #1: Thinking Token 제한

```
{{
  "env": {{
    "MAX_THINKING_TOKENS": "10000"
  }}
}}
```

Claude는 응답 전에 "생각"합니다. 이 토큰이 무제한이면 복잡한 작업에서 수만 토큰을 소비합니다. 10,000으로 제한하면 대부분의 작업에 충분하면서 비용이 크게 줄어듭니다.

세팅 #2: 자동 압축 타이밍

```
{{
  "env": {{
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }}
}}
```

기본값(80~95%)을 50%로 낮추면 일찍 압축이 시작되어 컨텍스트가 더 효율적입니다.

세팅 #3: Opusplan 모드

계획은 Opus(강력한 모델)로, 실행은 Sonnet(가성비 모델)으로.

```
> /model opusplan
```

비용 절감 체크리스트

항목	절감 효과	난이도
CLAUDE.md 500줄→100줄	매 세션 10~15%	☆
안 쓰는 MCP 끄기	컨텍스트 20~30% 확보	☆
MAX_THINKING_TOKENS	thinking 30~50%↓	☆☆
AUTOCOMPACT 50%	효율 20%↑	☆☆
주제 변경 시 /clear	불필요 토큰 제거	☆
Opusplan 모드	Opus 대비 60~70%↓	☆☆
서브에이전트 탐색 위임	메인 컨텍스트 보호	☆☆
구체적 지시 (넓은 탐색↓)	불필요 파일 읽기↓	☆

종합 효과

위 세팅을 모두 적용하면 **월 비용 40~60% 절감**이 가능합니다. Pro 구독자도 Rate Limit에 덜 걸리게 됩니다.

CHAPTER 17

Agent Teams — AI 팀 운영법

서브에이전트 vs Agent Teams

	서브에이전트	Agent Teams
비유	심부름꾼	프로젝트 팀
역할	시킨 일 하고 보고	역할 분담 + 소통 + 협업
컨텍스트	독립 (결과만 전달)	독립 + 팀원 간 공유
토큰 비용	보통	서브에이전트의 ~7배
추천 사용	탐색, 분석, 단일 작업	복잡한 다중 파일 작업

Agent Teams 실전 예시

```
> 회원가입 기능을 만들어줘.
  프론트엔드는 React 폼,
  백엔드는 Express API가 필요해.
  에이전트 팀으로 동시에 작업해줘.

Claude: 에이전트 팀을 구성합니다.
  🎨 Frontend Agent → React 회원가입 폼
  ⚙️ Backend Agent → Express API + DB 스키마
  🔗 Integration Agent → 연동 + 테스트
```

⚠ 주의

Agent Teams는 비용이 많이 듭니다. 서브에이전트의 약 7배 토큰을 사용합니다. 단순한 작업에는 사용하지 마세요. 프론트+백엔드 동시 구현, 대규모 리팩토링 같은 복잡한 작업에만 쓰세요.

언제 Agent Teams를 쓰고, 언제 안 쓸지

상황	추천
파일 1~3개 수정	일반 Claude
코드 탐색/분석	서브에이전트
프론트+백 동시 개발	Agent Teams ✅
대규모 리팩토링	Agent Teams ✅
간단한 버그 수정	일반 Claude

CHAPTER 18

자동화 끝판왕 — Hooks + MCP + 스킬 조합

세 가지 자동화 도구 비교

도구	역할	비유	실행 방식
스킬	작업 레시피	자판기 버튼	호출하면 실행
MCP	외부 서비스 연결	전화기	연결 후 사용
Hooks	자동 반응 규칙	센서등	조건 충족 시 자동

워크플로우 조합 예시: 블로그 자동 발행

```
# 전체 흐름
나: /blog AI 마케팅 자동화

1. CLAUDE.md 규칙에 따라 해요체로 작성
2. /blog 스킬이 SEO 최적화 글 구조 잡기
3. 글 완성 → Notion MCP로 노션에 자동 업로드
4. PostToolUse Hook → 맞춤법 검사 자동 실행
5. Notification Hook → "블로그 발행 완료" 알림
```

여러분이 한 일: `/blog AI 마케팅 자동화` 한 줄 입력. 나머지는 전부 자동입니다.

/loop와 /schedule: 반복 자동화

```
# /loop: 로컬에서 반복 (최대 3일)
> /loop "매 6시간마다 경쟁사 블로그를 체크하고
   새 글이 있으면 요약해서 Slack에 보내줘"

# /schedule: 클라우드에서 반복 (컴퓨터 꺼도 실행)
> /schedule "매일 아침 9시에 어제 커밋을
   요약해서 팀 Slack에 보내줘"
```

자동화 설계 원칙

① 수동으로 3번 이상 한 작업 → 스킬로 만들기

② 외부 서비스가 필요한 작업 → MCP 연결

③ 매번 똑같이 실행되어야 하는 것 → Hook으로 자동화

④ 정기적으로 반복하는 것 → /loop 또는 /schedule

Hook 실전 레시피 모음

바로 복사해서 settings.json에 넣을 수 있는 Hook 레시피 5가지입니다.

레시피 1: macOS/Windows 알림

```
// Notification Hook - macOS
"Notification": [{{
  "hooks": [{{
    "type": "command",
    "command": "osascript -e 'display notification \\"$NOTIFICATION_MESSAGE\\" with title \\"Claude Code\\" sound name \\"Glass\\"'"
  }}]
}}]
```

레시피 2: 자동 포맷팅 (Prettier)

```
// PostToolUse Hook
"PostToolUse": [{{
  "matcher": "Edit|Write",
  "hooks": [{{
    "type": "command",
    "command": "npx prettier --write $FILE_PATH 2>/dev/null || true"
  }}]
}}]
```

레시피 3: 위험 파일 보호

```
// PreToolUse Hook - .env 파일 접근 차단
"PreToolUse": [{{
  "matcher": "Edit|Write",
  "hooks": [{{
    "type": "command",
    "command": "if echo $FILE_PATH | grep -q '.env'; then echo 'BLOCKED: .env 파일 수정 금지'; exit 1; fi"
  }}]
}}]
```

레시피 4: 커밋 전 테스트 자동 실행

```
// PreToolUse Hook - git commit 전 테스트
"PreToolUse": [{{
  "matcher": "Bash(git commit*)",
  "hooks": [{{
    "type": "command",
    "command": "npm test 2>&1 || (echo '테스트 실패! 커밋 차단.'; exit 1)"
  }}]
}}]
```

레시피 5: Slack 알림

```
// Stop Hook - 작업 완료 시 Slack 알림
"Stop": [{{
  "hooks": [{{
    "type": "http",
    "url": "https://hooks.slack.com/services/YOUR/WEBHOOK/URL",
    "method": "POST",
    "body": "{{\n\"text\": \"Claude Code 작업이 완료되었습니다.\n$HOOK_MESSAGE\"\
}}"
  }}]
}}]
```

⚠ 주의

Hook은 100% 확실하게 실행됩니다. CLAUDE.md는 80% 정도 따르지만, Hook은 예외 없이 매번 실행됩니다. 따라서 '반드시 지켜야 할 것'은 Hook으로, '가이드라인'은 CLAUDE.md로 구분하세요.

MCP 서버 실전 활용 가이드

MCP(Model Context Protocol)로 Claude를 외부 서비스와 연결하는 실전 패턴입니다.

가장 많이 쓰는 MCP 서버 Top 5

MCP 서버	연결 대상	활용 예시
Notion MCP	Notion 페이지/DB	블로그 자동 업로드, 태스크 관리
GitHub MCP	GitHub 리포	이슈 관리, PR 생성, 코드 검색
Supabase MCP	Supabase DB	데이터 조회/수정, 사용자 관리
Slack MCP	Slack 워크스페이스	메시지 전송, 채널 검색
Google Sheets MCP	스프레드시트	데이터 읽기/쓰기, 차트 생성

MCP 설정 예시: Notion

```
// .claude/settings.json에 추가
{{
  "mcpServers": {{
    "notion": {{
      "command": "npx",
      "args": ["-y", "@notionhq/notion-mcp-server"],
      "env": {{
        "NOTION_API_KEY": "secret_xxx..."
      }}
    }}
  }}
}}
```

```
# 연결 후 사용 예시
> Notion의 "블로그 아이디어" 데이터베이스에서
   상태가 "초안"인 항목을 모두 가져와줘.

> 방금 작성한 블로그 글을 Notion의
   "발행 완료" 페이지에 업로드해줘.
```

MCP vs 스킬: 언제 뭘 쓸까

상황	추천	이유
외부 서비스 데이터 필요	MCP	실시간 데이터 접근
작업 방법론/레시피 필요	스킬	프롬프트 템플릿
Notion에 글 올리기	MCP	API 호출 필요
SEO 최적화 블로그 쓰기	스킬	작업 단계/규칙 정의
두 가지 다 필요	스킬 + MCP	스킬이 MCP를 활용

ℹ 알아두기

최근 트렌드: 많은 MCP 서버가 잘 만든 스킬로 대체 가능합니다. 스킬은 내부를 직접 볼 수 있어 보안이 투명하지만, MCP는 블랙박스입니다. 가능하면 스킬을 먼저 시도하고, 꼭 필요할 때만 MCP를 쓰세요.

워크플로우 설계 프레임워크

자신만의 자동화 워크플로우를 설계하는 체계적인 방법입니다.

Step 1: 반복 작업 발굴

```
# 지난 1주일간의 작업을 돌아보세요

1. 매번 하는 작업은? → 스킬/커맨드 후보
2. 외부 서비스와 주고받는 작업은? → MCP 후보
3. 실수하면 안 되는 작업은? → Hook 후보
4. 정기적으로 반복하는 작업은? → /loop, /schedule 후보
```

Step 2: 도구 매칭 표 만들기

내 반복 작업	스킬로	MCP로	Hook으로
블로그 글 작성	/blog ✓	Notion 업로드	맞춤법 검사
코드 커밋	/commit ✓	GitHub PR	테스트 자동 실행
주간 보고서	/report ✓	Slack 전송	-
(내 작업 1)			
(내 작업 2)			

Step 3: 가장 높은 ROI부터 구현

자동화 ROI 공식

ROI = (매번 아끼는 시간 × 주당 반복 횟수) ÷ 설정 소요 시간

예: 블로그 글 작성이 매번 2시간, 주 2회 → 주 4시간 절약
/blog 스킬 설정에 30분 소요
ROI = (4시간 × 4주) ÷ 0.5시간 = 32배

Step 4: 테스트 + 개선

자동화를 설정한 후 1주일간 사용하면서:

- 안 쓰는 기능 → 제거
- 빠진 기능 → 추가
- 스킬 프롬프트 → 더 정확하게 다듬기
- 교훈 → CLAUDE.md에 추가

> 💡 **꿀팁**
> 완벽한 워크플로우는 한 번에 만들어지지 않습니다. Boris 팀도 CLAUDE.md를 일주일에 여러 번 업데이트합니다. 1주일 단위로 개선하는 습관이 중요합니다.

Claude Code의 미래: 2026년 하반기 전망

현재 빠르게 발전하고 있는 Claude Code의 방향성을 정리합니다.

확정된 방향

기능	현재	전망
Agent Teams	초기 단계	더 정교한 역할 분담, 비용 최적화
플러그인	베타	마켓플레이스 확대, 원클릭 설치
클라우드 실행	/schedule 초기	24시간 무중단 에이전트
멀티모달	이미지 입력	화면 공유, 실시간 디버깅
팀 협업	기본	실시간 공동 편집, 리뷰 자동화

비개발자를 위한 의미

- **더 쉬운 시작**: 설치 과정이 더 간소화되고, GUI(그래픽) 인터페이스가 강화될 것
- **더 강력한 자동화**: /schedule이 본격화되면 "매일 아침 블로그 초안 작성" 같은 워크플로우가 컴퓨터 없이 돌아감
- **더 많은 템플릿**: 업종별, 직무별 스타터 팩이 늘어날 것

- **비용 하락**: 모델 효율화로 같은 비용에 더 많은 작업 가능

Claude Code vs Cursor vs GitHub Copilot

	Claude Code	Cursor	GitHub Copilot
접근 방식	에이전트 (직접 실행)	IDE 내 어시스턴트	코드 자동완성
비개발자 적합	★★★	★★	★
코드 품질	★★★	★★★	★★
자동화	★★★	★★	★
확장성	스킬+MCP+Hook	확장 프로그램	제한적
추천 사용	프로젝트 생성, 자동화	코드 편집, 리팩토링	코드 보조

> *Claude Code는 코드를 '보조'하는 도구가 아니라 '실행'하는 도구입니다. 이 차이가 비개발자에게는 특히 중요합니다. 코드를 읽을 필요 없이 결과를 얻을 수 있으니까요.*
>
> — 에이나우 커뮤니티 분석

도구를 선택하는 기준

① 코딩을 전혀 모른다 → Claude Code (유일한 선택)
② 개발자인데 프로젝트 생성이 잦다 → Claude Code + Cursor 병행
③ 개발자인데 기존 코드 편집이 많다 → Cursor + Copilot
④ 자동화/에이전트가 핵심 → Claude Code

보안 완전 가이드

Claude Code는 컴퓨터에서 파일을 읽고, 쓰고, 명령어를 실행합니다. 편리한 만큼 보안에 신경 써야 합니다.

보안 체크리스트 (반드시 확인)

#	항목	확인
1	CLAUDE.md에 API 키가 없는지	☐
2	settings.json이 Git에 올라가지 않는지 (.gitignore)	☐
3	.env 파일이 Claude 수정 불가인지 (PreToolUse Hook)	☐
4	rm -rf 같은 위험 명령이 차단되는지	☐
5	MCP 서버가 신뢰할 수 있는 것인지	☐
6	월 1회 보안 스캔 실행하는지	☐
7	dangerously-skip-permissions를 무분별하게 사용하지 않는지	☐

.gitignore 필수 항목

```
# .gitignore에 반드시 추가
.env
.env.local
.env.production
.claude/settings.json  # API 키가 포함될 수 있음
node_modules/
```

permissions 설정으로 Claude 권한 제한

```
# settings.json의 permissions 섹션
{{
  "permissions": {{
    "allow": [
      "Read(**)",          // 모든 파일 읽기 허용
      "Edit(src/**)",      // src 폴더만 수정 허용
      "Bash(npm test*)",   // 테스트만 실행 허용
      "Bash(git *)"        // Git 명령 허용
    ],
    "deny": [
      "Edit(.env*)",       // .env 수정 차단
      "Bash(rm -rf*)",     // 위험 삭제 차단
      "Bash(curl*)",       // 외부 다운로드 차단
      "Bash(sudo*)"        // 관리자 권한 차단
    ]
  }}
}}
```

⚠ 주의

절대 하지 말아야 할 것:

- CLAUDE.md에 API 키, 비밀번호 적기
- settings.json을 공개 GitHub에 올리기
- dangerously-skip-permissions로 항상 실행
- 검증되지 않은 MCP 서버 연결
- Claude에게 sudo 명령 허용

비용 관리 대시보드 만들기

Claude Code 사용 비용을 추적하고 시각화하는 간단한 대시보드를 만들어봅니다.

```
> Claude Code 사용 비용을 추적하는 대시보드를 만들어줘.

  기능:
  1. 일별 토큰 사용량 그래프
  2. 모델별 사용 비율 (파이 차트)
  3. 이번 달 누적 비용 추정
  4. 비용 알림 (일 $10 초과 시 경고)

  데이터: Claude Code의 사용 로그 파일 파싱
  기술: HTML + Chart.js (단일 파일)
```

장기적 비용 최적화 전략

단계	전략	예상 절감
1단계 (즉시)	AUTOCOMPACT 50%, THINKING 10K	20~30%
2단계 (1주)	안 쓰는 MCP 제거, /clear 습관	+10~20%
3단계 (1개월)	Opusplan 모드, 서브에이전트 활용	+15~25%
4단계 (3개월)	커스텀 스킬로 반복 작업 최적화	+10~15%

총 절감 효과: 55~90%. 같은 작업량을 절반 이하의 비용으로 처리할 수 있습니다.

Claude Code 활용 성숙도 모델

자신의 현재 레벨을 확인하고 다음 단계를 계획하세요.

레벨	특징	기간
Lv.1 초보	설치 완료, 간단한 대화 가능	1주
Lv.2 활용	CLAUDE.md 사용, 프로젝트 1개 완성	2~3주
Lv.3 실전	슬래시 커맨드, 병렬 세션, Plan Mode	1~2개월
Lv.4 프로	Hook, MCP, 스킬 자체 제작, 비용 최적화	2~3개월
Lv.5 마스터	Agent Teams, 자동화 파이프라인, 팀 도입	3~6개월

이 책의 로드맵

Part 1~2 → Lv.1~2 달성 (1~2주)
Part 3~4 → Lv.3 달성 (1개월)
Part 5~6 → Lv.4 달성 (2~3개월)
부록 활용 → Lv.5 달성 (3~6개월)

매일 30분씩 꾸준히 하면, 3개월 후에는 Lv.4 '프로' 레벨에 도달합니다.

부록

부록 A: Boris 42가지 팁 요약

Boris Cherny가 2026년 1~2월에 5개 시리즈로 공개한 42가지 팁의 핵심 요약입니다. 전문은 howborisusesclaudecode.com에서 확인할 수 있습니다.

Part 1: 기본 워크플로우 (1월 2일, 13개 팁)

#	팁	핵심
1	5개 터미널 탭에서 병렬 실행	탭 번호로 관리, 알림 설정
2	웹에서 5~10개 추가 실행	텔레포트로 터미널↔웹 이동
3	아침에 폰으로 세션 시작	Claude iOS 앱 활용
4	Opus + thinking 기본 사용	조종 덜 필요 → 오히려 빠름
5	CLAUDE.md는 짧게 (100줄)	모든 줄이 실제 문제 해결
6	Plan Mode로 세션 시작	Shift+Tab 2번, 계획 확정 후 실행
7	Slash Command 적극 활용	매일 반복 작업을 명령으로
8	서브에이전트 활용	Code Simplifier, Verify App 등
9	Post-tool Hook으로 포맷팅	Claude가 수정 후 자동 정리
10	GitHub App으로 PR 리뷰	/install-github-app
11	PR 리뷰 프롬프트 커스텀	버그/보안만 집중, 간결하게
12	여러 체크아웃 동시 작업	각 세션이 독립 git 상태
13	settings.json 최소화	바닐라 설정으로도 충분

Part 2: 팀 팁 (1월 31일, 10개 팁)

#	팁	핵심
14	Git Worktrees 3~5개	팀 전체의 #1 생산성 팁
15	복잡한 작업은 Plan Mode	두 번째 Claude로 계획 리뷰
16	CLAUDE.md에 교훈 추가	"업데이트해" 한 마디로 자기학습
17	피드백 루프 포함	테스트 명령 → 품질 2~3배↑
18	구체적으로 지시	파일명, 함수명 명시
19	한 세션 한 주제	/clear로 깔끔하게
20	이미지로 컨텍스트 제공	UI 스크린샷, 다이어그램
21	에러 통째로 붙여넣기	Claude가 알아서 진단
22	Plan Mode에서 검증도	구현 후 검증 단계도 계획
23	notes 디렉토리 유지	작업별 학습 내용 축적

Part 3: 커스터마이제이션 (2월 11일, 12개 팁)

#	팁	핵심
24	/terminal-setup 실행	Shift+Enter 줄바꿈 활성화
25	High effort 모드 사용	Boris는 항상 High
26	/plugin으로 설치	LSP, MCP, 스킬, 에이전트 통합 관리
27	LSP 플러그인 필수	모든 주요 언어 지원
28	커스텀 Output Style	~/.claude/output-styles/ 활용
29	PreToolUse Hook으로 보안	위험 파일 접근 차단
30	Prompt Hook으로 AI 판단	상황에 따라 동적 승인/거부
31	Agent Hook 활용	서브에이전트 자동 실행
32	팀 마켓플레이스 설정	settings.json에 등록
33	Session Start Hook	세션 시작 시 환경 자동 설정
34	Stop Hook	작업 완료 시 자동 후처리
35	HTTP Hook으로 웹훅	Slack/Discord 알림 연동

Parts 4-5 (팁 36~42)는 병렬 작업 고급 패턴, 디버깅 전략, 성능 최적화를 다룹니다. 전문은 howborisusesclaudecode.com에서 확인하세요.

부록

부록 B: 업종별 CLAUDE.md 템플릿 5종

📣 템플릿 1: 마케터용

```
# CLAUDE.md — 마케팅 프로젝트

## 기본
- 한국어, 해요체
- 마케팅 카피는 고객 pain point부터 시작
- 숫자/데이터 근거 포함
- 3단계 이상 → Plan Mode 필수

## 코드 규칙
- 모바일 퍼스트 반응형
- 페이지 로딩 3초 이내
- CTA 버튼 항상 노출
- Google Analytics 이벤트 추적 포함

## 교훈
- (실수할 때마다 추가)
```

✍️ 템플릿 2: 크리에이터용

```
# CLAUDE.md — 콘텐츠 프로젝트

## 기본
- 한국어, ~요체, 대화하듯 친근하게
- SEO 키워드 자연스럽게 포함
- 제목은 숫자 + 키워드 조합

## 콘텐츠 규칙
- 도입부 3줄 이내에 hook
- 소제목마다 핵심 한 줄 요약
- 비유 최소 2개
- 마무리에 CTA

## 교훈
- (실수할 때마다 추가)
```

🏥 템플릿 3: 병원 마케팅용

```
# CLAUDE.md — 병원 블로그

## 기본
- 한국어, 존댓말
- 의학 용어 → 괄호 안에 쉬운 설명

## 의료 규칙 (필수)
- 과장/허위 절대 금지
- 부작용/주의사항 반드시 포함
- "개인차가 있을 수 있습니다" 명시
- 의료법 광고 규정 준수

## 교훈
- (실수할 때마다 추가)
```

💼 템플릿 4: 사업가/MVP용

```
# CLAUDE.md — MVP 프로젝트

## 원칙
- 한국어, 해요체
- 최소 기능만 (over-engineering 금지)
- 3단계 이상 → Plan Mode 필수

## 기술 스택
- Frontend: React + Tailwind
- Backend: Supabase
- Deploy: Vercel

## 교훈
- (실수할 때마다 추가)
```

🎲 템플릿 5: 교육자용

```
# CLAUDE.md — 교육 콘텐츠

## 기본
- 한국어, 해요체
- 모든 전문 용어에 쉬운 설명 병기
- 비유와 예시 적극 활용

## 콘텐츠 규칙
- 학습 목표를 먼저 제시
- 단계별 난이도 표시 (☆~☆☆☆)
- 실습 과제 포함
- 핵심 정리로 마무리

## 교훈
- (실수할 때마다 추가)
```

> 💡 **꿀팁**
> 모든 템플릿의 공통점: **'교훈' 섹션**. Boris의 자기학습 루프를 적용한 것입니다. Claude가 실수할 때마다 '이 실수를 CLAUDE.md에 추가해줘'라고 말하세요.

부록

부록 C: 추천 GitHub 리포 20선

☆ 필수 (Top 5)

#	리포	Stars	용도
1	everything-claude-code	50K+	통합 설정 시스템
2	claude-code-best-practice	10K+	Boris 팁 + 커뮤니티 정리
3	awesome-claude-code-skills	22K+	1,234개 스킬 모음
4	Anthropic Cookbook	15K+	공식 예제/레시피
5	howborisusesclaudecode	-	Boris 57가지 팁 통합

🔧 도구/유틸 (5개)

#	리포	Stars	용도
6	gws (Google Workspace CLI)	4.9K+	Google 서비스 MCP
7	BMAD Framework	5K+	AI SDLC 프레임워크
8	MCP Servers (공식)	공식	공식 MCP 서버 모음
9	Claude GitHub App	공식	PR 자동 리뷰
10	AgentShield	ECC 내	보안 스캐너

🎨 스킬/디자인 (5개)

#	리포	Stars	용도
11	frontend-design (공식)	277K 설치	프로 웹 디자인
12	Boaz's production skills	커뮤니티	프로덕션 최적화
13	strategic-compact	ECC 내	스마트 압축 스킬
14	code-simplifier	커뮤니티	코드 간소화
15	seo-optimizer	커뮤니티	SEO 최적화

📚 학습/참고 (5개)

#	리포/URL	용도
16	code.claude.com/docs	공식 문서
17	builder.io/blog/claude-code-tips	50가지 팁 정리
18	mindwiredai.com (Boris 분석)	Boris 워크플로우 해설
19	getpushtoprod.substack.com	실전 사용기
20	ranthebuilder.cloud	프로덕션 사례

부록

부록 D: 트러블슈팅 FAQ 30선

설치/시작 (1~10)

Q1. "command not found: claude"가 나와요

A. npm install이 제대로 안 됐거나 PATH에 안 잡힌 경우. `npm install -g @anthropic-ai/claude-code`를 다시 실행하세요.

Q2. 로그인이 안 돼요

A. claude.ai 계정이 있어야 합니다. Pro 이상 구독이 필요합니다. 브라우저 팝업 차단을 해제하세요.

Q3. Windows에서 한글이 깨져요

A. Git Bash 또는 WSL 사용을 추천합니다. 프로젝트 폴더 경로에 한글이 없는지 확인하세요.

Q4. "Rate limit exceeded"가 나와요

A. 사용량 한도에 도달했습니다. Max 플랜으로 업그레이드하거나, Sonnet 모델로 전환하세요. `/model sonnet`

Q5. 어떤 모델을 써야 하나요?

A. 시작은 기본(Opus). 비용이 부담되면 Sonnet. 최적 조합은 opusplan (계획=Opus, 실행=Sonnet).

CLAUDE.md / 설정 (6~15)

Q6. CLAUDE.md를 어디에 만들어야 해요?

A. 프로젝트 폴더의 루트(최상위)에 만드세요. 파일명은 정확히 `CLAUDE.md` (대문자).

Q7. CLAUDE.md가 너무 길어졌어요

A. Boris의 100줄 원칙을 적용하세요. 실제로 문제를 해결하지 않는 줄은 삭제. 교훈 섹션만 계속 추가.

Q8. 여러 프로젝트에서 공통 규칙을 쓰고 싶어요

A. `~/.claude/CLAUDE.md`에 글로벌 규칙을, 각 프로젝트에 프로젝트별 규칙을 넣으세요. 둘 다 적용됩니다.

Q9. Claude가 CLAUDE.md를 무시해요

A. CLAUDE.md는 "80% 확률로 따르는 가이드"입니다. 반드시 지켜야 할 것은 Hook으로 만드세요.

Q10. settings.json은 어디 있어요?

A. `~/.claude/settings.json` (글로벌) 또는 `.claude/settings.json` (프로젝트). Claude에게 "settings.json 열어줘"라고 하면 됩니다.

작업 중 문제 (11~20)

Q11. Claude가 같은 실수를 반복해요

A. 자기학습 루프를 적용하세요. "방금 실수를 CLAUDE.md에 규칙으로 추가해줘."

Q12. 컨텍스트가 부족하다고 해요

A. `/compact`로 압축하거나 `/clear`로 새로 시작. AUTOCOMPACT_PCT를 50으로 설정.

Q13. 에러가 계속 안 고쳐져요

A. 같은 방법을 반복하고 있을 수 있습니다. "지금까지 시도한 것 전부 버리고, 근본 원인부터 다시 찾아줘."

Q14. Claude가 너무 많은 파일을 수정해요

A. 구체적으로 지시하세요. "src/auth.js의 login 함수만 수정해줘. 다른 파일은 건드리지 마."

Q15. 작업이 중간에 멈춰요

A. Claude가 허락을 기다리고 있을 수 있습니다. 터미널을 확인하세요. 또는 네트워크 문제일 수 있습니다.

비용/성능 (16~20)

Q16. 비용이 너무 많이 나와요

A. Part 6의 비용 절감 세팅을 적용하세요. MAX_THINKING_TOKENS, AUTOCOMPACT, opusplan.

Q17. 속도가 너무 느려요

A. Sonnet 모델로 전환하세요. `/model sonnet`. Opus보다 훨씬 빠릅니다.

Q18. Pro 구독인데 자주 제한돼요

A. 사용 패턴을 효율화하세요. /clear 자주 쓰기, 불필요한 파일 읽기 줄이기, 서브에이전트 활용.

Q19. MCP 서버를 몇 개까지 연결해도 되나요?

A. 동시에 3~4개까지가 적정. 각 MCP의 도구 설명이 토큰을 잡아먹으므로, 안 쓰는 건 끄세요.

Q20. API 사용인데 월 비용을 예측하고 싶어요

A. `/cost`로 세션 비용 확인. 가벼운 사용 $50~100/월, 보통 $100~200/월, 헤비 $300~500+/월.

고급 기능 (21~30)

Q21~Q30: Git Worktree 설정, Hook 디버깅, 플러그인 충돌 해결, Agent Teams 비용 관리, 텔레포트 사용법, 커스텀 Output Style, 팀 설정 공유, CI/CD 연동, 원격 서버 사용, 데스크톱 앱 설정 등은 에이나우 디스코드 #클로드코드-스터디 채널에서 실시간으로 질문하실 수 있습니다.

EPILOGUE

코딩을 모르는 당신이 가장 강력한 AI 생산자가 될 수 있는 이유

Boris Cherny의 100줄 CLAUDE.md가 800줄짜리를 이기는 이유,
변호사가 500명의 개발자를 이긴 이유,
이 모든 것의 공통분모는 같습니다.

도메인 전문성 × 올바른 도구 사용법

여러분은 이미 전문성을 가지고 있습니다.
이 책이 올바른 도구 사용법을 드렸습니다.

이제 만드세요.

에이나우 AI 교육 플랫폼 | ainow.co.kr
© 2026 AINOW. All rights reserved.

부록

부록 E: 용어 사전 40선

용어	설명
API	프로그램끼리 대화하는 통로. 레스토랑에서 주방(서버)과 손님(앱) 사이의 웨이터.
CLI	Command Line Interface. 글자로 명령하는 방식. Claude Code가 CLI 도구.
CLAUDE.md	Claude에게 주는 규칙서. 프로젝트 루트에 놓으면 매 세션 자동 로드.
컨텍스트 윈도우	Claude가 한 번에 기억할 수 있는 양. 약 200K 토큰 (영어 15만 단어 분량).
컴팩션	오래된 대화를 요약 압축하는 과정. /compact로 수동 실행 가능.
Deploy (배포)	만든 것을 인터넷에 올려서 다른 사람도 볼 수 있게 하는 것.
Git	코드의 변경 이력을 관리하는 시스템. '되돌리기'가 가능한 저장.
Git Worktree	하나의 저장소에서 여러 작업 공간을 만드는 Git 기능.
Hook	특정 이벤트 발생 시 자동 실행되는 규칙. "만약 ~하면 ~해라."
IDE	통합 개발 환경. 코드 작성에 편리한 프로그램. VS Code, JetBrains 등.
LSP	Language Server Protocol. 코드 에러를 자동 감지하는 기술.
MCP	Model Context Protocol. Claude를 외부 서비스(Notion, GitHub 등)에 연결.
MVP	Minimum Viable Product. 최소 기능만 갖춘 제품. 빠르게 만들어 검증.
Node.js	JavaScript 실행 환경. Claude Code가 돌아가는 엔진.
npm	Node Package Manager. 프로그램을 설치하는 도구. 앱스토어 같은 것.
Opus	Claude의 가장 강력한 모델. 느리지만 정확. Boris가 기본으로 사용.
Plan Mode	코드를 건드리지 않고 설계만 하는 모드. Shift+Tab 2번.
PR (Pull Request)	코드 변경을 제안하고 리뷰받는 것. Git의 협업 기능.
서브에이전트	메인 Claude의 조수 AI. 탐색/분석을 대신하고 결과만 보고.
Skill (스킬)	Claude에게 특정 작업을 잘하는 방법을 알려주는 레시피 파일.

용어	설명
Slash Command	/blog, /fix 같은 사용자 정의 명령어. .claude/commands/에 저장.
Sonnet	Claude의 중간 모델. 속도와 품질의 균형. 일상 작업에 적합.
Haiku	Claude의 경량 모델. 가장 빠르고 저렴. 서브에이전트에 적합.
Opusplan	계획은 Opus로, 실행은 Sonnet으로 하는 하이브리드 모드.
Token (토큰)	Claude가 텍스트를 읽는 단위. 영어 4글자 ≈ 1토큰. 한글 1자 ≈ 1~2토큰.
Teleport (텔레포트)	터미널 세션을 웹으로 (또는 반대로) 이동하는 기능.
Vercel	웹사이트를 무료로 인터넷에 올릴 수 있는 서비스.
Supabase	데이터베이스+인증+API를 제공하는 백엔드 서비스. Firebase 대안.
React	웹 UI를 만드는 도구. 가장 인기 있는 프론트엔드 프레임워크.
Tailwind CSS	CSS를 빠르게 작성하는 도구. 미리 만들어진 스타일을 조합.
TypeScript	JavaScript에 타입 검사를 추가한 언어. 에러를 미리 잡아줌.
JSON	데이터를 저장하는 형식. settings.json 같은 설정 파일에 사용.
Markdown	간단한 서식이 있는 텍스트 형식. # 제목, ** 굵게, - 목록.
Agent Teams	여러 Claude가 역할 분담하여 협업하는 기능. 토큰 7배 소모.
/loop	로컬에서 프롬프트를 반복 실행 (최대 3일).
/schedule	클라우드에서 프롬프트를 예약 실행 (컴퓨터 꺼도 동작).
SEO	Search Engine Optimization. 검색엔진에서 잘 노출되게 하는 작업.
CTA	Call To Action. "지금 시작하기" 같은 행동 유도 요소.
Rate Limit	사용량 제한. 일정 시간 내 요청 횟수가 초과되면 대기.
환경변수 (.env)	API 키 등 비밀 정보를 저장하는 파일. 코드에 직접 넣으면 위험.

이 책을 통해 알게 된 모든 것을 실전에 적용하세요

① CLAUDE.md에 교훈을 매일 추가하세요
② /clear와 /compact를 습관화하세요
③ Plan Mode로 시작하는 습관을 들이세요
④ 에이나우 커뮤니티에서 함께 성장하세요

🔗 ainow.co.kr | 디스코드: #클로드코드-스터디

부록

부록 F: 실전 프롬프트 모음 30선

바로 복사해서 쓸 수 있는 프롬프트 30개입니다.

🚀 시작할 때 (1~5)

1. 새 프로젝트 시작

```
> [서비스명]을 위한 웹사이트를 만들어줘. [서비스 설명]. 먼저 어떤 페이지와 기능이 필요한지 계획을 세워줘.
```

2. 기존 프로젝트 파악

```
> 이 프로젝트의 전체 구조를 파악해줘. 주요 파일별 역할과 기술 스택을 정리해줘.
```

3. CLAUDE.md 초기화

```
> /init
```

4. 프로젝트 상태 점검

```
> 이 프로젝트의 현재 상태를 점검해줘. 미완성 기능, 에러, 개선 가능한 부분을 정리해줘.
```

5. 기술 스택 추천

```
> [목적]을 위한 프로젝트를 만들 건데, 초보자에게 적합한 기술 스택을 추천해줘. 이유도 설명해줘.
```

🔨 만들 때 (6~15)

6. 랜딩페이지

```
> [서비스]의 랜딩페이지를 만들어줘. 히어로, 기능 소개, 고객 후기, CTA 섹션 포함. 모바일 반응형으로.
```

7. 대시보드

```
> [데이터]를 시각화하는 대시보드를 만들어줘. 차트, 요약 숫자, 필터 기능 포함.
```

8. 폼 + 데이터 수집

```
> [정보]를 수집하는 폼을 만들어줘. 제출하면 Google Sheets에 자동 저장되게.
```

9. 다크모드 추가

```
> 이 프로젝트에 다크모드를 추가해줘. 토글 버튼 + 선택 기억 기능 포함.
```

10. 로그인 기능

```
> 이메일+비밀번호 로그인 기능을 추가해줘. 회원가입도 포함. 로그인 안 하면 메인 페이지만 볼 수 있게.
```

11. API 연동

```
> [외부 서비스] API를 연동해줘. API 키는 환경변수(.env)에서 읽어오게. 에러 핸들링도 포함.
```

12. 검색 기능

```
> 이 프로젝트에 검색 기능을 추가해줘. 실시간 검색(타이핑하면서 결과 표시)으로.
```

13. 이메일 템플릿

```
> [목적]을 위한 HTML 이메일 템플릿을 만들어줘. 모든 이메일 클라이언트에서 잘 보이게.
```

14. 자동화 스크립트

```
> [반복 작업]을 자동화하는 스크립트를 만들어줘. 매일 [시간]에 실행되게.
```

15. 간단한 게임

```
> 브라우저에서 동작하는 간단한 [게임 종류]를 만들어줘. 점수 기록 + 재시작 기능 포함.
```

🔧 고칠 때 (16~20)

16. 에러 수정

```
> 이 에러를 고쳐줘. 고친 다음 테스트해서 확인해줘. 뭘 고쳤는지 한 줄로 설명해줘.
[에러 메시지]
```

17. 반복 에러

```
> 같은 에러가 계속 나와. 지금까지 시도한 것: [목록]. 근본 원인을 찾아서 완전히 해결해줘.
```

18. 성능 개선

```
> 이 프로젝트의 성능을 점검하고, 느린 부분을 찾아서 고쳐줘.
```

19. 보안 점검

```
> 이 프로젝트에 보안 문제가 있는지 점검해줘. 초보자가 흔히 놓치는 것 위주로.
```

20. 코드 정리

```
> 이 프로젝트의 코드를 정리해줘. 중복 제거, 변수명 개선, 주석 추가.
```

📝 콘텐츠 (21~25) / 🚀 배포 (26~30)

21~25: 블로그 글, 이메일 뉴스레터, SNS 캡션, 유튜브 대본, 카드뉴스 생성

26~30: Vercel 배포, GitHub Pages 배포, 도메인 연결, SSL 설정, CI/CD 파이프라인

(상세 프롬프트는 에이나우 디스코드 #프롬프트-공유 채널에서 확인하세요)

부록

부록 G: 에이나우 리소스 가이드

에이나우 커뮤니티

채널	내용	접속
디스코드	실시간 Q&A, 스터디, 프로젝트 공유	discord.gg/ainow
네이버 카페	정보 공유, 후기, 모집	cafe.naver.com/ainow
카카오톡	공지, 이벤트	open.kakao.com/ainow

에이나우 교육 프로그램

프로그램	내용	기간
AI 생산자 부트캠프	60일 집중 과정, 팀 프로젝트	8주
Claude Code 마스터 VOD	이 책의 영상 버전 + 실습	자율
AI 마케팅 실전	퍼포먼스 마케팅 + AI 자동화	4주

이 책의 업데이트

Claude Code는 빠르게 발전하는 도구입니다. 이 책도 함께 업데이트됩니다.

- **버전 히스토리**: v1.0 (2026년 3월) — 초판
- **업데이트 알림**: 에이나우 디스코드에서 새 버전 공지
- **피드백**: 오류 제보, 추가 요청은 디스코드 #책-피드백 채널

감사의 말

이 책이 나오기까지 도움을 주신 분들에게 감사드립니다.

- **Boris Cherny** — Claude Code를 만들고, 워크플로우를 공개적으로 공유해주신 것에 감사합니다.
- **Affaan Mustafa** — Everything Claude Code를 오픈소스로 공개해주신 것에 감사합니다.
- **에이나우 커뮤니티 멤버 여러분** — 매일 함께 성장하는 1만 명의 AI 생산자들에게 감사합니다.
- **알잘스튜디오 팀** — 실전에서 Claude Code를 함께 활용하며 교훈을 쌓아주셔서 감사합니다.

마지막으로

이 책을 읽었다면, 이제 만드세요.
완벽하지 않아도 됩니다.
CLAUDE.md 5줄부터 시작하세요.
교훈을 하나씩 쌓으세요.
1개월 후, 여러분은 이미 AI 생산자입니다.

— 신영선, 에이나우

부록

부록 H: Claude Code 업데이트 히스토리

Claude Code의 주요 업데이트 타임라인입니다. 어떤 기능이 언제 추가되었는지 참고하세요.

날짜	버전/이벤트	주요 변경
2025년 중반	Claude Code 출시	Anthropic이 개발한 에이전틱 코딩 도구 첫 공개
2025년 9월	Forum Ventures 해커톤	Affaan 팀 우승, Everything Claude Code 탄생
2025년 말	MCP 도입	외부 서비스 연결 기능 추가
2026년 1월 2일	Boris Tips Part 1	13가지 워크플로우 팁 공개 (병렬 세션, Plan Mode)
2026년 1월 31일	Boris Tips Part 2	10가지 팀 팁 (Worktree, 피드백 루프)
2026년 2월 11일	Boris Tips Part 3	12가지 커스터마이제이션 (플러그인, Hook, Output Style)
2026년 2월	Opus 4.6 해커톤	비개발자 Mike Brown 1등 (13K 지원자 중)
2026년 2월	플러그인 시스템	/plugin 명령으로 LSP, MCP, 스킬 통합 관리
2026년 2월	Agent Teams	여러 Claude가 역할 분담 협업
2026년 초	/loop, /schedule	반복 실행과 예약 실행 기능
2026년 3월	커뮤니티 스킬 1,234개+	전 세계 커뮤니티 기여 스킬 급증

ℹ 알아두기

Claude Code는 거의 매주 업데이트됩니다. 최신 변경사항은 code.claude.com/docs 또는 Anthropic 공식 블로그에서 확인하세요.

이 책에서 다루지 않은 고급 주제

이 책은 왕초보~중급자를 대상으로 합니다. 다음 주제는 향후 업데이트 또는 에이나우 고급 강의에서 다룰 예정입니다.

- **CI/CD 연동**: GitHub Actions에서 Claude Code 자동 실행
- **프로덕션 모니터링**: 배포된 서비스의 에러 자동 감지 + 수정
- **멀티 리포 관리**: 여러 프로젝트에 걸친 리팩토링
- **커스텀 모델 파인튜닝**: 특정 도메인에 최적화
- **엔터프라이즈 도입**: 대기업에서의 보안/규정 준수 설정
- **REST API 서버 구축**: Claude Code로 백엔드 서비스 전체 구현
- **모바일 앱 개발**: React Native로 iOS/Android 앱 만들기

부록

부록 I: 빠른 참조 카드

인쇄해서 모니터 옆에 붙여두세요.

필수 명령어

명령어	기능
`claude`	시작
`claude --continue`	마지막 세션 이어하기
`/clear`	대화 초기화
`/compact`	대화 압축
`/cost`	비용 확인
`/model [name]`	모델 변경
`/init`	CLAUDE.md 자동 생성
`Shift+Tab`	Plan Mode 전환
`Esc`	작업 중단
`Esc+Esc`	체크포인트 복원
`!`	터미널 명령 직접 실행
`@file`	특정 파일 참조

CLAUDE.md 최소 템플릿

```
# CLAUDE.md
- 한국어, 해요체
- 코드 수정 전 설명
- 3단계 이상 → Plan Mode

## 교훈
- (실수할 때마다 추가)
```

settings.json 최소 템플릿

```
{{
  "env": {{
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }}
}}
```

Boris 핵심 3원칙

매일 기억하세요

① **CLAUDE.md는 짧게** — 100줄 이내, 모든 줄이 실제 문제 해결

② **자기학습 루프** — Claude가 실수하면 CLAUDE.md에 교훈 추가

③ **증명을 요구하라** — '이게 맞다고 증명해봐. 테스트를 돌려.'

에이나우 클로드 코드 실전 마스터 가이드

v1.0 | 2026년 3월 | 신영선 저

© 2026 AINOW. All rights reserved.

ainow.co.kr | 디스코드: #클로드코드-스터디

실습 튜토리얼 A: 개인 블로그 처음부터 끝까지

이 튜토리얼은 Claude Code로 개인 블로그를 처음부터 만들고 배포하는 전 과정을 단계별로 따라합니다. 소요 시간 약 1시간.

준비물

- Claude Code 설치 완료 (Part 1 참고)
- GitHub 계정 (무료)

- Vercel 계정 (무료, GitHub으로 가입)

Step 1: 프로젝트 시작 (5분)

```
mkdir my-blog && cd my-blog
claude

# CLAUDE.md 만들기
> CLAUDE.md를 만들어줘:
   - 한국어, ~요체
   - 모바일 반응형 필수
   - 깔끔하고 미니멀한 디자인
   - 다크 테마 기본, 라이트 모드 전환 가능
   - 페이지 로딩 빠르게
```

Step 2: 블로그 구조 설계 (10분)

```
> [Plan Mode — Shift+Tab 2번]

> 개인 블로그를 만들어줘.

   페이지:
   1. 메인 (최신 글 목록)
   2. 글 읽기 페이지
   3. About 페이지

   기능:
   - 태그별 필터링
   - 다크/라이트 모드 전환
   - 글 목록에 날짜, 제목, 미리보기 표시
   - 부드러운 페이지 전환

   기술:
   - HTML + CSS + JavaScript (프레임워크 없이)
   - 글은 HTML 파일로 관리 (CMS 없이)

   구조를 먼저 설계해줘. 어떤 파일이 필요한지.
```

Claude가 파일 구조를 제안하면 검토하고, 필요한 수정을 요청합니다.

```
Claude: 다음 구조를 제안합니다:
  index.html       - 메인 (글 목록)
  about.html       - 소개 페이지
  posts/           - 블로그 글 폴더
    post-1.html    - 첫 번째 글
  css/style.css    - 스타일
  js/main.js       - 다크모드, 필터 등

> 좋아. 이 구조로 진행해줘.
```

Step 3: 구현 (20분)

```
> [일반 모드로 전환]
> 위 계획대로 블로그를 구현해줘.

  디자인 참고:
  - 배경: #0f0f0f (거의 검은색)
  - 텍스트: #e0e0e0 (밝은 회색)
  - 강조: #4f9cf7 (부드러운 파란색)
  - 카드: #1a1a1a (약간 밝은 검은색)
  - 둥근 모서리, 부드러운 그림자

  샘플 블로그 글 2개도 만들어줘:
  - 글 1: "AI 시대에 글쓰기가 더 중요해진 이유"
  - 글 2: "Claude Code로 1시간 만에 블로그 만들기"
```

Step 4: 확인 + 수정 (10분)

```
# 브라우저에서 index.html 열어 확인 후

> 몇 가지 수정해줘:
  1. 글 카드에 마우스 올리면 살짝 위로 올라오는 효과
  2. 다크↔라이트 전환 시 부드러운 트랜지션
  3. 모바일에서 글 카드가 한 줄에 1개씩 나오게
  4. 페이지 맨 위로 이동하는 버튼 추가
```

```
# 글 내용 수정
> 첫 번째 글의 도입부를 좀 더 후킹하게 바꿔줘.
  "AI가 글쓰기를 대체할까요?"로 시작하는 질문으로.
```

Step 5: 배포 (10분)

```
> Git을 초기화하고 GitHub에 올려줘.
  리포 이름: my-blog
  public 리포로.

# Claude가 단계별로 안내합니다
# 1. git init + git add + git commit
# 2. GitHub 리포 생성 안내
# 3. git push
```

```
# Vercel 배포
> Vercel에 배포하는 방법을 알려줘.
  GitHub 리포를 이미 올렸어.
```

Step 6: 새 글 추가하는 워크플로우

```
# 슬래시 커맨드로 블로그 글 추가 자동화
> 새 블로그 글을 추가하는 슬래시 커맨드를 만들어줘.
  /new-post [제목]을 실행하면:
  1. posts/ 폴더에 새 HTML 파일 생성
  2. index.html의 글 목록에 자동 추가
  3. SEO 메타 태그 자동 생성
  4. 날짜 자동 입력
```

완성! 🎉

1시간 만에 완성된 결과물:

✅ 다크모드 개인 블로그

✅ 글 2개 + 새 글 추가 자동화

✅ Vercel 배포 완료

✅ GitHub 연동 (코드 Push = 자동 업데이트)

실습 튜토리얼 B: 이메일 템플릿 시스템

자주 보내는 이메일 유형별 템플릿을 Claude Code로 만들고, 슬래시 커맨드로 즉시 생성합니다.

Step 1: 이메일 유형 정의

```
> [Plan Mode]
> 우리 회사에서 자주 보내는 이메일 유형:
  1. 신규 고객 환영 이메일
  2. 견적서 발송 이메일
  3. 프로젝트 완료 안내
  4. 미팅 일정 확인
  5. 피드백 요청

  각 유형별로 HTML 이메일 템플릿을 만들어줘.
  변수 부분([고객명], [금액] 등)은 나중에
  쉽게 바꿀 수 있는 구조로.
```

Step 2: 템플릿 구현

```
> [일반 모드]
> 5가지 이메일 템플릿을 만들어줘.

  공통 디자인:
  - 회사 로고 자리 (상단)
  - 깔끔한 레이아웃
  - 모바일에서도 잘 보이게
  - 하단에 회사 정보 + 연락처
  - 브랜드 색상: #2563eb (파란색)

  각 템플릿을 별도 HTML 파일로.
```

Step 3: 슬래시 커맨드 생성

```
# .claude/commands/email.md

이메일 작성 도우미

유형: $ARGUMENTS

## 유형별 동작
- "환영": templates/welcome.html 기반, 고객명/서비스명 입력 받기
- "견적": templates/quote.html 기반, 항목/금액/기한 입력 받기
- "완료": templates/complete.html 기반, 프로젝트명/결과 입력 받기
- "미팅": templates/meeting.html 기반, 날짜/시간/장소 입력 받기
- "피드백": templates/feedback.html 기반, 프로젝트명 입력 받기

## 출력
- 완성된 HTML 이메일을 output/ 폴더에 저장
- 브라우저에서 미리보기 가능하게
```

```
# 사용 예시
> /email 환영

Claude: 고객명을 알려주세요.
> 김철수

Claude: 서비스명을 알려주세요.
> AI 마케팅 자동화 프로

Claude: ✓ output/welcome-김철수.html 생성 완료
```

> 💡 **꿀팁**
> 생성된 HTML 이메일을 Gmail에서 보내려면, 브라우저에서 열고 Ctrl+A(전체 선택) → Ctrl+C(복사) → Gmail 작성 창에 Ctrl+V(붙여넣기)하면 서식이 유지됩니다.

실습 튜토리얼 C: CSV 데이터 분석 + 시각화

엑셀 대신 Claude Code로 데이터를 분석하고 차트를 만들어봅니다.

시나리오

온라인 쇼핑몰의 매출 데이터(CSV 파일)를 분석해서 인사이트를 추출하고, 시각적인 리포트를 만듭니다.

```
> sales-data.csv 파일을 분석해줘.

  알고 싶은 것:
  1. 월별 매출 추이 (그래프)
  2. 상품 카테고리별 매출 비율 (파이 차트)
  3. 요일별 주문 패턴 (막대 그래프)
  4. 평균 주문 금액 추이
  5. 상위 10개 상품

  결과를 하나의 HTML 대시보드로 만들어줘.
  Chart.js를 사용해서 예쁜 차트로.
```

데이터가 없다면: 더미 데이터 생성

```
> 온라인 쇼핑몰의 6개월치 매출 더미 데이터를 만들어줘.
  CSV 형식으로.

  컬럼: 날짜, 주문ID, 상품명, 카테고리, 수량, 단가, 총액
  카테고리: 의류, 식품, 전자기기, 생활용품, 화장품
  데이터 양: 약 1,000건
  현실적인 패턴으로 (주말 주문 많고, 연말 매출 높고)
```

심화: 자동 인사이트 추출

```
> 이 데이터에서 사업적으로 의미 있는 인사이트를
  자동으로 추출해줘.

  예시:
  - "화장품 카테고리는 주말에 매출이 40% 높음"
  - "평균 주문 금액이 3개월간 15% 상승"
  - "화요일이 가장 매출이 낮음"

  인사이트를 대시보드에 텍스트로 표시해줘.
```

비개발자도 데이터 분석가

엑셀의 피벗 테이블보다 강력하고, Tableau보다 쉽습니다. Claude에게 '이 데이터에서 뭘 알 수 있어?'라고 물어보면 됩니다.

심화 가이드: n8n + Claude Code 자동화

n8n은 무료 자동화 도구(Zapier의 오픈소스 버전)입니다. Claude Code와 조합하면 강력한 자동화 파이프라인을 만들 수 있습니다.

자동화 파이프라인 예시: 경쟁사 모니터링

단계	도구	행동
1	n8n (RSS)	경쟁사 블로그 RSS 피드 구독
2	n8n (HTTP)	새 글이 올라오면 본문 수집
3	Claude API	본문을 한국어로 요약
4	n8n (Discord)	요약을 팀 디스코드에 전송

```
# Claude Code로 n8n 워크플로우 JSON 생성
> n8n 워크플로우 JSON을 만들어줘.

  흐름:
  1. RSS 피드 (경쟁사 블로그 URL) 모니터링
  2. 새 글 발견 시 HTTP로 본문 가져오기
  3. Claude API로 한국어 요약 (200자 이내)
  4. Discord 웹훅으로 전송

  JSON을 import할 수 있는 형태로 만들어줘.
```

자동화 파이프라인 예시: 리드 → CRM → 이메일

단계	도구	행동
1	랜딩페이지	이메일 수집
2	Google Sheets	데이터 저장
3	n8n (Trigger)	새 행 감지
4	Claude API	개인화된 환영 이메일 생성
5	n8n (Gmail)	이메일 자동 발송
6	Notion MCP	CRM에 고객 정보 추가

ℹ 알아두기

n8n을 모르는 분: n8n.io에서 무료로 시작할 수 있습니다. 드래그 앤 드롭으로 자동화를 만드는 도구입니다. Claude Code와 함께 쓰면, Claude가 n8n 워크플로우 JSON까지 만들어줍니다.

Claude Code + 외부 도구 조합 총정리

목적	도구 조합
웹사이트 제작 + 배포	Claude Code + Vercel
데이터 저장 + API	Claude Code + Supabase
작업 자동화	Claude Code + n8n
팀 협업	Claude Code + GitHub + Slack
콘텐츠 관리	Claude Code + Notion MCP
데이터 수집	Claude Code + Google Sheets
이메일 마케팅	Claude Code + Gmail API
모니터링	Claude Code + n8n + Discord

프롬프트 엔지니어링 심화

기본 원칙(Part 1)을 넘어서, Claude Code에서 더 나은 결과를 얻는 고급 기법입니다.

기법 1: Few-shot 프롬프팅

좋은 예시와 나쁜 예시를 보여주면 Claude가 패턴을 파악합니다.

```
> 블로그 글 제목을 만들어줘.

  좋은 예시:
  - "2026년 AI 마케팅 자동화 7가지 방법"
  - "월 100만 원 절약하는 클로드 코드 활용법"

  나쁜 예시:
  - "AI 마케팅에 대하여"
  - "클로드 코드 사용법"

  주제: "비개발자의 앱 개발"
```

기법 2: 역할 + 제약 조건

```
> 너는 10년차 UX 디자이너야.
  이 랜딩페이지의 사용자 경험을 평가해줘.

  제약 조건:
  - 모바일 사용자가 70%
  - 타겟 연령대: 30~40대
  - 평균 체류 시간 목표: 2분 이상

  각 섹션별로 점수(1~10)와 개선 방안을 알려줘.
```

기법 3: Chain-of-Thought (단계별 사고)

```
> 이 프로젝트에서 성능 병목을 찾아줘.

  다음 순서로 분석해줘:
  1. 먼저 전체 파일 구조를 파악해
  2. 각 파일의 크기와 복잡도를 측정해
  3. 잠재적 병목 포인트를 나열해
  4. 각 병목의 영향도를 평가해 (1~10)
  5. 가장 영향이 큰 것부터 해결 방안을 제시해
```

기법 4: 반복 개선 (Iterative Refinement)

```
# 1차: 빠르게 만들기
> 가격 비교 표를 만들어줘. 빠르게, 기본적인 것만.

# 2차: 개선하기
> 좋아. 이제 디자인을 개선해줘.
  가장 인기 있는 플랜을 강조하고,
  CTA 버튼을 각 플랜에 추가해.

# 3차: 세부 조정
> 모바일에서 표가 가로로 넘치는데,
  모바일에서는 카드 형태로 바꿔줘.
```

기법 5: 제한 프롬프팅 (Constraint-based)

```
> 이 기능을 만들되, 다음 제한을 지켜줘:
  - 새 파일을 만들지 마 (기존 파일만 수정)
  - 외부 라이브러리 추가하지 마
  - 기존 API 인터페이스를 바꾸지 마
  - 코드 추가는 50줄 이내로
```

> 💡 **꿀팁**
>
> **가장 효과적인 순서:**
> ① 역할 부여 → ② 맥락 제공 → ③ 구체적 지시 → ④ 제약 조건 → ⑤ 출력 형식 지정
> 이 5가지를 한 프롬프트에 담으면 90% 이상의 상황에서 원하는 결과를 얻습니다.

AI 생산자의 마인드셋

도구를 배우는 것을 넘어서, AI 시대를 살아가는 마인드셋에 대해 이야기합니다.

> *AI 시대의 경쟁력은 '무엇을 알고 있는가'가 아니라 '무엇을 만들 수 있는가'입니다. 지식은 AI가 가지고 있습니다. 여러분이 가져야 할 것은 문제를 정의하고 해결책을 설계하는 능력입니다.*
>
> — 신영선, 에이나우

소비자 vs 생산자

	AI 소비자	AI 생산자
행동	ChatGPT에 질문 → 답변 받음	Claude Code로 서비스/도구 만듦
가치	정보를 얻음	가치를 만듦
수입	기존 업무 효율화	새로운 수익 창출
경쟁력	AI 사용 능력	도메인 전문성 + AI 실행력
미래	AI가 대체할 수 있음	AI로 대체할 수 없음

AI 생산자가 되는 3단계

1. **자기 분야의 문제를 정확히 파악하기**
   여러분이 매일 겪는 반복적인 고통, 비효율, 불편함을 리스트업하세요. 이것이 가장 귀한 자산입니다. 개발자는 이걸 모릅니다.
2. **Claude Code로 해결책 프로토타입 만들기**
   완벽할 필요 없습니다. '대충 되는' MVP를 빠르게 만들어 검증하세요. 이 책의 Part 5가 이 단계를 안내합니다.
3. **피드백으로 개선하고 확장하기**
   사용자(또는 자기 자신)의 피드백을 기반으로 개선합니다. CLAUDE.md에 교훈을 쌓듯, 제품에도 학습을 쌓아가세요.

에이나우의 약속

에이나우는 AI 소비자가 아닌 **AI 생산자**를 키우는 플랫폼입니다. 이 책은 그 여정의 첫걸음입니다.

- **커뮤니티**: 혼자가 아닌 함께 성장합니다
- **실전**: 이론이 아닌 만들면서 배웁니다
- **지속**: 한 번이 아닌 꾸준히 업데이트합니다

이 책의 마지막 미션

이 책을 다 읽었다면, 딱 하나만 하세요:

1. 터미널을 여세요
2. `mkdir my-first-project && cd my-first-project`
3. `claude`
4. CLAUDE.md 5줄을 만드세요
5. 여러분이 매일 겪는 문제 하나를 Claude에게 말하세요

시작이 반입니다. 나머지 반은 에이나우가 함께합니다.

실습 튜토리얼 D: 자동 포트폴리오 생성기

JSON 파일 하나만 수정하면 포트폴리오 전체가 업데이트되는 시스템을 만듭니다.

```
> JSON 기반 포트폴리오 생성기를 만들어줘.

  data.json에 내 정보를 넣으면 자동으로
  포트폴리오 웹사이트가 생성되는 구조.

  data.json 구조:
  - name, title, bio
  - skills: [skill1, skill2, ...]
  - projects: [{{name, description, url, image}}]
  - experience: [{{company, role, period, description}}]
  - contact: {{email, github, linkedin}}

  HTML 템플릿이 data.json을 읽어서 렌더링.
  디자인: 미니멀, 다크 테마, 부드러운 애니메이션.
```

왜 이 구조가 좋은가

- **콘텐츠와 디자인 분리**: JSON만 수정하면 내용 업데이트 완료
- **비개발자 친화적**: JSON은 구조가 단순해서 누구나 수정 가능
- **재사용 가능**: 다른 사람에게 data.json 양식만 보내면 됨
- **자동화 가능**: Claude에게 "새 프로젝트 추가해줘"만 말하면 됨

```
# data.json 예시
{{
  "name": "홍길동",
  "title": "디지털 마케터 & AI 생산자",
  "bio": "5년차 퍼포먼스 마케터. AI로 마케팅 ROI 200% 달성.",
  "skills": [
    {{"name": "퍼포먼스 마케팅", "level": 95}},
    {{"name": "AI 자동화", "level": 85}},
    {{"name": "콘텐츠 기획", "level": 90}},
    {{"name": "데이터 분석", "level": 80}}
  ],
  "projects": [
    {{
      "name": "AI 블로그 자동화 시스템",
      "description": "월 30개 블로그 글 자동 생성, SEO 점수 95+ 유지",
      "url": "https://example.com",
      "tags": ["AI", "마케팅", "자동화"]
    }}
  ]
}}
```

꿀팁

이 포트폴리오 생성기를 GitHub에 올리고, README에 사용법을 잘 적어두면 다른 사람도 Fork해서 쓸 수 있습니다. 오픈소스 기여의 첫 걸음!

실습 튜토리얼 E: 가격 계산기 + 견적서 생성기

서비스업, 프리랜서, 에이전시에서 유용한 자동 견적 도구입니다.

```
> 자동 견적서 생성기를 만들어줘.

  입력 화면:
  - 서비스 종류 선택 (드롭다운)
  - 옵션 체크 (추가 기능들)
  - 수량/기간 입력
  - 고객 정보 입력

  서비스 종류별 기본 가격:
  - 웹사이트 제작: 300만원~
  - 로고 디자인: 50만원~
  - SNS 관리: 월 100만원~
  - SEO 최적화: 월 80만원~

  옵션:
  - 반응형 추가: +50만원
  - 다국어 지원: +100만원
  - 유지보수 계약: 월 30만원

  결과:
  - 실시간 가격 계산 (체크할 때마다 업데이트)
  - "견적서 생성" 버튼 → 깔끔한 PDF 형태 HTML 출력
  - 견적서에 회사 로고, 항목별 가격, 합계, 유효 기간 포함
```

견적서 출력 예시

```
┌──────────────────────────────────────────────────────────┐
│          [회사 로고]                                      │
│                                                          │
│   견적서                                                  │
│   견적 번호: Q-2026-001                                   │
│   날짜: 2026년 3월 28일                                   │
│                                                          │
│   고객: 홍길동 님                                         │
│                                                          │
│   항목            수량    단가       금액 │
│   ──────────────────────────────────────────────────────  │
│   웹사이트 제작    1   3,000,000  3,000,000 │
│   반응형 추가      1     500,000    500,000 │
│   유지보수(월)     3     300,000    900,000 │
│   ──────────────────────────────────────────────────────  │
│   합계                          4,400,000원 │
│   부가세(10%)                     440,000원 │
│   총액                          4,840,000원 │
│                                                          │
│   유효 기간: 발행일로부터 30일                            │
└──────────────────────────────────────────────────────────┘
```

실전 활용

이 견적서 생성기를 웹사이트에 올려두면:

① 고객이 직접 옵션을 선택하고 예상 비용 확인

② '견적 요청' 버튼 → 이메일로 자동 전송

③ 상담 전에 예산 범위 조율 완료 → 미팅 효율 2배

실습 튜토리얼 F: 인터랙티브 프레젠테이션

파워포인트 대신, 웹으로 프레젠테이션을 만듭니다. 클릭/스크롤로 넘기고, 애니메이션도 넣을 수 있습니다.

```
> 웹 프레젠테이션을 만들어줘.

  주제: "AI 시대, 마케터가 알아야 할 5가지"

  슬라이드:
  1. 타이틀 (큰 제목 + 부제)
  2. 문제 제기 (마케팅의 변화)
  3~7. 5가지 포인트 (각 1슬라이드)
  8. 요약
  9. Q&A + 연락처

  기능:
  - 화살표 키로 슬라이드 이동
  - 프로그레스 바 (현재 위치)
  - 각 슬라이드 진입 시 텍스트 페이드인
  - 발표자 노트 (N키로 토글)
  - 전체화면 (F키)

  디자인: 다크, 깔끔, 글꼴 큰 사이즈
```

💡 꿀팁

왜 웹 프레젠테이션인가:

- URL 하나로 공유 (파일 전송 불필요)
- 모바일에서도 볼 수 있음
- 인터랙티브 요소(차트, 애니메이션) 자유롭게
- 버전 관리 가능 (Git)
- Vercel 배포 후 어디서든 접속

Claude Code로 프레젠테이션 자료 준비하기

```
# 슬래시 커맨드
# .claude/commands/slides.md

웹 프레젠테이션 생성

주제: $ARGUMENTS

## 단계
1. 주제 분석 → 핵심 메시지 도출
2. 슬라이드 구성 제안 (8~12장)
3. 각 슬라이드의 핵심 텍스트 작성
4. HTML 파일 생성 (reveal.js 스타일)
5. 발표자 노트 포함

## 규칙
- 슬라이드당 텍스트 3줄 이내
- 핵심 숫자/키워드는 크게
- 시각적 대비 (큰 텍스트 + 작은 부연)
- 마지막 슬라이드에 CTA
```

```
> /slides 비개발자를 위한 Claude Code 소개 - 30분 발표
```

프로젝트 아이디어 50선: 업종별

무엇을 만들지 모르겠다면, 이 목록에서 골라보세요.

📣 마케팅/영업 (10개)

#	프로젝트	난이도
1	리드 수집 랜딩페이지	⭐
2	A/B 테스트 페이지	⭐⭐
3	자동 견적서 생성기	⭐⭐
4	이메일 템플릿 시스템	⭐⭐
5	경쟁사 모니터링 대시보드	⭐⭐⭐
6	광고 카피 생성기	⭐
7	UTM 링크 생성기	⭐
8	고객 후기 수집 폼	⭐
9	QR코드 생성 + 랜딩	⭐⭐
10	마케팅 ROI 계산기	⭐⭐

🎨 콘텐츠/크리에이터 (10개)

#	프로젝트	난이도
11	개인 블로그	⭐
12	카드뉴스 자동 생성기	⭐⭐
13	유튜브 대본 생성기	⭐
14	콘텐츠 리퍼포징 시스템	⭐⭐
15	뉴스레터 자동화	⭐⭐
16	포트폴리오 사이트	⭐
17	온라인 이력서	⭐
18	팟캐스트 쇼노트 생성	⭐⭐
19	해시태그 분석기	⭐⭐
20	콘텐츠 캘린더	⭐⭐

💼 사업/운영 (10개)

#	프로젝트	난이도
21	고객 CRM	★★★
22	매출 대시보드	★★
23	재고 관리 도구	★★★
24	직원 근태 관리	★★★
25	프로젝트 타임라인	★★
26	SaaS 대기자 페이지	★
27	피드백 수집 도구	★★
28	자동 인보이스 생성	★★
29	KPI 트래커	★★
30	회의록 자동 정리	★★

🏥 의료/건강 (10개)

#	프로젝트	난이도
31	시술 비교 블로그 자동화	★★
32	환자 FAQ 챗봇 페이지	★★★
33	시술 가격표 웹페이지	★
34	의료진 소개 페이지	★
35	진료 예약 폼	★★
36	건강 정보 뉴스레터	★★
37	시술 후기 관리 도구	★★
38	의료 콘텐츠 SEO 분석	★★★
39	원내 교육 자료 생성	★★
40	환자 만족도 조사 도구	★★

📚 교육/기타 (10개)

#	프로젝트	난이도
41	퀴즈 자동 생성기	☆☆
42	학습 진도 트래커	☆☆
43	단어장/플래시카드	☆☆
44	수강생 관리 시스템	☆☆☆
45	온라인 수료증 생성	☆☆
46	이벤트 초대장 페이지	☆
47	레시피 블로그	☆
48	운동 루틴 트래커	☆☆
49	독서 기록 앱	☆☆
50	가계부 대시보드	☆☆

50개 중 1개만 고르세요

지금 가장 필요한 것, 또는 가장 재미있어 보이는 것 딱 1개만 골라서 시작하세요.

☆ = 30분~1시간
☆☆ = 1~3시간
☆☆☆ = 1~3일

Part 5의 실전 프로젝트 패턴(Plan Mode → 구현 → 배포)을 그대로 따르면 됩니다.

Claude Code 성공을 위한 10가지 습관

도구를 아는 것과 잘 쓰는 것은 다릅니다. 이 10가지 습관이 차이를 만듭니다.

1. **매일 CLAUDE.md에 교훈 1개 추가**
   Boris의 자기학습 루프. 1주일이면 7개, 1개월이면 30개. Claude가 점점 똑똑해지는 걸 체감합니다.
2. **작업 전에 /clear**
   깨끗한 컨텍스트에서 시작하면 품질이 올라갑니다.
3. **큰 작업은 항상 Plan Mode**
   5분의 계획이 30분의 재작업을 방지합니다.
4. **에러를 두려워하지 않기**
   에러 = 피드백. '고쳐줘' 한 마디면 됩니다.

5 **한 입 크기로 쪼개기**
한 프롬프트에 한 작업. 이 습관이 결과 품질을 3배 올립니다.

6 **결과를 직접 테스트하기**
Claude의 '완료'를 믿지 말고, 직접 확인하세요.

7 **반복 작업을 커맨드로 만들기**
3번 이상 한 작업은 슬래시 커맨드로 자동화하세요.

8 **주 1회 CLAUDE.md 정리**
쌓인 교훈을 카테고리별로 정리하면 더 효과적입니다.

9 **커뮤니티에 참여하기**
다른 사람의 설정, 프롬프트, 프로젝트에서 배웁니다.

10 **작은 것부터 시작하기**
쇼핑몰이 아니라 자기소개 페이지부터. 성공 경험을 쌓으세요.

> *도구를 배우는 가장 좋은 방법은 매일 쓰는 것입니다. 특별한 프로젝트를 기다리지 마세요. 오늘의 사소한 불편함을 해결하는 것부터 시작하세요.*
>
> — 에이나우 커뮤니티 멘토

감사합니다

이 책을 끝까지 읽어주셔서 감사합니다.

여러분은 이미 대부분의 사람보다 앞서 있습니다.
Claude Code를 '알게 된 것'만으로도 상위 5%입니다.
'실제로 쓰기 시작한 순간' 상위 1%가 됩니다.

에이나우에서 함께 성장합시다.

— 신영선, 에이나우 대표

Claude Code 실전 워크북: 4주 미션

이 책을 읽고 나서, 4주간 따라하면 되는 주간 미션입니다.

Week 1: 기초 다지기

Day	미션	예상 시간
1	Claude Code 설치 + 첫 대화	30분
2	CLAUDE.md 5줄 생성	10분
3	자기소개 웹페이지 만들기	30분
4	웹페이지 수정 3가지 요청	20분
5	에러 발생 → 해결 경험하기	30분
6	/clear, /compact 연습	10분
7	CLAUDE.md에 교훈 3개 추가	15분

Week 2: 활용 확장

Day	미션	예상 시간
8	Plan Mode로 프로젝트 설계	30분
9	설계대로 구현 시작	1시간
10	기능 추가 + 수정	1시간
11	Git 초기화 + 커밋	15분
12	Vercel 또는 GitHub Pages 배포	30분
13	서브에이전트 사용해보기	20분
14	주간 회고 + CLAUDE.md 정리	20분

Week 3: 자동화 시작

Day	미션	예상 시간
15	슬래시 커맨드 1개 생성	20분
16	Boris 팁 3개 적용	30분
17	병렬 세션 (2개) 시도	1시간
18	Everything CC 보안 스캔	10분
19	비용 최적화 설정 적용	15분
20	두 번째 프로젝트 시작	1시간
21	주간 회고 + CLAUDE.md 정리	20분

Week 4: 프로 레벨 진입

Day	미션	예상 시간
22	Hook 1개 설정 (알림 또는 포맷팅)	20분
23	커스텀 스킬 1개 제작	30분
24	두 번째 프로젝트 완성 + 배포	1시간
25	콘텐츠 리퍼포징 시도	30분
26	자동화 파이프라인 1개 구축	1시간
27	팀 또는 동료에게 데모	30분
28	월간 회고 + 다음 달 계획	30분

4주 완주 결과

✅ 프로젝트 2개 완성 + 배포
✅ CLAUDE.md 교훈 20개+ 축적
✅ 슬래시 커맨드, Hook, 스킬 모두 경험
✅ 비용 최적화 설정 완료

이것이 Level 3 '실전' 레벨입니다. 여기서부터는 스스로 성장할 수 있습니다.

자주 묻는 질문: 심화편

Q. Claude Code와 claude.ai 채팅은 뭐가 달라요?

claude.ai 채팅은 대화만 가능합니다. Claude Code는 대화 + **파일 생성/수정/삭제 + 명령어 실행**이 가능합니다. 실제로 뭔가를 만들고 싶으면 Claude Code가 필요합니다.

Q. Claude Code로 모바일 앱도 만들 수 있나요?

네, React Native나 Flutter를 이용하면 가능합니다. 먼저 웹앱(PWA)으로 만들고, 사용자가 늘면 네이티브 앱으로 전환하는 전략을 추천합니다.

Q. Claude Code로 만든 것의 저작권은?

여러분에게 있습니다. 상업적 사용도 자유입니다. 다만 Claude가 참고한 오픈소스 라이선스는 확인이 필요합니다.

Q. 보안이 걱정돼요. 내 코드가 유출되지 않나요?

Anthropic은 사용자 데이터를 모델 학습에 사용하지 않습니다. 민감한 기업 데이터는 Enterprise 플랜을 고려하세요.

Q. 오프라인에서도 쓸 수 있나요?

아니요. 인터넷 연결이 필수입니다.

Q. 팀 전체가 같은 CLAUDE.md를 써야 하나요?

프로젝트 CLAUDE.md는 Git에 커밋해 공유하고, 개인 글로벌 CLAUDE.md는 각자 유지합니다.

Q. 어떤 프로그래밍 언어를 지원하나요?

사실상 모든 언어. 가장 잘 되는 건 JavaScript/TypeScript와 Python입니다.

Q. API 사용과 Pro 구독 중 뭐가 좋아요?

시작은 Pro($20/월). 사용량이 많아지면 API(종량제)가 가성비 좋아집니다.

Q. Claude Code가 만든 코드를 믿어도 되나요?

대부분 좋지만 반드시 직접 확인하세요. 보안/결제/개인정보 코드는 전문가 리뷰 필수.

Q. 학생/교육용으로 쓸 수 있나요?

네. Pro 플랜이면 학습에 충분합니다.

Claude Code 생태계 전체 지도

에이나우 리소스

리소스	내용	접근
이 책 (v1.0)	Claude Code 실전 마스터	지금 읽고 있는 것
유튜브	영상 가이드	신영선의 AI탐구
디스코드	실시간 Q&A	discord.gg/ainow
부트캠프	60일 집중 과정	ainow.co.kr

ℹ 알아두기

이 책의 업데이트: Claude Code는 빠르게 발전합니다. 분기별 업데이트는 에이나우 디스코드에서 안내합니다.

독자 노트: 나만의 학습 기록

내 CLAUDE.md 교훈 기록

날짜	교훈

내 프로젝트 아이디어

#	아이디어	우선순위	상태
1			
2			
3			
4			
5			

이번 달 목표

주	목표	달성
1주차		☐
2주차		☐
3주차		☐
4주차		☐

> **독자 노트: 프롬프트 & 커맨드 기록**

효과적이었던 프롬프트

#	프롬프트	용도
1		
2		
3		
4		
5		

내 슬래시 커맨드 목록

커맨드	설명	생성일

메모

자유롭게 메모하세요...

> **실전 사례 모음: 에이나우 부트캠프 수강생 프로젝트**

에이나우 60일 AI 생산자 부트캠프 1기 수강생들이 실제로 Claude Code로 만든 프로젝트를 소개합니다.

프로젝트 1: 네일 샵 예약 + 포트폴리오 사이트

제작자: 네일 아티스트 (코딩 경험 0)

제작 기간: 5일 (하루 1~2시간)

네일 디자인 포트폴리오를 보여주고, 카카오톡 상담 예약까지 연결되는 원페이지 사이트입니다. 기존에는 인스타그램 DM으로만 예약을 받았는데, 사이트를 통해 문의가 40% 증가했습니다.

```
# 핵심 프롬프트
> 네일 아트 포트폴리오 + 예약 사이트를 만들어줘.
  갤러리: 사진 12장 (3열 그리드)
  사진 클릭하면 크게 보이는 라이트박스
  하단: 카카오톡 상담 버튼 + 전화 버튼
  분위기: 핑크 + 골드, 고급스럽고 여성스러운 느낌
```

프로젝트 2: 부동산 매물 카드 자동 생성기

제작자: 부동산 중개보조원 (코딩 경험 0)

제작 기간: 3일

매물 정보를 입력하면 인스타그램에 올릴 수 있는 매물 카드 이미지(HTML)를 자동 생성합니다. 기존에 Canva로 30분 걸리던 작업이 2분으로 단축됐습니다.

```
# 슬래시 커맨드로 자동화
> /property 역삼동 34평 아파트 15억 2024년 리모델링
```

프로젝트 3: 학원 숙제 관리 + 학부모 알림

제작자: 영어 학원 원장 (기본 컴퓨터 사용 가능)

제작 기간: 1주일

Google Sheets에 학생별 숙제 완료 여부를 입력하면, 미완료 학생의 학부모에게 카카오톡 알림톡을 보내는 시스템입니다.

> 💡 **꿀팁**
> 부트캠프 수강생들의 공통점: 모두 **자기 업무의 반복적 고통을 정확히 알고 있었습니다.** Claude Code는 그 고통을 해결하는 도구일 뿐, 핵심은 '어떤 문제를 풀 것인가'를 아는 것입니다.

프로젝트 4: 카페 메뉴보드 디지털 사이니지

제작자: 카페 사장님 (코딩 경험 0)

제작 기간: 2일

태블릿에서 풀스크린으로 보여주는 디지털 메뉴보드입니다. Google Sheets에서 메뉴와 가격을 수정하면 실시간으로

반영됩니다. 기존 아크릴 메뉴판 교체 비용(월 5~10만 원) 절감.

```
> 카페 디지털 메뉴보드를 만들어줘.
  태블릿 풀스크린용 (1024x768)
  카테고리: 커피, 음료, 디저트, 시즌 메뉴
  데이터: Google Sheets에서 실시간 로드
  디자인: 따뜻한 브라운 톤, 깔끔한 산세리프체
  시즌 메뉴는 "NEW" 뱃지 자동 표시
```

프로젝트 5: 요가 강사 온라인 클래스 예약 시스템

제작자: 요가 강사 (코딩 경험 0)

제작 기간: 1주일

주간 수업 시간표를 보여주고, 수강생이 직접 예약/취소할 수 있는 웹앱입니다. 수강생 30명의 출석/결제 관리까지 포함. Supabase로 데이터 관리.

프로젝트 6: 인테리어 업체 시공 사례 갤러리

제작자: 인테리어 업체 대표 (기본 컴퓨터 사용)

제작 기간: 4일

시공 전/후 사진을 슬라이더로 비교하는 갤러리 사이트입니다. 카테고리별 필터(주방, 거실, 욕실 등)와 견적 문의 폼 포함. 블로그 SEO로 월 유입 200% 증가.

부트캠프 성과 요약

1기 30명 중 28명이 실제 배포 가능한 프로젝트를 완성했습니다.
코딩 경험 0인 수강생이 67%. 평균 제작 기간 5일.

핵심은 '코딩 실력'이 아니라 '문제 정의 능력'이었습니다.

Claude Code로 수익화하는 5가지 모델

Claude Code를 배워서 실제로 수익을 만드는 방법입니다.

1. **프리랜서 웹 개발**
   랜딩페이지, 포트폴리오 사이트, 소규모 웹앱을 제작해서 판매합니다. Claude Code 덕분에 개발자가 아니어도 건당 50~200만 원의 프로젝트를 수주할 수 있습니다. 크몽, 숨고, 클래스101에서 시작하세요.
2. **SaaS 제품 출시**
   반복적인 문제를 해결하는 웹 서비스를 만들어 월 구독료를 받습니다. Part 5의 MVP 프로세스를 따르면 3일 만에 첫 버전을 출시할 수 있습니다. 월 5만 원 × 100명 = 월 500만 원.

3 자동화 컨설팅
기업의 반복 업무를 분석하고, Claude Code + n8n으로 자동화 시스템을 구축해줍니다. 건당 300~1,000만 원. 마케팅 에이전시, 병원, 학원, 쇼핑몰이 주요 고객입니다.

4 콘텐츠/교육
Claude Code 사용법을 가르치는 강의, 전자책, 유튜브 채널을 운영합니다. 에이나우처럼 커뮤니티 기반으로 확장하면 지속적인 수익이 가능합니다.

5 템플릿/스킬 판매
업종별 CLAUDE.md 템플릿, 슬래시 커맨드, 스킬을 패키지로 판매합니다. Gumroad, 네이버 스마트스토어에서 디지털 상품으로 판매 가능. 건당 1~5만 원 × 무한 판매.

수익화 로드맵

단계	기간	행동	예상 수익
1	1~2개월	Claude Code 학습 + 포트폴리오 2~3개	0원 (투자 기간)
2	2~3개월	크몽/숨고에서 첫 프로젝트 수주	50~100만 원/월
3	4~6개월	자동화 컨설팅 or SaaS 출시	200~500만 원/월
4	6~12개월	교육/커뮤니티 확장	500만 원~/월

⚠ 주의
수익화는 Claude Code '실력'보다 **'영업 + 마케팅'**이 더 중요합니다. 기술은 Claude가 해결합니다. 여러분이 해야 할 일은 고객을 찾고, 문제를 파악하고, 가치를 전달하는 것입니다.

Claude Code vs 외주 개발: 비용 비교

직접 Claude Code를 사용하는 것과 외주를 맡기는 것의 현실적인 비교입니다.

프로젝트	외주 개발 비용	Claude Code 비용	절감율
랜딩페이지 1개	50~150만 원	Pro 구독 $20 + 1시간	95% ↑
포트폴리오 사이트	100~300만 원	Pro 구독 $20 + 3시간	97% ↑
블로그 시스템	200~500만 원	Pro 구독 $20 + 1일	98% ↑
간단한 웹앱 (CRM 등)	500~1,500만 원	Max 구독 $100 + 3일	95% ↑
쇼핑몰 상세페이지 10개	50~100만 원	Pro 구독 $20 + 2시간	97% ↑
카드뉴스 월 20개	월 40~80만 원	Pro 구독 $20 + 월 2시간	96% ↑

ℹ 알아두기

외주가 여전히 필요한 경우:

- 대규모 서비스 (수만 명 동시 접속)
- 결제/금융 시스템 (보안 인증 필요)
- 모바일 네이티브 앱 (앱스토어 심사)
- 기존 레거시 시스템 연동

이 외의 대부분의 '처음 만드는 것'은 Claude Code로 충분합니다.

크리에이터를 위한 콘텐츠 제작 속도 비교

콘텐츠	수동 제작	Claude Code	절감
블로그 글 (2,000자)	2~3시간	15분	90%↑
유튜브 대본 (10분)	3~4시간	30분	85%↑
인스타 캐러셀 10장	1~2시간	10분	90%↑
이메일 뉴스레터	1시간	10분	83%↑
카드뉴스 텍스트	30분	5분	83%↑
SEO 메타 설명	15분	1분	93%↑

시간 = 돈

주 5개 블로그 글을 쓴다면:

수동: 5 × 2.5시간 = 12.5시간/주

Claude Code: 5 × 15분 = 1.25시간/주

절약되는 **11시간**을 전략, 영업, 또는 새 프로젝트에 투자하세요.

에이나우의 AI 생산자 선언문

> *AI를 소비하는 시대는 끝났다.*
> *AI로 만드는 시대가 왔다.*
>
> *우리는 질문하는 사람이 아니라 만드는 사람이다.*
> *우리는 코드를 읽는 사람이 아니라 문제를 읽는 사람이다.*
> *우리는 도구에 의존하는 사람이 아니라 도구를 쓰는 사람이다.*
>
> *에이나우는 AI 생산자를 키운다.*
> *이 책은 그 여정의 첫 챕터다.*

다음 단계: 이 책을 읽은 후

1. **지금 당장**
   터미널을 열고, `mkdir my-project && cd my-project && claude`를 실행하세요. CLAUDE.md 5줄을 만드세요. 오늘 겪은 불편함 하나를 Claude에게 말하세요.
2. **이번 주**
   Part 5의 업종별 프로젝트 중 하나를 완성하세요. Vercel에 배포하세요. 링크를 주변에 공유하세요.
3. **이번 달**
   4주 워크북(부록)을 따라하세요. CLAUDE.md 교훈을 20개 이상 쌓으세요. 슬래시 커맨드 3개를 만드세요.
4. **3개월 후**
   프로젝트 3개 이상 완성. Hook과 MCP를 활용한 자동화 파이프라인 구축. 주변에 Claude Code를 소개할 수 있는 수준.
5. **6개월 후**
   수익화 모델 중 하나를 실행. 에이나우 커뮤니티에서 멘토 역할. AI 생산자로서의 개인 브랜드 구축.

에이나우와 함께 성장하세요

채널	내용
유튜브: 신영선의 AI탐구	매주 Claude Code 실전 영상 업로드
인스타: @ainow.official	AI 팁, 비하인드, 커뮤니티 소식
디스코드	#클로드코드-스터디 채널에서 실시간 Q&A
AI 생산자 부트캠프	60일 집중 과정 (다음 기수 모집 중)
ainow.co.kr	VOD 강의, 템플릿, 리소스

마지막으로

이 책을 끝까지 읽어주셔서 감사합니다.

여러분은 이미 대부분의 사람보다 앞서 있습니다.
Claude Code를 '알게 된 것'만으로도 상위 5%입니다.
'실제로 쓰기 시작한 순간' 상위 1%가 됩니다.

에이나우에서 함께 성장합시다.

— 신영선, 에이나우 대표

독자 워크시트 1: 내 프로젝트 기획서

아래 양식을 채워보세요. Claude Code에게 이대로 말하면 바로 시작할 수 있습니다.

1. 문제 정의

내가 매일 겪는 불편함	
이 문제 때문에 낭비되는 시간	
이상적인 해결 모습	

2. 솔루션 설계

만들고 싶은 것	
필수 기능 (3개 이내)	1. 2. 3.
타겟 사용자	

3. 기술 선택

프론트엔드	☐ 순수 HTML ☐ React ☐ 모르겠음 (Claude에게 맡기기)
데이터 저장	☐ Google Sheets ☐ Supabase ☐ 필요 없음
배포	☐ Vercel ☐ GitHub Pages ☐ 모르겠음

4. 일정

시작일	년 월 일
목표 완성일	년 월 일
배포 예정일	년 월 일

💡 꿀팁

이 기획서를 Claude에게 그대로 보여주세요.

```
> 이런 프로젝트를 만들고 싶어. [기획서 내용 복사] Plan Mode로 구조를 설계해줘.
```

독자 워크시트 2: CLAUDE.md 성장 일지

Week 1

날짜	추가한 교훈	이유

Week 2

날짜	추가한 교훈	이유

Week 3

날짜	추가한 교훈	이유

Week 4

날짜	추가한 교훈	이유

ℹ 알아두기

4주 후 이 일지를 돌아보세요. 20~30개의 교훈이 쌓여 있을 것입니다. 이것이 여러분만의 Claude Code 매뉴얼이 됩니다.

독자 워크시트 3: 나만의 슬래시 커맨드 설계

커맨드 1

이름	/
용도	
입력 ($ARGUMENTS)	
단계	1. 2. 3. 4.
출력 형식	

커맨드 2

이름	/
용도	
입력 ($ARGUMENTS)	
단계	1. 2. 3. 4.
출력 형식	

커맨드 3

이름	/
용도	
입력 ($ARGUMENTS)	
단계	1. 2. 3. 4.
출력 형식	

> 💡 **꿀팁**
> 이 워크시트를 채운 후, Claude Code에서 `> .claude/commands/ 폴더에 이 커맨드를 만들어줘` 라고 하면 바로 생성됩니다.

v1.0 | 2026년 3월 | 신영선 저

© 2026 AINOW. All rights reserved.

ainow.co.kr | 디스코드: #클로드코드-스터디